

Author / Eingereicht von
Malte Roloff
Matriculation number /
Matrikelnummer
K12037645

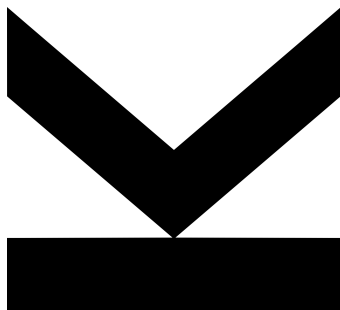
Submission / Angefertigt am
Institute for Machine
Learning

Thesis Supervisor / First
Supervisor / BeurteilerIn /
ErstbeurteilerIn /
ErstbetreuerIn
Prof. Dr. Sepp
Hochreiter

Assistant Thesis Supervisor /
Mitbetreuung
Andreas Radler

Februar 2023

Optimisation of testing strategies for damage mitigation of animal epidemics with neural networks



Master Thesis

to obtain the academic degree of

Master of Science

in the Master's Program

Artificial Intelligence

Kurzfassung

Kurzfassung auf Deutsch.

Abstract

Connections between animal herds, for instance through the movement of animals to other herds or premises (livestock trade), play a major role in the spread of infectious animal diseases. The movements of animals itself between animal premises, animal farms and other livestock holdings form complex livestock trade networks [1]. The theoretical literature is aware of a number of disease spread models on networks and contains basic insights into which network nodes are crucial for the dynamics of spreading. These insights can help to develop testing heuristics such as to test animal farms with higher number of connections in the network more frequently [2].

Based on promising testing heuristics in view of the literature, the project's aim is to find sophisticated test designs or test budget allocations for concrete epidemics. Therefore, we combine deep reinforcement learning (DRL) and graph neural networks (GNNs). A livestock trade network can be represented as a graph consisting of vertices or nodes and edges. GNNs represent a class of deep learning methods designed to perform inference on data that can be described by graphs as such. Proximal policy optimisation (PPO) is a DRL actor-critic (AC) algorithm where both its actor and critic function are neural networks.

By combining GNNs and DRL, we show that the damage of infectious disease outbreaks in animal trade networks can be reduced up to 35% compared to naïve test strategy heuristics. Even if we gradually reduce the total test budget available, the found close-to-optimal test budget allocations prevail against current often applied test design in terms of estimated average number of infected animals at the time of the outbreak detection, i.e. the outbreak size. However, in comparison to "naïve" testing heuristics that are already prominent in literature, the improvements remain small.

Contents

1. Introduction	1
1.1. Background	2
1.2. Literature Review	7
1.3. Thesis Overview	10
2. Data	12
2.1. The German swine trade network	12
2.2. The susceptible-infectious epidemic model with stochastic transmission .	15
2.3. Research questions and hypotheses	19
3. Methods	20
3.1. Introduction to blackbox, derivative-free and derivative-based optimisation methods	20
3.1.1. Derivative-free optimisation	22
3.1.1.1. Evolution strategies	23
3.1.1.2. Metropolis–Hastings and simulated annealing	24
3.1.2. Derivative-based optimisation	30
3.1.2.1. Introduction to neural networks	31
3.1.2.2. Deep reinforcement learning	35
3.2. Graph neural networks	39
3.3. Blackbox optimisation with deep reinforcement learning and graph neural networks	41
3.3.1. The building blocks	41
3.3.1.1. Transmission networks	41
3.3.1.2. Test strategy	42
3.3.1.3. SI model	43
3.3.1.4. Single step proximal policy optimisation	44
3.3.1.5. MetaLayer	51
3.3.2. All parts come together	54
3.3.2.1. The set-up	55
3.3.2.2. The optimisation process	56
3.3.2.3. Proximal policy optimisation parameters	59

Contents

4. Results	61
4.0.1. “Naïve” heuristics	61
4.0.2. Deep reinforcement learning and graph neural networks	62
4.0.3. Evaluation of PPO and SI model parameters, and the performance of MetaLayer	69
4.0.3.1. PPO parameters	69
4.0.3.2. SI model parameters	70
4.0.3.3. GNNs and MLPs	70
5. Discussion	72
5.1. The findings and the curse of dimensionality	72
5.2. The findings in context of related work	73
5.3. The effect of reducing the test budget	73
5.4. Limitations of our approach and future work	74
A. Appendix	86
A.1. The degree distributions of the transmission networks	86
A.2. The learning progress during optimisation	90
A.3. The mean damage for “naive” heuristics for all network types and network sizes	92
A.4. The mean damage for “naive” test strategies in comparison to optimal strategies	102

List of Figures

2.1.	The main structure of the German swine trade network based on transmission data from the HI-TIER dataset from 2011 to 2014. Live animal holdings with a degree below 3 are excluded.	13
2.2.	First samples of the static network representations with 120 nodes. First sample of AB networks (left), first sample of the SYN networks (bottom), and the first sample of the WX networks (top right). The greater the size of the node and the brighter the color, the higher its degree.	14
3.1.	A visualisation of a good and a bad surrogate function. Taken from Audet et al. [12], Figure 13.1.	22
3.2.	Each box-plot shows the minimum, first quartile, median, third quartile, and maximum of the expected outbreak size at the time of detection in an ensemble of 100 synthetic networks with 1040 nodes and 13500 edges. The expected outbreak size for each transmission network is estimated by running 10,000 SI simulations and summing up the capacities of nodes infected at the time of detection. The red marker shows the mean for each test strategy and number of sentinels for the ensemble of 100 synthetic networks. Taken from Ansari et al. [30], Figure 5.	28
3.3.	The differences in performance of the “naïve” testing strategies compared to the optimal testing strategies found by MCMC and SA on the ensemble of 10 real swine-trade networks with 57,590 nodes and 810,000 edges. Each point represents the estimated expected number of infected animals at the time of detection for one real network, whereas the different colors show the influence of the number of nodes selected as sentinel nodes ($k = N/12, N/26$ or $N/52$). Selecting only $1/52$ of nodes and testing them weekly performed considerably better than, e.g., selecting $1/26$ of nodes and testing them biweekly independent of network type and network size. MCMC and SA reduce the estimated average number of infected animals in comparison to the “naïve” testing strategies the most. The standard error in the averaging of infected cases over all 1,000 SI runs and over all 10 networks is on average 500 for heuristics and 1,200 for MCMC and SA. Taken from Ansari et al. [30].	29

List of Figures

3.4.	Sentinel points found by MCMC on a synthetic network $n = 120$, each color shows one community in the network and the smiling faces mark the optimal set of sentinels as selected by the algorithm. Most community centers are chosen as sentinels. The other sentinels chosen form bridges between communities. These nodes are linked to the centers of three or four communities. It turns out they are all fattening farms, i.e., in the middle of the pork production chain. Taken from Ansari et al. [30], Figure 3.	30
3.5.	An abstract neuron. Taken from Rojas [81], Figure 1.14.	32
3.6.	Functional model of an example artificial neural network. Taken from Rojas [81], Figure 1.15.	33
3.7.	Neural network as a blackbox. Taken from Rojas [81], Figure 2.1.	33
3.8.	Agent and environment. Taken from Dong et al. [18], Figure 2.1.	37
3.9.	Undirected graph (left) and directed graph (right) with 7 nodes and 11 edges in non-Euclidean space.	39
3.10.	The general design of a graph neural network. The input data is graph structured data. The GNN consists of one or more GNN layer, where the output is an embedding and the loss function depends on the specific task. Adapted from Zhou et al. [25], Fig. 2.	41
3.11.	First column: AB networks. Second column: SYN networks. Third column: WX networks. All networks consist of 120 nodes. The larger the size of the node and the brighter the node color, the higher its degree. The SYN network lies in between the AB network and the WX network in terms of degree distribution and centrality.	42
3.12.	The actor and critic networks.	46
3.13.	The actor-critic architecture. In Sutton and Barto [18], Fig. 3.5.	47
3.14.	A full GN block predicts node, edge, and global output attributes based on incoming node, edge, and global attributes. It follows the <i>update</i> and <i>aggregation</i> process that Battaglia et al. [24] introduce. The neural networks are multilayer perceptrons with different layer sizes. Adapted from Battaglia et al. [24], Fig. 4a.	54
3.15.	The optimisation process for a single transmission network.	59
4.1.	The mean entropy measure, mean pearson correlation coefficient and the corresponding mean p-value over all WX networks (top) and SYN networks (bottom) with 1040 nodes during the optimisation process. While the effective number of nodes that receive budget shares decreases over time, the correlation between the nodes' assigned budget shares and their numbers of incoming transmissions increases.	65

List of Figures

4.2.	The first test budget allocation selected by the agent (left) and the test budget allocation selected in the last episode (right) for the first samples of the transmission networks ensemble with 120 nodes. The first row shows the first AB network, the second row shows the first WX network, and the first SYN network is shown in the last row.	66
4.3.	The optimised budget allocations for the first sample from the transmission networks ensembles with 120 nodes. From left to right the budgets are $B = N$, $B = N/2$, and $B = N/4$. The node color represents the node's budget share received, whereas the larger the node size the greater its degree. The first AB network (top), the first WX network (middle), and the first SYN network (bottom).	67
4.4.	The performance of "naïve" testing strategies for varying inconspicuous times for one SYN transmission network with 1040 nodes. The results for AB and WX networks are similar.	70
4.5.	The performance of GNNs (MetaLayer) as both actor and critic in comparison to MLPs. Top: The mean damage for a SYN network with 120 nodes. Bottom: The mean damage for a SYN network with 1040 nodes. We apply MLPs with one hidden layer that has twice the dimension of the input layer for both actor and critic.	71
A.1.	The degree distribution of (A) real pig trade network with $n = 57,590$, (B) one realization of a synthetic data with $n = 57,590$, and (C) ensemble of 10 synthetically generated networks with the same size from the model. In Ansari et al. [1], Figure 8.	86
A.2.	The degree distribution over 100 AB networks with 120 (left) and over 100 AB networks with 1040 nodes (right).	87
A.3.	The degree distribution over 100 SYN networks with 120 (left) and over 100 SYN networks with 1040 (right) nodes.	87
A.4.	The degree distribution of 100 WX networks with 120 (left) and 100 WX networks with 1040 (right) nodes.	88
A.5.	Pair plots including data from all generated networks showing the differences in the distribution of degree, the capacities and the number of total transmissions. Top: All networks with 120 nodes. Bottom: All networks with 1040 nodes.	89
A.6.	The change in the estimated average number of infected animals over time over 100 BA networks with 120 nodes and 1040 nodes.	90
A.7.	The change in the estimated average number of infected animals over time over 100 WX networks with 120 nodes and 1040 nodes.	91
A.8.	The change in the estimated average number of infected animals over time over 100 SYN networks with 120 nodes and 1040 nodes.	92

List of Figures

A.9. The mean damage for the “naive” test strategies over all AB transmission networks with 120 nodes (right) and 1040 (left) nodes if $S_{B=N}$. The test strategies $HD_{S_{B=N}}$ and $HIT_{S_{B=N}}$ are more effective than the test strategies $UNI_{S_{B=N}}$ and $HC_{S_{B=N}}$	93
A.10. The mean damage for the “naive” test strategies over all WX transmission networks with 120 nodes (right) and 1040 nodes (left) if $S_{B=N}$. The test strategies $HD_{S_{B=N}}$ and $HIT_{S_{B=N}}$ are more effective compared to the test strategies $UNI_{S_{B=N}}$ and $HC_{S_{B=N}}$	94
A.11. The mean damage for the “naive” test strategies over all SYN transmission networks with 120 nodes (right) and 1040 nodes (left) if $S_{B=N}$. The test strategies $HD_{S_{B=N}}$, $HIT_{S_{B=N}}$ and $HC_{S_{B=N}}$ are more effective compared to the test strategy $UNI_{S_{B=N}}$	95
A.12. The mean damage for the “naive” test strategies over all AB transmission networks with 120 nodes (right) and 1040 nodes (left) if $S_{B=N/2}$. The test strategies $HD_{S_{B=N/2}}$ and $HIT_{S_{B=N/2}}$ are more effective compared to the test strategy $UNI_{S_{B=N/2}}$ and $HC_{S_{B=N/2}}$. However, $HD_{S_{B=N/2}}$ and $HIT_{S_{B=N/2}}$ perform worse than $UNI_{S_{B=N}}$	96
A.13. The mean damage for the “naive” test strategies over all WX transmission networks with 120 nodes (right) and 1040 nodes (left) if $S_{B=N/2}$. The test strategies $HD_{S_{B=N/2}}$ and $HIT_{S_{B=N/2}}$ are more effective compared to the test strategy $UNI_{S_{B=N/2}}$ and $HC_{S_{B=N/2}}$. However, $HD_{S_{B=N/2}}$ and $HIT_{S_{B=N/2}}$ perform worse than $UNI_{S_{B=N}}$	97
A.14. The mean damage for the “naive” test strategies over all SYN transmission networks with 120 nodes (right) and 1040 nodes (left) if $S_{B=N/2}$. The test strategies $HD_{S_{B=N/2}}$ and $HIT_{S_{B=N/2}}$ are more effective compared to the test strategy $UNI_{S_{B=N}}$, $UNI_{S_{B=N/2}}$ and $HC_{S_{B=N/2}}$	98
A.15. The mean damage for the “naive” test strategies over all AB transmission networks with 120 nodes (right) and 1040 nodes (left) if $S_{B=N/4}$. The test strategies $HD_{S_{B=N/4}}$ and $HIT_{S_{B=N/4}}$ are more effective compared to the test strategy $UNI_{S_{B=N/4}}$, $UNI_{S_{B=N/2}}$ and $HC_{S_{B=N/4}}$. However, $HD_{S_{B=N/4}}$ and $HIT_{S_{B=N/4}}$ perform worse than $UNI_{S_{B=N}}$	99
A.16. The mean damage for the “naive” test strategies over all WX transmission networks with 120 nodes (right) and 1040 nodes (left) if $S_{B=N/4}$. The test strategies $HD_{S_{B=N/4}}$ and $HIT_{S_{B=N/4}}$ are more effective compared to the test strategy $UNI_{S_{B=N/4}}$, $UNI_{S_{B=N/2}}$ and $HC_{S_{B=N/4}}$. However, $HD_{S_{B=N/4}}$ and $HIT_{S_{B=N/4}}$ perform worse than $UNI_{S_{B=N}}$	100
A.17. The mean damage for the “naive” test strategies over all SYN transmission networks with 120 nodes (right) and 1040 nodes (left) if $S_{B=N/4}$. The test strategies $HD_{S_{B=N/4}}$ and $HIT_{S_{B=N/4}}$ are more effective compared to the test strategies $UNI_{S_{B=N/4}}$, $UNI_{S_{B=N/2}}$, $UNI_{S_{B=N}}$ and $HC_{S_{B=N/4}}$	101

List of Figures

A.18.The mean damage for “naive” test strategies and optimal test strategies with three different total test budgets over all AB transmission networks with 120 nodes.	102
A.19.The mean damage for “naive” test strategies and optimal test strategies with three different total test budgets over all WX transmission networks with 120 nodes.	103
A.20.The mean damage for the “naive” test strategies and the optimal strategies with three different total test budgets over all SYN transmission networks with 120 nodes.	104

List of Tables

- 4.1. Performances in terms of the mean damage of optimised strategies $S_{B=N}^*$, $S_{B=N/2}^*$, and $S_{B=N/4}^*$ in comparison to the baselines for all network types and network sizes and the respective standard deviation in parentheses. . 63

1. Introduction

In this thesis, we show that blackbox optimisation (BBO) with deep reinforcement learning (DRL) and graph neural networks (GNN) can be a valuable alternative to other algorithms such as Markov Chain Monte Carlo (MCMC), simulated annealing (SA), evolutionary algorithms (EA) and Gaussian Processes (GP). As a use case, we analyse spreading dynamics of an infectious zoonotic disease among animal farms in the German swine trade network. Live animal trade can be represented by a network structure, whereby the holdings are nodes and animal movements between holdings are links or arcs when a direction is given. The animal movements along the connections between nodes can represent the possible transmission paths for the diffusion of a disease [3].

We synthetically generate temporal animal trade networks deduced from real swine trade data from the German swine trade network using the model from [1]. Our goal is to find an optimal allocation of a limited budget that is distributed between all animal holdings in the network to test for diseases. A close-to-optimal test budget or test strategy should reduce the expected damage of an epidemic outbreak to a minimum by detecting it as early a stage as possible. As a measurement of the expected damage of an outbreak for a specific test strategy, we focus on the estimated average number of infected animals predicted by a susceptible-infectious (SI) epidemic model. It models the early stages of epidemic dynamics that we are interested in.

By applying DRL and GNN to find a close-to-optimal test strategy, we are able to reduce the estimated average number of infected animals up to 35% compared to conventional test strategies such as distributing the test budget uniformly among all farms. Also, we find that by reducing the budget even further, but distributing it more effectively among network participants, it is still possible to achieve similar results to common test strategies based on a large budget. Ultimately, we find vulnerable entities that are particularly exposed to transmitted infections. For instance, significant improvements were made by concentrating the test budget on network participants with many incoming live animal

1. Introduction

transmissions. However, the number of infected animals may be reduced even further, by allocating a small part of the test budget to network participants with only a few links between large but only loosely connected groups of participants. Finding an optimal balance point of mitigating the damage of an epidemic outbreak and reasonable economic costs spent on test strategies may be very helpful for future public health policymakers.

The next chapter presents the scientific background for our research project, and in the following we elaborate on literature and the outline of the thesis. Subsequently, we briefly discuss the susceptible-infectious (SI) epidemic model and the resulting research questions and hypotheses surrounding the thesis.

1.1. Background

In recent years, the sociological and economical costs of large-scale outbreaks of infectious diseases like the swine fever epidemics in the 1990s and 2000s and recently the SARS-CoV-2 virus in 2019 have not only led to increased interest in epidemic research, but also to worldwide changes in public health management [1] [4] [5]. An estimated 60% of micro-organisms transmitted from vertebrate animals (including livestock) to humans are human pathogens. However, the precise disease transmission dynamics between vertebrate animals and humans are still hardly understood [4]. Zoonotic diseases occurred as a result of agriculture and the domestication of animals through which humans started co-living with other vertebrate animals in close contact [6]. In many countries it is still very common for humans to share living spaces with livestock for dairy and meat production. Thus, it is often impossible to distinguish disease transmission pathways since people are in close contact with animals privately as well as occupationally [6, 7].

Most epidemic outbreaks such as HIV/AIDS, SARS, Ebola or pandemic influenza originated from animals and are a consequence of ecological, behavioural, or socioeconomic changes of people, animals and their respective surrounding [4]. One major ecological threat for emerging zoonotic diseases is the progressing tropical deforestation that many countries are facing today. It does not only often disturb and displace human populations, mostly indigenous that live in or close to the forest, but also leave them exposed to disease reservoirs [7]. The risk of infectious disease outbreaks is suggested to be higher in rapidly

1. Introduction

deforested tropical regions and up to 50% of zoonotic (human) diseases are likely to be associated with agricultural drivers [8].

Ecological factors that facilitate emergence/outbreaks of zoonotic (human) diseases are predicted to increase in the coming years (intense livestock farming, deforestation, reduce antibiotic resistances). Hence, an early detection of epidemic outbreaks is crucial to reduce their sociological and economical impact in the future [9]. To model the very complex epidemic dynamics, already in the 1920s McKendrick and Kermack [10] introduced a deterministic compartmental model to describe infectious disease spreading behaviour in a population. Many epidemic models have been deployed ever since, most of them going back to the original version [9]. The simplest standard epidemic model (susceptible-infectious or SI) divides a group into compartments of individuals that are infected and not infected. To also account for recovered or vaccinated individuals, more sophisticated compartment models exist today such as susceptible-infectious-susceptible (SIS), susceptible-infectious-recovered (SIR), or susceptible-exposed-infectious-recovered (SEIR).

One drawback of such simulation-based models is that they can not easily be (globally) optimised for. The term blackbox optimisation (BBO) has been used to describe the optimisation process/problem of models for which the evaluation of a simulation is usually computationally costly and the objective and constraint functions often remain unknown, hidden, unreliable or inaccessible [11, 12]. To address this problem, derivative-free and derivative-based methods for BBO exist. The former do not require the availability of derivatives, whereas the latter first approximate the gradient and then enforce line-search methods. They can be used to construct efficient global optimisation algorithms by fitting response surfaces to data points collected by evaluating the objective and constraint functions at a few points (surrogate-based) [13]. As computing power increased drastically in the last decades, many optimisation algorithms have made significant advances [14]. Due to this, neural networks have risen in popularity among BBO methods to which some refer to as the 'Reinforcement Learning renaissance' [15, 16, 17].

In this work, we utilise prominent Reinforcement Learning algorithms to find close-to-optimal test strategies for an early detection of disease outbreaks in the German swine trade network. Because the total number of tests that can be carried out is limited due to economic and practical reasons, it is feasible to find the most efficient allocation of tests between all participants in the network. A common test strategy applied is to test every

1. Introduction

network participant (here: different types of animal farms in an animal trade network) on average once per year ¹. Depending on the particular test strategy, an epidemic disease outbreak can be either detected early and contained quickly, or rather late with potentially large economic and social costs. This is exacerbated by the approximately exponential growth in the early phase of an outbreak. We estimate the expected size of a disease outbreak by an epidemic SI model with a dynamic spreading process. Since actual spreading dynamics are stochastic and depend strongly on a number of uncertain aspects, most prominently on the time and location of the original infection in the network, we take the average over multiple simulation runs in our optimisation process. By feeding the model a unique test strategy, we let it predict the scope of an epidemic outbreak in terms of the expected number of infected animals at the time the outbreak is detected for this particular test strategy. Our goal is to find characteristics in close-to-optimal test strategies that allow us to make general assumptions about distributing test budgets in an effective manner to mitigate the damage of an epidemic outbreak in the future. However, the objective function of the SI model is hidden and noisy with random estimation errors. We cannot optimise the test strategy directly, hence, we must rely on BBO methods.

Deep reinforcement learning (DRL) algorithms have been very popular and successful in various applications lately, for instance DeepMind’s AlphaZero and AlphaFold, Google Go [18]. They couple deep neural networks (DNN) and reinforcement learning (RL) algorithms to solve challenging decision-making problems [19]. Many of these algorithms can be viewed as derivative-based algorithms in context of BBO. They consider a problem in which a learner or agent makes actions within an environment or state and receives a reward or signal. The DRL algorithm’s task is to progressively update the agent’s behaviour so as to maximize the reward accumulated over time [20]. Essentially, the agent/learner must explore the environment on its own by a trial-and-error search, adapting its behaviour by taking its past experience collected into account [18]. In our case, an agent’s action is a specific test strategy that is evaluated in the environment by Monte–Carlo SI model simulations. The output of these simulations, i.e. the estimated average number of infected animals at the time the outbreak is detected for the agent’s proposed unique test strategy (or action of an agent), is used to set the reward. In a feedback-loop the agent’s goal is to select actions (test strategies) which maximize his reward (here: minimize the expected number of infected animals).

¹The information has been privately communicated at stakeholder dialogues during the project.

1. Introduction

DRL algorithms differ in terms of a range of assumptions about what quantities are represented, how to update the agent’s behaviour (or policy), and hence how that influences its decisions [20]. Here, the main novelty is the application of a single-step proximal policy optimisation (PPO) algorithm with graph neural networks as both its actor and critic. Single-step PPO has already been demonstrated to be successful in a number of open-loop control problems [21, 19, 22], but its comparison against state-of-the-art optimisation techniques such as evolution strategies or MCMC has yet not been researched extensively. In single-step PPO the optimal policy to be learnt is state-independent, and the actor gets only one attempt per episode to find the optimal action [21].

PPO is a so-called actor-critic (AC) approach. Both the actor and the critic are neural networks, whereas the actor determines the next action taken, the critic evaluates it and its output alters the parameters of the actor. We implement the algorithm by incorporating graph neural networks as both its actor and critic, since a livestock trade network can be represented as a graph consisting of vertices or nodes and edges [23, 1]. In our case, vertices or nodes represent different types of animal farms, whereas edges represent animal movements between two vertices/nodes. To utilise this information in our optimisation process, we apply a class of neural networks that are capable to operate on graphs, and thereby structure their computations accordingly [24].

GNNs are a class of deep learning methods designed to perform inference on data that can be described by graphs. They have been developed and explored rather extensively for decades already but have risen in popularity only in recent years [24]. Zhou et al. [25] give a very extensive review of methods and applications by describing most prominent graph neural networks and its applications. A very general GNN framework has been proposed by Battaglia et al. [24] as a joint-work between DeepMind, GoogleBrain and MIT. In reference to that, PyTorch released a so called MetaLayer (ML) module [26]. It allows to build any kind of graph network. We use ML in combination with stable-Baselines3, which is a DRL open-source project by the German Aerospace Center (Deutsches Zentrum für Luft- und Raumfahrt) - Institute of Robotics and Mechatronics (RM) [27].

Our data consists of three different network types with varying properties. Spreading dynamics of diseases are strongly influenced by the topology of a network [3]. Thus, by comparing optimised test strategies for networks with different structural characteristics, we hope to find indicators of network vulnerabilities to spreading behaviors of infectious diseases. So-called Waxman networks (WX) are more decentralised [28], whereas

1. Introduction

Albert–Barabási (AB) networks are more centered [29]. The network topology of the real German swine trade network lies in between both Waxman and Barabási–Albert networks. However, German real swine trade data can not be made publicly available due to legal obligations. Thus, as a third variant, we draw back on synthetically (SYN) created trade network data based on real animal movements in the German swine trade network from 2011–2014 [1]. The real German swine trade network consists of roughly 60,000 relevant animal farms [1]. However, because of computational costs, we limit the size of all networks to dimensions of 120 or 1040 total nodes.

There exist varying types of farms in the German swine trade network. The pork production chain begins in breeding farms, which transport animals at a certain age to traders who transport the animals to fattening farms. Eventually, traders will transfer animals to slaughterhouses. Each farm type holds different specific importance properties, for instance, degree, betweenness, incoming and outgoing transmissions. Degree represents the total number of other farms a farm trades with, whereas incoming and outgoing transmissions summarise the number of live animal transports the specific farm has received or transmitted. Betweenness reflects how much a specific node is in-between others regarding the transmission of animals. Traders operate between different life stages of the animals, controlling most of the flow of animals in the network. Both traders and fattening farms have high values for degree and betweenness, which make them vulnerable for both spreading and receiving diseases [1, 3]. Slaughterhouses or finishing units possess a potential higher risk for introduction of contagious diseases. They have a very high number of incoming transmissions and incoming infection chain centrality values [3]. We exclude them from further analysis, because the live animals are killed upon arrival and testing would only reveal but not prevent an infectious outbreak [30]. In general, due to the hierarchical structure of swine production chains, the swine trade networks appear less structurally vulnerable than ruminant trade networks [3]. However, the spread of a disease in an early stage of the production chain, say in breeding farms, can favour larger outbreaks [3].

By analysing optimised test strategies and the specific nodes that receive budget shares for disease testing purposes, we are able to show that importance properties of farms in the network such as degree and the number of incoming transmissions are, indeed, key drivers of epidemic outbreaks in the swine trade network. Our findings are coherent with what network analysis methods have shown in other animal trade networks with

1. Introduction

similar network topologies [3, 23]. Corresponding to Ansari’s et al. results [1], our machine learning approach leads to a reduction of estimated average numbers of infected animals up to 35% in comparison to prevailing testing strategies at the time of outbreak detection. However, our findings suggest that distributing the test budget not uniformly between sentinel farms may be preferable in some cases. Giving large test budget shares to farms with very high values of degree and number of incoming transmissions, but also some smaller tests budget to farms connecting modular groups, can potentially lead to even fewer numbers of infected animals. Eventually, we also show that it is sufficient to distribute a quarter of a commonly applied test budget by a machine learning algorithm to reduce the estimated average number of infected animals to a level that is expected when using a naïve scatter-gun approach.

The project is funded by the BMBF – Federal Ministry of Education and Research and Öffentliche Gesundheitsdienste (ÖGD) and is led by the Potsdam Institute for Climate Research Impact (PIK)². The thesis incorporates data and important insights from previous work done by Ansari et al. [1, 30, 31].

1.2. Literature Review

In this thesis, we combine an epidemic susceptible-infectious model (SI), graph neural networks (GNNs) and deep reinforcement learning (DRL) to solve an optimisation task. Literature following similar approaches is rare, thus, before we eventually elaborate on the handful of publications that explore combinations of them, we briefly introduce broad research for the individual methods. However, first we give insights from network science regarding our specific application case, the German swine trade network.

Network science and the modeling of epidemic spreads are fundamentally linked to each other and there exist a broad literature researching the complex relationships between networks and epidemic model [2, 23, 32, 33]. In previous studies, particularly techniques from social network analysis (SNA) are utilised by veterinary epidemiologists to model spatial temporal livestock disease dynamics and to identify actors that are key drivers for spreading infectious diseases in networks [23]. SNA approaches that account for temporal

²Project lead by Dr. Jobst Heitzig, Mar 01, 2019 until Oct 31, 2022, Funding Agency BMBF, PT DLR. <https://www.pik-potsdam.de/en/output/projects/all/700>.

1. Introduction

order of movements, like temporal contact chain(s) analyses and disease spreading models, have been successful in capturing real-world epidemic dynamics [34]. The disease spread is highly dependent on the structure of the network, hence the network topology may reveal crucial information regarding the exposition to infections of particular holdings or communities [32]. Although live animal trade networks tend to differ structurally nation-wise, many pork production chains are hierarchically organised. Also, the characteristics of different types of holdings often match each other [35]. Hence, findings by network analyses of live animal trade networks could give us more general insights into the epidemic spreading behaviour in live animal trade networks.

For instance, by modelling the African swine fever virus spread among domestic swine holdings in France, highly connected holdings are identified as potential key spreaders, whereas slaughterhouses that primarily have high numbers of incoming connections, bear less risks of spreading the disease due to their low number of outgoing connections [35]. Analyses based on trade data of between-farm animal movement among all farms of the Brazilian state show that targeting farms with high degree reduces the number of infected farms drastically [36]. Results obtained by incorporating live animal trade transmission data from the German swine trade network into a SIR model suggest that the spreading of diseases is likely influenced by how often existing trade connections are active. Also, small changes of network activity potentially lead to dramatic effects on outbreak probabilities [37]. In general, risk-based surveillance strategies can be optimised to account for highly connected holdings, geographical regions and time periods with greatest number of swine transmissions. It appears crucial to identify trade communities and prevalent large epidemic outbreaks by targeting these in test strategies [35]. A further study [23] focuses on German swine trade data spanning over several years. The data is also utilised in this thesis. It concludes that within the German swine trade network, nodes of small degree tend weakly to connect with nodes of large degree. Implementing control measures and surveillance only to nodes with large degrees could therefore be already sufficient enough to mitigate epidemic damages.

Live animal trade data can also be represented as a temporal network or graph, whereas holdings are represented as nodes and movements or trades between holdings are represented as edges in the graph. By aggregating all swine movements over time, the data forms a static graph [30]. GNNs have recently become more popular as machine learning architectures that are capable of utilising network data. This neural network structure in-

1. Introduction

herently supports combinatorial generalisation and combinatorial optimisation problems [24, 38]. For an extensive introduction to deep learning for graphs see [39]. In terms of combinatorial generalisation, Battaglia et al. [40] show that graph networks can be trained to take one-step physical state predictions, which simulate thousands of future time steps. Hamrick et al. [41] and Wang et al. [42] find that decision-making policies by graph neural networks transfer to novel numbers of entities. A number of real-world problems can be reduced to combinatorial optimisation on a graph, either to find a subset or an ordering of vertices that maximise (minimise) some objective function [43]. Studies [44, 45, 46, 47] have shown the potential of GNNs for generalisation by demonstrating that they perform well on networks with different sizes than they had been trained on and produce useful node embeddings for unseen data [48]. In this thesis, the GNN architecture “MetaLayer” presented by Battaglia et al. [24] is used. It allows for generalisation and “extends various approaches for neural networks that operate on graphs, and provides a straightforward interface for manipulating structured knowledge and producing structured behavior” [24].

For some time past, deep reinforcement learning has been used in various applications that concern sequential decision making in higher dimensional problems. Introduced as game playing algorithms by Arthur Samuel in 1959 [49], DRL has been successfully applied to real-world problem domains, i.e. autonomous driving, power management, flows or traffic light control [20]. Lately, DeepMind [50] introduced an algorithm that is able to play seven different Atari games using raw pixels as input, followed by AlphaGo which defeated the number one GO player Jie Ke in China. Its successors AlphaZero, 2017, and MuZero, 2019, were able to improve the performance even further [51]. With the emergence of the SARS-CoV-2 pandemic in 2019, DRL has found its way into epidemic research. Several studies have been researching its potential for, i.e., intelligent vaccine distribution strategies [52], optimising lockdown policies for epidemic control [53], and for large-scale epidemic control [54]. Bent [9] gives an introduction into utilising machine learning, primarily DRL, for optimal prediction, control and planning by using epidemiological models of disease transmission. In addition, GNNs and DRL have been combined to explore the potential of utilising graph structured data. For instance, Wang et al. [55] propose a transistor sizing method that utilises graph convolutional neural networks and reinforcement learning to learn on the circuit topology representation, where vertices are transistors and edges are wires. It consistently outperforms conventional blackbox optimisation methods such as Bayesian optimisation and evolutionary algorithms. Almasan et al. [56] and Chen et

1. Introduction

al. [57] integrate GNNs into DRL agents to optimise routing in optical networks. Their results show that by combining both methods, they are able to outperform state-of-the-art solutions in topologies never seen during the training process. In the past, neural networks have been applied mostly as surrogates for the actual objective function and have been disregarded for optimisation tasks. However, by modifying classical DRL policy gradient algorithms, they are increasingly used as blackbox optimisers [19]. During our project, it emerged that we might be dealing with a special optimisation case, where the optimal policy learnt by a neural network does not need to depend on any form of “state” other than the given network topology. Although such cases have not been researched extensively, studies in computational fluid mechanics (CFD) context, including shape optimisation [58], drag reduction [19] and conjugate heat transfer control [59] highlight its potential.

This thesis is part of an ongoing project where previous work has been done by Ansari et al. [30]. Here, the focus lies on the design of disease testing strategies to detect zoonotic disease outbreaks as early as possible, based on Markov chain Monte Carlo (MCMC) Metropolis–Hastings algorithm and Simulated Annealing (SA). Their goal is to distribute a fixed total test budget uniformly among animal trade market actors. Therefore, it is determined between how many market actors the budget is split evenly before the optimisation process, e.g 10% of the farms. The algorithm’s task is to find the optimal subset of market actors, so-called sentinel nodes. As data, they utilise synthetically generated networks proposed by Ansari et al. [1], which are also utilised in this thesis. However, additionally to network sizes of 120 and 1040 nodes, they run their algorithm on networks with sizes of 57,590 nodes, which have the same dimensions as the real German swine trade network. By this, they are able to show that optimised strategies reduce the damage of an outbreak considerably compared to naïve testing heuristics. Splitting the total budget among $N/52$ sentinels equally, where N is the size of number of nodes in the network, performed best in that study.

1.3. Thesis Overview

The thesis consists of five chapters with a number of subsections. This chapter gives a brief overview of the thesis outline. In chapter 2, we discuss the data that is utilised in our work and we explain the parameters surrounding the susceptible-infectious epidemic model

1. Introduction

we apply, followed by presenting research questions and hypotheses. In chapter 3 we introduce the methods we apply. First, we introduce conventional blackbox optimisation methods, and in particular describe Ansari’s et al. [30] MCMC and SA approach to find an optimal set of sentinel nodes utilising the same German swine trade network data we work with. Before we finally introduce deep reinforcement learning as an alternative blackbox optimisation algorithm, we shortly present our trial-and-error search for suitable methods. We also give a more in-depth reasoning why we use graph neural networks as both actor and critic for PPO. In chapter 5 of the thesis, we discuss the results of Ansari’s et al. [30] work. Subsequently, we present our findings in detail. The thesis concludes with main insights we gained by our work with regard to the research questions. We also show shortcomings of our approach and the potential for future work in this field.

2. Data

In this chapter, we present the network trade data and the SI model we utilise in the thesis. Eventually, we present research questions and hypotheses.

2.1. The German swine trade network

For live animal network data, we fall back on Ansari's et al. preliminary works [1, 30]. They extract trade data from January 1, 2011 to December 31, 2014 from HI-TIER, a comprehensive database that tracks swine movements in the German swine trade network since 2006. The trade data consists of a set of animal transmissions that include personal information about the source and the target holdings involved in trades, the batch size, i.e. the number of animals moved in trades, as well as the dates of trades. The dataset contains more than 6.2 million single trades between 57,590 animal holdings. A potential static network representation of the German swine trade network is shown in 2.1.

However, German real swine trade data can not be made publicly available due to legal obligations [30]. Therefore, Ansari et al. [1] create transmission network data based on the German swine trade data they extract from HI-TIER. They generate 100 networks with 120 nodes, as well as 100 networks with 1040 nodes, and additionally, 10 networks with 57,590 nodes. The degree distribution in appendix A.1 shows the strong similarity between the real swine trade network and the synthetic networks with 57,590 nodes.

2. Data

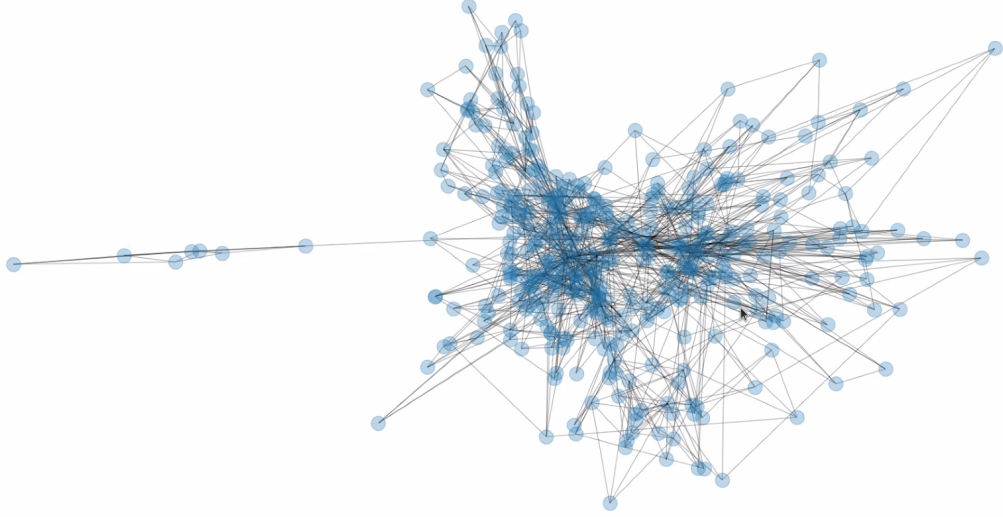


Figure 2.1.: The main structure of the German swine trade network based on transmission data from the HI-TIER dataset from 2011 to 2014. Live animal holdings with a degree below 3 are excluded.

In this work, we exclude synthetically generated networks with 57,590 nodes due to the high computational costs. Instead, we also generate so-called Waxman networks (WX) that are more modular structured [28], and Albert–Barabási (AB) networks that are more centered [29]. To make the analyses more robust, we create 100 network samples for both network types and for both network sizes, i.e. 120 and 1040 nodes.

The first samples of the static network representations of the SYN, WX and AB networks with 120 nodes that visualize the difference in their topological structure are shown in section 2.2.

2. Data

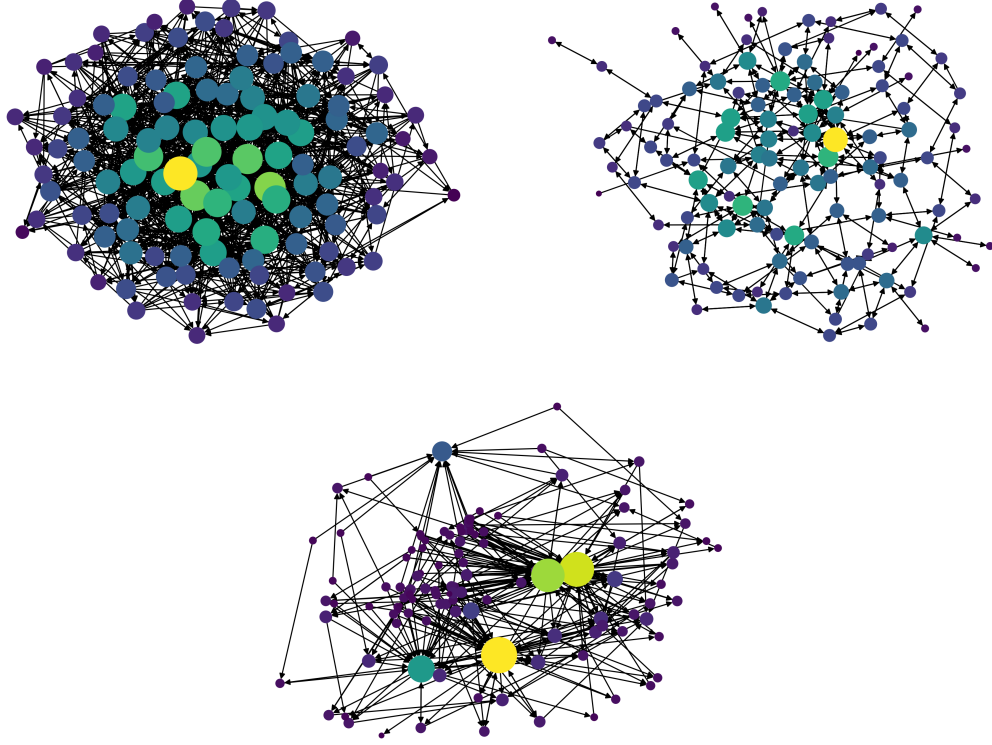


Figure 2.2.: First samples of the static network representations with 120 nodes. First sample of AB networks (left), first sample of the SYN networks (bottom), and the first sample of the WX networks (top right). The greater the size of the node and the brighter the color, the higher its degree.

The network topology of the real German swine trade network lies in between AB, WX and SYN networks in terms of degree distribution and level of centralization vs. modularization. For a more detailed overview of the degree distributions see appendix A.1. By utilising all three network types, we are able to analyse optimal test strategies with regard to the specific network topological features.

2.2. The susceptible-infectious epidemic model with stochastic transmission

Mathematical models have been used to evaluate the success of control and prevention strategies for epidemic diseases since the beginning of the twentieth century. Such diseases, which can be viruses, bacteria, protozoa or toxins, are infectious if they can be transmitted from one host to another. Transmission can happen by direct physical contact, aerosols, water or food, as well as through disease vectors and by birth. Epidemic models aim to describe the transmission process of the disease [60]. In the first epidemic models that were published, a discrete time epidemic model was proposed for the transmission of measles in 1906 by W.H. Hamer [61]. In addition, a similar model showing that malaria is a consequence of the bite of a mosquito was published by R. Ross in 1911 [62]. The popular works on epidemic modeling by Kermack and McKendrick were published between 1927 and 1939 [10]. They introduced a susceptible-infectious-recovered (SIR) model, in which a fixed population and instantaneous incubation period of the infectious agent are assumed. Additionally, the population is completely homogeneous with no age, spatial, or social structure. To date, more accurate versions referring to the original models have been proposed.

In this work, we utilise a network-based susceptible-infectious (SI) epidemic model with stochastic transmission to estimate the outbreak size of an infectious disease in a live animal trade network similar to the German swine trade network. In the SI model the population is divided into two groups, the susceptible individuals, S , who are yet not infected, and the infected individuals, I . We ignore epidemic state transitions such as transitions to death or recovery states, since we are only concerned about early stages of the disease spreading [30]. Generally, in terms of disease outbreaks, an infected individual may remain asymptomatic at the early stage of an infection, only later developing clinical symptoms and becoming an infected case. It also applies that a certain number of cases has to be above a threshold within a period of time for a disease outbreak to occur. To be epidemic, the disease has to spread rapidly to many people and also be endemic, it has to remain in the population over a longer period of time. A disease outbreak becomes pandemic, when the disease spreads on a global scale to many countries and continents as it has been the case with SARS-CoV-2 in 2019 [60].

2. Data

In the SI model we define, the spreading process is dynamic. The live animal trade network first is represented as a transmission network $G = (V, Z)$. It consists of a set V of live animal holdings as the nodes of the network and a set Z of live animal transmissions between holdings as edges ζ . Every edge ζ represents a directed and weighted connection which is established if a transmission of w animals from holding i to holding j on day t happens [30]. The SI model receives the particular transmission network and a test strategy as inputs. The test strategy represents a test budget that is distributed among all holdings or network nodes, which then can be used for performing tests for diseases in a SI simulation run. It is represented by a one dimensional vector v , where each entry v_i represents the share of a test budget the holding or the network node i receives in a particular test strategy.

We assume the outbreak starts at a single, randomly drawn initially infected node at an arbitrary time step. For simplicity, we assume that all live animals of a node are infected as soon as at least one of them becomes infected. This is justified for many diseases which spread rapidly between the animals in a holding. It allows us to abstract from the single-animal level and treat the whole farm as either infected or not infected, thereby reducing the complexity of the simulation vastly. In our model, each neighbour node j of node i has the chance of being infected, if trade between both nodes i and j occurs after the infection time of node i [30]. If the SI model is run, in the beginning all nodes but one are in the susceptible state. The one node not in a susceptible state is initially randomly selected as infected. The disease transmission probability β represents the probability of the infected node transmitting the disease whenever a transmission of animals happens to another node. In our experiments it is set to 0.5¹. Ansari et al. [30] show that a transmission probability β (in their paper its called infectious rate) leads to higher numbers of infected cases. However, they also show that the higher the transmission probability, the greater the naïve heuristics outperform the baseline, i.e. testing each holding on average only once per year. Hence, they suggest more sophisticated test strategies for highly contagious diseases that spread quickly.

We determine several parameters before the SI simulations are run. For instance, the transmission probability, the probability that a node gets infected from outside the network during one day, the probability that the target node of a transmission gets infected when

¹Hyper-parameters of the SI model have been discussed with epidemiologists in talks over the course of the project.

2. Data

the source node was infected and the probability that a test at an infected node is indeed positive (=sensitivity of a test). We assume tests may be false-negative, i.e., the probability to test positive 0.9. Simultaneously, we assume tests can not be false positive, meaning a positive test result implies the node is infected. We also define the number of days between becoming infected and becoming positively testable, the number of days between becoming infected and becoming infectious and the number of days after which an infection is automatically detected without test because of obvious symptoms. We set the time the infection should be detected automatically due to obvious symptoms even without a test to 60 days. In the simulation, we additionally pre-determine the inconspicuous time. The inconspicuous time refers to the time period, in which the individual becomes infected and the first appearance of obvious symptoms that cause immediate detection with a test [30]. Different infectious diseases have varying inconspicuous times, however, due to computational cost we only consider 30 days in our optimisation process. Still, to show the impact, we provide an evaluation of naïve heuristics for a range of values for the inconspicuous time. There it can be shown that the larger the inconspicuous time, the more the naïve heuristics outperform the baseline, i.e. testing each holding once a year.

The trade networks we let the SI model run on are temporal, thus each trading connection between nodes has a time property. In the simulation, the time of a trade between a source and a destination node is compared with the infection time of the former. Thereby, we check whether the disease is being transmitted to the destination node during the trade. Both causality and the infection probability are taking into account in the spreading process of the disease during a run of the SI simulation. The spreading process of the disease continues until it reaches a node that a test budget has been distributed to. Here, the simulation performs one test for a disease. In case the test result is positive ² the outbreak is detected at this node and the simulation is stopped after an additional small number of days (parameter) representing the time the authorities need to take measures to stop further trading. After that, the total number of infected animals is estimated on the basis of the holding capacities of the infected farms. Based on the fact that the SI model, i.e. the objective function, is very stochastic, we perform 10,000 independent SI model simulations for a specific test strategy and let it predict the average infected animals. If not a single node is tested positively, the SI model simulation continues until the end of the

²As stated earlier: we allow for false-negative rate of 0.1, hence the probability to test positive is 0.9. For simplicity, we assume this probability rate is equal for all holdings and constant over time.

2. Data

inconspicuous time, and since the disease is assumed to then be detected from symptoms, we proceed as if a positive test had occurred at that point of time.

In traditional epidemic models that are still being used to date, it is assumed that the relevant population is homogeneously mixing [63]. In this case, the rate at which susceptible individuals can become infected is a product of the number of contacts each individual has per unit time, r , the probability of transmission of infection per contact, β , and the share of the population that is infectious, I/N . The total population size, N , is the sum of the number of susceptible, S , and infected individuals, I . Hence, the Equations 2.1 for traditional homogeneous mixing population epidemic SI models are:

$$\frac{dS}{dt} = -\beta r S \frac{I}{N} \quad \frac{dI}{dt} = +\beta r S \frac{I}{N} \quad (2.1)$$

Since the population size is fixed, the system is reduced to one dimension with the substitution $S = N - I$ to provide the logistic Equation 2.2.

$$\frac{dI}{dt} = +\beta r (N - I) \frac{I}{N} \quad (2.2)$$

In the last step, by solving the analytical Equation 2.2 and inserting the initial condition $I(0) = I_0$, it follows Equation 2.3:

$$I(t) = \frac{I_0 N}{(N - I_0)e^{-\beta r t} + I_0} \quad (2.3)$$

More realistically, even for moderate population sizes N , populations differentiate in many characteristics such as age, vulnerability, social and ethnic groups. Also, spatial spread leads to heterogeneity in disease transmissions as an infectious disease outbreak typically happens in one location and spreads elsewhere [60]. Network models represent a class of models that are capable of taking heterogeneity of a population into account[63]. In this work, we utilise temporal networks that can also be represented as transmission networks. The topological features differ from standard network ensembles such as random, random geometric, small-world, or scale-free networks [30], which are extensively described in [1, 23].

2.3. Research questions and hypotheses

In the last years, neural networks have been successful in many fields and they have shown their potential for solving many complex tasks, see [18]. However, for some time they have been widely disregarded as blackbox optimisation methods, only now becoming more popular. In this work, our goal is to examine the potential of neural networks, and in particular deep reinforcement learning, to find optimal policies for an epidemic SI model that models a complex environment with no analytical representation. For combinatorial optimisation, Ansari et al. [30] have already shown that Monte Carlo Markov Chains (MCMCs) and simulated annealing (SA) are suitable methods. We apply classical mathematical optimisation, by combining both graph neural networks and deep reinforcement learning. Although both methods are individually well known, their joint use has been studied rarely. Also, by applying single-step proximal policy optimisation, we follow an additional unconventional approach that has yet to be studied more extensively. A key research question is whether our GNN+DRL approach is suitable for blackbox optimisation. Therefore, we compare our results to Ansari's et al. [30] findings³ and a number of naïve heuristics that have been determined beforehand. Graph neural networks are inherently designed to generalize over graphs of different sizes and structures, hence they should perform superior to architectures such as multilayer perceptrons (MLP). Also, we compare our findings to insights from social network analysis, in particular Lentz' et al. [23] findings regarding key epidemic drivers in the German swine trade network. Are our findings coherent, and has blackbox optimisation of an epidemic model the potential to give additional important insights into spreading behaviour of infectious diseases in live animal trade networks?

³Ansari's et al. [30] seek an optimal budget distribution, where the number of nodes, so-called sentinel nodes, receiving budget shares is predetermined and the budget is split uniformly among the sentinel nodes. Furthermore, it turned out that the predictions of Ansari's SI model simulations for a particular test strategy differ in terms of the estimated average number of infected animals compared to the predictions of the SI model for the same particular test strategy that is utilised in this thesis. Therefore, a direct comparison of the performances of both methods is not possible.

3. Methods

To predict the average number of infected animals in a live trade network, we rely on an epidemic SI model. The evaluation of the SI model is computationally costly and its objective remains not accessible. Also, it is very stochastic, i.e. the output of the simulation tends to differ substantially when executed using the same input more than once. In this chapter, we briefly introduce blackbox optimisation methods for solving stochastic and nonlinear optimisation problems. Subsequently, we describe Ansari’s et al. [30] approach to utilise the MCMC Metropolis–Hasting algorithm and simulated annealing to improve test strategies for infectious diseases in live animal trade networks that also form the data basis for this thesis. Following that, we discuss neural networks, in particular DRL, as newly rediscovered blackbox optimisation algorithms. PPO is one of the most prominent algorithm used in DRL. It utilises two neural networks for evaluation decisions and adjusting weights. We know, the swine trade network can also be represented as a large weighted directed graph incorporating node features (a.e. degree, capacity), edge features (transmissions) and graph features (SI model parameters). Therefore, we propose GNNs as neural network architectures for PPO.

3.1. Introduction to blackbox, derivative-free and derivative-based optimisation methods

With increasing computational power, computer simulations have become an essential tool of the 21st century. They are being applied in particle physics, chemistry, global supply chains, epidemiology and various other fields and are faced with increasingly complex tasks. Consequently, they usually are nonlinear and very stochastic. A single simulation may run for a few minutes up to days and weeks and the output rarely is unambiguous. The objective and/or constraint functions underlying complex simulations

3. Methods

are often hidden and inaccessible, making them so-called black boxes [12]. However, computer simulations could produce fully differentiable functions, but the output may not include derivatives. For solving optimisation problems as such, derivative-free algorithms that do not use derivatives have become popular methods in the last two decades [64, 11, 12, 16]. If evaluations or reliable approximations of the gradients are possible, then derivative-based optimisation methods should be used [12, 16].

A function $f : \Omega \rightarrow \mathbb{R}, \Omega \subseteq \mathbb{R}^n$ is considered to be a blackbox, if you can evaluate $y = f(x)$ at $x \in \Omega$ without prior knowledge of its analytical form. Hence, BBO algorithms aim to find an optimal $x^* = \arg \min_{x \in \Omega} f(x)$ [15], typically with minimal evaluations [12]. Difficulties that arise in real-world BBO applications involve the fact that one may not be able to evaluate the function f precisely but only get noisy estimates of it, as is the case in our application, where the true expectation value of the number of infected animals can only be estimated via a Monte–Carlo simulation. This estimate of $f(x)$ is not only noisy but a second Monte–Carlo simulation estimation of $f(x)$ will also return a slightly different value due to that noise [12]. To reduce deviations in our experiments, we run 10,000 SI model simulations for every single test strategy, and then take the average over the outputs. Also, often in BBO the evaluation of the objective and constraint function is very computationally expensive. As one solution, surrogate optimisation problems are available, however, here we face less accurate evaluations. If we consider a constrained optimisation problem of the form $\min_{x \in X \subset \mathbb{R}^n} \{f(x) : c(x) \leq 0\}$, then $\min_{x \in X \subset \mathbb{R}^n} \{\tilde{f}(x) : \tilde{c}(x) \leq 0\}$ is a surrogate if $\tilde{f} : X \rightarrow \mathbb{R} \cup \{\infty\}$ and $\tilde{c} : X \rightarrow (\mathbb{R} \cup \{\infty\})^m$ share some similarities with f and c but its simulations are much faster to evaluate. Then both the functions $\tilde{f}$ and $\tilde{c}$ are said to be surrogate functions of the true functions f and c [12]. Good approximations to the objective function may be bad optimisation surrogate functions and vice versa as is shown in 3.1.

3. Methods

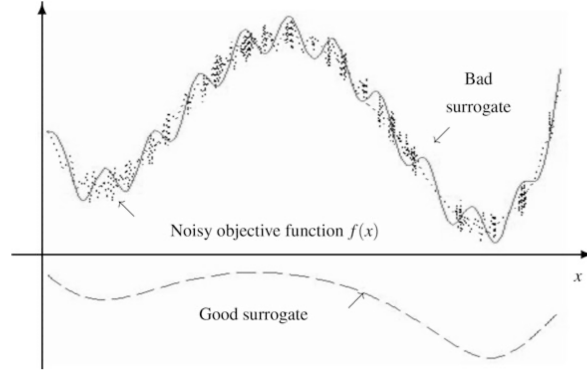


Figure 3.1.: A visualisation of a good and a bad surrogate function. Taken from Audet et al. [12], Figure 13.1.

BBO methods can also be split into heuristics and non-heuristic algorithms. A heuristic is a framework that does not guarantee to be successful. They are designed to solve a problem in a fast and efficient fashion by sacrificing optimality and are easy to understand and to implement. Hence, they are more widely applied compared to non-heuristic methods [12]. However, many non-heuristic algorithms allow for imbedded heuristic subroutines. Often non-heuristic methods converge locally to a critical point, but not necessarily to a global minimum. Embedding a heuristic into a non-heuristic algorithm allows for breaking out of a local minimum, i.e. a not optimal solution to a problem [12].

3.1.1. Derivative-free optimisation

Popular DFO algorithms are evolutionary algorithms (EA), MCMC Metropolis–Hasting (MCMC), simulated annealing (SA), hill climbing (HC) and Nelder–Mead methods (NM). In the following, we briefly explain evolution strategy (ES) techniques that are a subclass of EA, because the method we apply, single-step PPO, inherits conceptual similarities to ES [21]. We also make some remarks about MCMC Metropolis–Hastings and simulated annealing since Ansari et al. [30] show that by applying both methods, they were able to optimise test strategies to reduce the number of infected animals in live animal trade networks.

3. Methods

3.1.1.1. Evolution strategies

ESs are part of EA that evolve iteratively utilising mutation, recombination, and selection applied to a population of individuals containing candidate solutions to find more optimal solutions. The algorithm has its origins in the mid 1960s when P. Bienert, I. Rechenberg, and H.-P. Schwefel [65] developed the first bionics-inspired structures applying Darwin's evolution principle [66]. In every iteration or generation g , candidate solutions $(x_1, \dots, x_\lambda)$ are sampled from a multivariate normal distribution $N(m^g, C^g)$, where m^g is the mean and C^g is the covariance matrix. The cost function is evaluated at the candidate solutions and the algorithm updates the search distribution for next g by utilising a weight recombination of the μ best individuals [21]. The covariance matrix adaption of evolutionary strategy (CMA-ES) is considered as the state-of-the-art algorithm. It utilises a full covariance matrix to accelerate convergence towards the optimal point. In the covariance matrix update process $C^{(g+1)} \leftarrow (1 - c_1 - c_\mu)C^g + c_\mu C_\mu + c_1 C_1$, the terms $c_\mu C_\mu$ and $c_1 C_1$ are two update processes, whereas the first term $(1 - c_1 - c_\mu)C^g$ represents a soft update from the current covariance matrix. The two learning rates c_1 and c_μ are set by popular heuristics, and are associated with the two types of updates so-called *rank - 1* and *rank - μ* . While the update process *rank - μ* incorporates information regarding the best individuals of current g , the rank-1 update process add correlation information across consecutive g by storing the average update direction in a so-called evolution path. By that, correlated updates sum up, decorrelated updates cancel each other out [21]. Single step PPO shows some similarities to ES and CMA-ES as J. Viquerat et al. [21] point out:

- Both generate new samples by exploiting the successive updates of a probability density function, which lets the algorithm converge to a local or a global minimum.
- Also, they both use normal distributions. Isotropic for ES, multivariate with full covariance matrix for CMA-ES and PPO.
- Single step PPO only takes positive-advantage actions into account when computing the policy loss, which allows the policy to stay consist with experiences collected. It is similar to the update rule in CMA-ES where individuals in each generation g are selected by some preferences.

3. Methods

- Single step PPO utilises information from previous steps or generations to update the network parameters, similar to CMA-ES utilising an evolution path to add information about correlations across consecutive generations.

3.1.1.2. Metropolis–Hastings and simulated annealing

To simulate complex and non-linear multivariate distributions, Markov Chain Monte Carlo (MCMC) methods have been popular for quite some time now [67]. They do not suffer as much as other algorithms under the ‘curse of dimensionality’¹, therefore, they are applied to solve high-dimensional stochastic problems in various disciplines [67]. A Markov chain is a stochastic process in which future states are independent of past states given the present state. If we consider drawing from θ at state t , then $\theta^{(t)}$ is a state at iteration t . To fulfill the Markov property $p(\theta^{(t+1)}|\theta^{(1)}, \theta^{(2)}, \dots, \theta^{(t)}) = p(\theta^{(t+1)}|\theta^{(t)})$, the next draw $\theta^{(t+1)}$ must only depend on the current draw $\theta^{(t)}$, and not on any past draws.

A Markov chain represents a number of draws of θ , where each $\theta^{(t)}$ differs slightly from the previous one. The chain explores the parameter space, but only takes the state of the last period into account. If we define a stationary distribution Ω to be some distribution M such that $\Omega = \Omega P$, where P represents a transition matrix², then eventually the Markov chain will typically converge to Ω regardless of the starting points used. If our desired posterior distribution $p(\theta|y)$ can be devised by a Markov chain with stationary distribution Ω , then by running the Markov chain we get draws that are approximately from $p(\theta|y)$ once the chain has converged.

To calculate quantities of interest for the posterior distribution, two MCMC algorithms are mostly applied: the Gibbs sampler and the Metropolis–Hastings algorithm. Ansari et al. [1, 30] successfully apply MCMC Metropolis–Hastings algorithms in two epidemiological optimisation problems. In [1] they remove links from a contact network between people in order to increase the epidemic threshold, whereas in [30] they optimise test strategies to minimise the number of infected animals in a live animal trade network, i.e. utilising

¹Richard Bellman [68] introduced the phrase the curse of dimensionality as a phenomena that describes the rapid growth in the complexity of problems as the number of variables (or the dimension) increases. For instance, the computational cost of an algorithm grows exponentially with dimension which makes the cost prohibitive for larger values of the dimension [69].

²A transition matrix for a discrete space (k possible states) would be a $k \times k$ matrix of transition probabilities.

3. Methods

the same data that we use. In general, the Metropolis–Hastings algorithm looks the following:

1. We begin by choosing a starting point $\theta^{(0)}$.
2. Each iteration t , a candidate θ^* from a distribution $J_t(\theta^*|\theta^{(t-1)})$ is drawn.
3. We compute the acceptance ration or probability $r = \frac{p(\theta^*|y)}{J_t(\theta^*|\theta^{(t-1)})} / \frac{p(\theta^{(t-1)}|y)}{J_t(\theta^{(t-1)}|\theta^*)}$.
4. We accept the candidate θ^* as $\theta^{(t)}$ with a probability of $\min(r, 1)$. If θ^* is not accepted, then $\theta^{(t)} = \theta^{(t-1)}$.
5. To get n draws from $p(\theta|y)$, we repeat steps 2 to 4 n times.

In Ansari et al. [30], the input of the MCMC simulation is the synthetic network data presented in chapter 1, the number of nodes k , so-called sentinels, they choose to distribute a test budget among, a strategy for generating a candidate θ^* in each MCMC step, the parameters of the target function that is used for evaluating θ^* , a probability function for turning the evaluation into an acceptance probability, and the number of MCMC steps/iterations which they set to a step-size of 1,000,000. The number of k sentinels varies and depends on the size on the total budget or total tests available. If they test a node or farm in the network on average once a year, then the total number of tests is the same as the number of nodes or farms in the network. They consider a fixed budget of $N = n$ and different sentinels, for instance $k = N/52, N/26, N/12$ or N . In their case, the MCMC Metropolis–Hastings algorithm performs the following steps:

1. The simulation starts from a set of k randomly selected nodes as initial state c_0 in the search space.
2. In each MCMC step or iteration:
 - a) A new candidate c' is generated by swapping one node between the current sentinel set c_g and the remaining non-sentinel nodes in the network.
 - b) The SI model evaluates the new set with the new candidate c' by averaging over 10^3 independent SI runs and estimates the average number of infected animals over all runs.
 - c) Then the acceptance rate α with the probability distribution $\alpha = \min(1, p(c')/p(c_t))$ is calculated.

3. Methods

- d) A new candidate is accepted or rejected by generating a random number u . If $u \leq \alpha$ the candidate is accepted by setting $c_{c+1} = c'$ and this is the new state in the Markov chain. If $u > \alpha$ then the candidate is rejected by setting $c_{t+1} = c_t$ and we do not move in search space.

3. The second step is repeated in this case 1,000,000 times or in general the number of MCMC-steps times.

Simulated annealing (SA) is a probabilistic method proposed in Kirkpatrick, Gelatt and Vecchi [70] and Cerny [71] to find the global minimum of a cost function. In the simple MCMC Metropolis–Hastings algorithm, there exist a temperature parameter that is fixed. Normally, the temperature parameter induces exploration, hence initially it helps not getting trapped in the later process it prevents the fine-tuning needed to find the location of the global optimum [30]. A solution often applied, is SA. SA decreases the temperature gradually from an initial positive value to zero (similar to a 'learning rate scheduler' normally used in deep learning). Another solution to prevent being stuck in local optima, is to start from an initial candidate solution $\theta^{(0)}$ that incorporates prior knowledge about the location of a global minimum. This approach prevents over-sampling of low probability regions and also, it essentially trims off the front of the Markov chain and shortens the path to settle into the equilibrium state [30].

To efficiently generate next optimal candidates c' , Ansari et al. [35] introduce two different approaches. In the general MCMC Metropolis–Hastings case, new samples are generated by swapping randomly one node between the current set of sentinels and the rest of the nodes. Alternatively, they propose:

1. Selecting the next optimal candidate c' with a probability proportional to the average distance of c' to the remaining sentinel set. This method favours nodes that can detect epidemic outbreaks in other regions of the network than the other sentinel nodes.
2. Selecting the next optimal candidate c' with a probability proportional to the degree of c' . Thereby, the node with the most number of connections has the highest chance to be in the new set of sentinels. As discussed, many social network analyses have shown that the farms or nodes in live animal trade networks with the highest degrees appear to be reasonable candidates to focus epidemic surveillance and test strategies on and to prevent large epidemic outbreaks (see for instance [23]).

3. Methods

Ansari's et al. [30] findings suggest that by applying MCMC and SA, it is possible to find test strategies that reduce the estimated average number of infected animals up to 55 % in comparison to prevailing test policies. They distribute a limited test budget uniformly among a prefixed number of network participants, i.e. to $N/12, N/26, N/52$ nodes, where N is the number of total nodes in the network. For instance, by distributing a test budget to $N/12$ nodes in a transmission network with 120 nodes, the test budget is shared among ten nodes, whereas each receives a share of 10. Ansari et al. utilise the same SYN transmission networks with 120 and 1040 nodes, and each 100 samples that we use in the thesis, see section 2. However, they also analyse test strategies by utilising 10 SYN transmission networks with 57,590 nodes and 10 real German swine trade transmission networks (REAL) with 57,590 nodes. The MCMC and SA optimisation process is explained in section 3.1.1.2 extensively.

Ansari et al. show that for all networks (SYN and REAL) and all sizes (120, 1040, 57590) the "naïve" test strategies $HIT_{S_{B=N}}$ and $HD_{S_{B=N}}$ perform considerably better compared to $UNI_{S_{B=N}}$ which corresponds to our findings in section 4.0.1. However, by applying MCMC and SA they are able to find a set of nodes that reduce the estimated average number of infected animals even further. They show that distributing the total test budget between $N/12$ and $N/26$ nodes in the network performs best compared to distributing the total test budget among larger sets of nodes. The differences in performance of the optimisation process if the total test budget is distributed among N (=UNI), $N/12$, $N/26$, and $N/52$ nodes (so-called sentinel nodes), where N is the number of total nodes in the network is shown in Figure 3.2. Although MCMC and SA achieve the lowest estimated average numbers of infected animals, the "naïve" testing strategies $HIT_{S_{B=N}}$ and $HD_{S_{B=N}}$ are almost as effective.

3. Methods

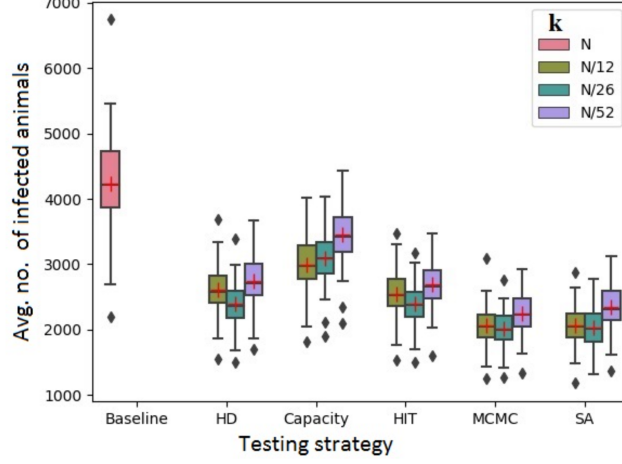


Figure 3.2.: Each box-plot shows the minimum, first quartile, median, third quartile, and maximum of the expected outbreak size at the time of detection in an ensemble of 100 synthetic networks with 1040 nodes and 13500 edges. The expected outbreak size for each transmission network is estimated by running 10,000 SI simulations and summing up the capacities of nodes infected at the time of detection. The red marker shows the mean for each test strategy and number of sentinels for the ensemble of 100 synthetic networks. Taken from Ansari et al. [30], Figure 5.

Ansari et al. find MCMC and SA most effective for real German swine transmission networks with 57,590 nodes, see Figure 3.3. By optimisation, they reduce the overall outbreak size by 89 % compared to $UNI_{S_{B=N}}$. However, utilising the “naïve” testing strategies HIT reduces the overall outbreak size already by 75 % compared to the baseline strategy $UNI_{S_{B=N}}$.

3. Methods

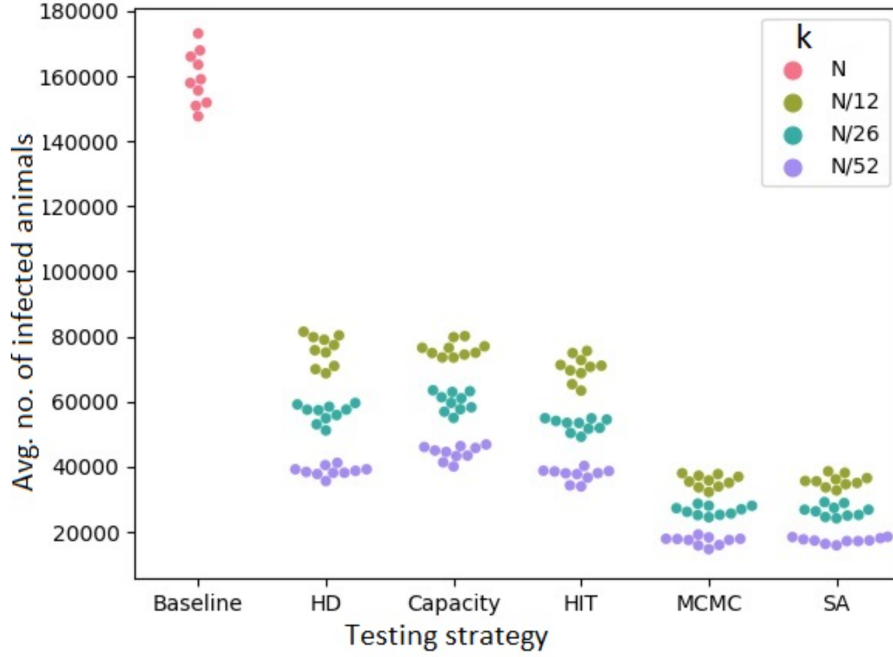


Figure 3.3.: The differences in performance of the “naïve” testing strategies compared to the optimal testing strategies found by MCMC and SA on the ensemble of 10 real swine-trade networks with 57,590 nodes and 810,000 edges. Each point represents the estimated expected number of infected animals at the time of detection for one real network, whereas the different colors show the influence of the number of nodes selected as sentinel nodes ($k = N/12, N/26$ or $N/52$). Selecting only $1/52$ of nodes and testing them weekly performed considerably better than, e.g., selecting $1/26$ of nodes and testing them biweekly independent of network type and network size. MCMC and SA reduce the estimated average number of infected animals in comparison to the “naïve” testing strategies the most. The standard error in the averaging of infected cases over all 1,000 SI runs and over all 10 networks is on average 500 for heuristics and 1,200 for MCMC and SA. Taken from Ansari et al. [30].

In a next step, Ansari et al. analyse the characteristics and positions of the nodes that receive a budget share if the budget is distributed optimally by MCMC. By utilising a community-detection algorithm in Gephi [72], they find six communities inside the network that are visualised and depicted in different colors in Figure 3.4. The 10 sentinel nodes that have been selected by the MCMC method are marked by a smiling face. As

3. Methods

further analysis by Ansari et al. shows, half of the sentinel nodes are centers of some communities, except the center of the red community which is a slaughterhouse, and has been disregarded as potential sentinel in the analysis (we do the same in this thesis, see section 2). The optimal set of sentinels nodes are almost evenly distributed over the network, whereby some nodes lie in between communities that they connect.

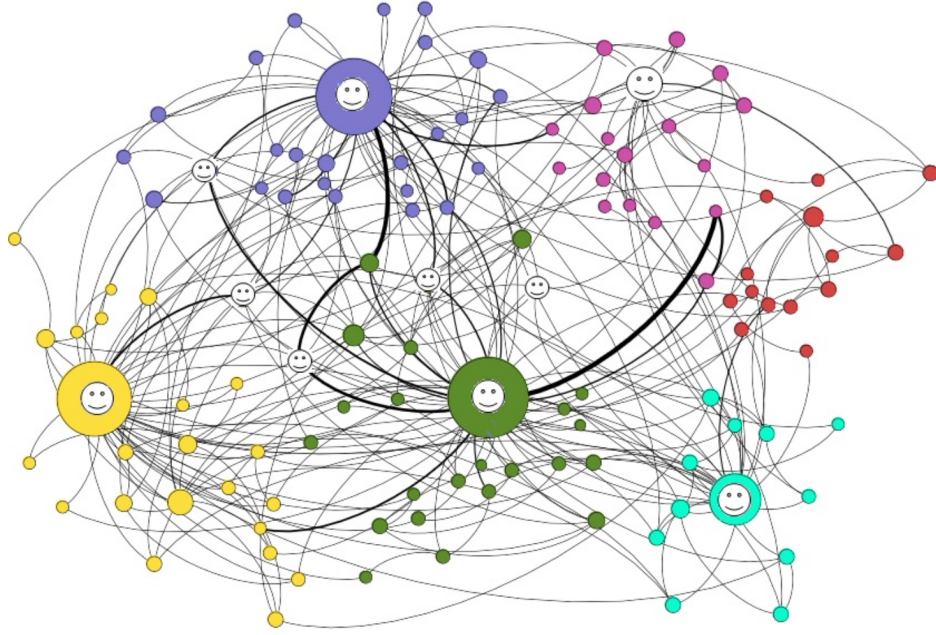


Figure 3.4.: Sentinel points found by MCMC on a synthetic network $n = 120$, each color shows one community in the network and the smiling faces mark the optimal set of sentinels as selected by the algorithm. Most community centers are chosen as sentinels. The other sentinels chosen form bridges between communities. These nodes are linked to the centers of three or four communities. It turns out they are all fattening farms, i.e., in the middle of the pork production chain. Taken from Ansari et al. [30], Figure 3.

3.1.2. Derivative-based optimisation

Derivative-based optimisation methods rely on a gradient estimation to determine the search direction. Therefore, they are restricted to differentiable functions, however, some methods can be applied successfully whenever the objective function is merely locally integrable [15]. The most popular algorithm is gradient descent, which is a technique to

3. Methods

find local optima, in particular in machine learning with neural networks. Its founders are Augustin-Louis Cauchy in 1847 [73] and Jacques J. Hadamard, who independently proposed the method in 1907 [74].

3.1.2.1. Introduction to neural networks

The earliest work on neural networks was done by McCulloch and Pitts in 1943 [75] by introducing rudimentary neurons as models of biological neurons that were conceptual components for circuits that could perform computational tasks [76]. When Minsky and Papert [77] indicated crucial flaws in perceptron models, most research in the field was abandoned. Only by the discovery of back-propagation and increasing computational power, interest renewed [78, 79, 80]. Artificial neural networks (ANNs) are a computational models designed to learn, to generalise, to cluster or structure data [81]. Why ANNs succeed in many fields compared to other existing methods, remains still an open question. To date, they are applied in many domains and their many frameworks develop rapidly [81].

Optimising the weights of ANNs is typically done by utilising gradient-based algorithms. Here, information from an output layer is propagated back through a number of layers of a network and all connections are numerically changed by that information [82]. In comparison to biological neural networks (BNNs), ANNs are less flexible and smaller (BNNs have an estimated number of 100 billion neurons, ten times more glia cells, magnitude more connections, whereas ANNs have only about 50–100 millions of weights or connections [82]). Also, BNNs areas can adapt, change and move function and grow out new connections. An ANN's number of neurons is normally fixed and remains that way during the learning process [82]. Another major difference between ANNs and BNNs is the power consumption. Whereas the brain only consumes about 20% of the body's energy and operates at about 20 Watt, the training process of neural networks requires multiple GPUs that may operate at up to 250 Watts and produce a lot of heat [82].

The major components of neural networks are processing units or neurons or cells which transmit information by sending signals to each other over a number of weighted connections. Each neuron receives an input from external sources or neighbours which is computed to an output signal that is sent to other neurons [81]. Most ANNs are hierarchically structured in form of layers, where each layer consists of a number of neurons.

3. Methods

However, different architectures exist, although they all resemble a flow of information usually from one layer to the next layer [81]. Layers can be distinguished by its types of neurons. Input layers and its neurons receive data from outside the neural network, i.e. the environment, output layer and its neurons send data out of the neural network, hidden layers and its neurons information flow remains within the neural network and they act as building blocks between the input neurons and the output neurons [81]. A representation of a neuron with n inputs is shown in 3.5. An arbitrary real-valued function f of one parameter, which is called the activation function, computes the output by usually multiplying the incoming information x_i by the corresponding weight w_i . The received information is integrated at the neuron, usually by computing the sum of the different signals, then evaluated by function f [76].

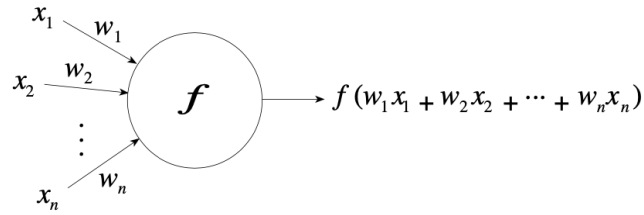


Figure 3.5.: An abstract neuron. Taken from Rojas [81], Figure 1.14.

Each neuron in the neural network can be conceived as an arbitrarily function that transforms input signals in an output signal. Hence, a neural network is a representation of networks of arbitrary functions. Neural networks typically differ in assumptions about the functions utilised, the interconnection pattern and the timing of the transmission of the information [81]. An example of a neural network can be seen in Figure 3.6. It can be understood as a function Φ that is evaluated at point (x, y, z) . It consists of four neurons that utilise arbitrarily functions f_1, f_2, f_3, f_4 which eventually compute Φ . By altering the weights α_i , different numerical outputs are emitted for the same inputs [81].

3. Methods

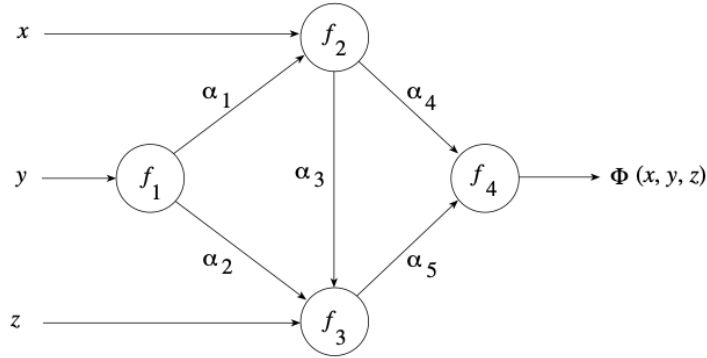


Figure 3.6.: Functional model of an example artificial neural network. Taken from Rojas [81], Figure 1.15.

Hence, the behaviour of neural networks is mainly driven by the arbitrary functions of the neurons, the topology of the network and the learning algorithm that is used to find numerical values for the weights of the network to produce the desired output $\Phi(x, y, z)$ [81]. Essentially, they can be understood as black boxes in the sense that for a certain input the weights of the network are chosen such that it emits a desired output. The exact causality how the network achieves the result is left to a self-organising process of the ANN and the learning algorithm [81]. In general neural networks work as “mapping machines” that model a function $F : \mathbb{R}^n \rightarrow \mathbb{R}^m$. Thus, they map an n -dimensional real input $(x_1, x_2, \dots, x_n)$ to an m -dimensional real output $(y_1, y_2, \dots, y_m)$ as shown in Figure 3.7 [81]. Furthermore, the universal approximation theorem says that large enough neural networks with at least one hidden layer are capable of approximating any continuous function with an input of certain range, even though that might require enormous numbers of neurons [83].

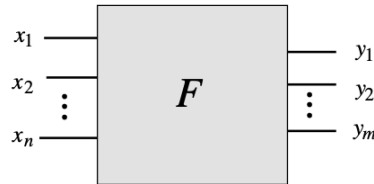


Figure 3.7.: Neural network as a blackbox. Taken from Rojas [81], Figure 2.1.

3. Methods

The most basic types of neural networks are feed-forward neural networks and recurrent neural networks. In feed-forward networks information is transmitted through the directed edges of the network in a hierarchical manner, i.e. in a one way street. Recurrent neural networks contain cycles, in which a output of a neuron is fed back to the same neuron resulting in recursive computations [81]. In the last decades many neural network architectures have been proposed that further develop both methods. A comprehensive overview over the developments can be found in [84, 85].

Backbone of all neural network types is the learning of a particular combination of the weights by the backpropagation algorithm [81]. In the learning process, first, the input of the neural network is propagated forward through the network to compute the output. The output is compared to a desired value, the difference is captured in an error signal. Then, secondly, the error signal is passed back to each neuron in the network and the weights are changed correspondingly [81]. The backpropagation algorithm utilises gradient descent to seek the minimum of the error function in weight space. The gradient or the slope of a function, represents the change in all weights with regard to the change in the error signal, i.e. it measures the change of an output for a small change of the inputs [81]. An optimal combination of weights would minimise the error function and represents a solution to the learning problem. To be able to compute the gradient of the error function at each iteration step, the error function has to be continuous and differentiable [81]. It also must not have any flat regions, since we want to follow the gradient direction to seek for a (generally local) minimum. Therefore, differentiable activation functions exist to make the function computed by a neural network differentiable. Consequently the error function becomes differentiable, too [81].

Besides different neural network architecture types, the learning situation can be distinguished in multiple sorts. In supervised learning, the algorithm learns to predict an output by utilising a training set in which inputs and outputs or labels are given by a “knowledgeable” external supervisor [20]. By finding crucial characteristics or patterns identifying a specific input for a specific output in the learning process, the algorithmic goal is to extrapolate, or generalise at some point to data that is not present in the training set. Unsupervised learning or self-organisation is typically utilised to find structural peculiarities in data without knowing what to look for [20]. The algorithm is not provided any information about the output, i.e. there is no a priori organisation of the data. As Dong et al. state, deep reinforcement learning could be understood as a subclass of unsupervised

3. Methods

learning since neither relies on labeled data in the training process [20]. However, in contrast to unsupervised learning, it is not designed to find hidden structures in the data but to maximise a reward signal [81]. We will take a closer look into deep reinforcement learning as a form of blackbox optimisation in the following.

3.1.2.2. Deep reinforcement learning

Reinforcement learning is a approach to imitate the human nature of learning from interaction [18, 20]. In recent years, many algorithms have been proposed for different learning behaviors and proved their efficiency for solving tasks in various environments. In particular Google’s DeepMind team has gained a lot of popularity by introducing the programs AlphaGo, AlphaZero, MuZero and AlphaFold [20, 86]. These programs utilise deep neural networks, supervised learning, Monte Carlo tree search and reinforcement learning and have been very successfully applied in a number of tasks. AlphaGo won against the 18-time world champion Lee Sedol in the board game of Go, AlphaZero and MuZero are its more efficient successors [20]. AlphaFold is used to predict structure folds of proteins and achieves accuracy levels comparable to experimental methods [87, 88]. Its release has led to the creation of AlphaFold DB10, a database that contains over 804K protein structures covering 21 species [89].

In reinforcement learning the goal is to map situations to actions in a way that achieves as high rewards as possible. The major components are the agent or the policy, a reward function, a value function, and the environment [20]. While the environments dynamics depend on the individual task, the agent represents a policy that has to be improved in the training process. In the beginning, the agent does not know which actions to take in the environment. The action space can either be discrete or continuous and describes all possible actions of an agent. By engaging in a feedback-loop, in which the agent receives a reward signal r for each action taken, it updates its policy to select actions that increase r [20]. Thus, the reward signal r gives immediate feedback about the short-term effects of the actions for the agent. It is used to change the agent’s behavior or its policy such that the agent may select different actions in the particular environment’s state in the future [20]. Whereas the reward signal gives immediate feedback and is emitted by the environment, the value function V must constantly be estimated and re-estimated. It guides the agent to take actions that lead to high rewards in the long-term, since in

3. Methods

most reinforcement learning approaches, the agent's goal is to maximise the accumulated reward in an episode, i.e. the (discounted) sum of all rewards in an episode. However, estimating the value of a present action with regard to future actions and its corresponding rewards is complex. Therefore, value estimation plays a central role in current research [18].

Reinforcement algorithms may differ in their representations of environments. In model-free reinforcement learning methods the agent interacts with the environment directly. It does not learn a model of the environment but rather tries to optimise its behaviour based on previous actions taken. Hence, model-free methods do not rely on transition and reward models, making them advantageous when the problem to be solved is rather difficult to learn accurately [18, 20]. In model-based methods the agent learns a model representation of the environment. In theory, when an agent learns an accurate representation of the real environment, it can derive the optimal policy from it without any additional interactions with the environment. Therefore, compared to model-free methods that are trial-and-error learners, model-based methods are more sample-efficient [20]. However, environments are often rather complex and their dynamics difficult to learn. When applied to real environments, it may often result in an estimation bias [18, 20].

Many of these reinforcement learning algorithms can be viewed as derivative-based algorithms in context of BBO. Essentially, the agent or learner must explore the environment (or learn a model representation of it) on its own by a trial-and-error search, adapting its behaviour by taking its past experience collected into account [18]. The relationship between an agent and the environment can be seen in Figure 3.8. In the learning process, the agent observes the current state of the environment S at time step t . The corresponding reward value for S_t is R_t . In a next step, the agent chooses an action A_t based on the state S_t and the reward R_t he receives by interacting with the environment, which eventually leads to a next state S_{t+1} and a next reward R_{t+1} [20, 18].

3. Methods

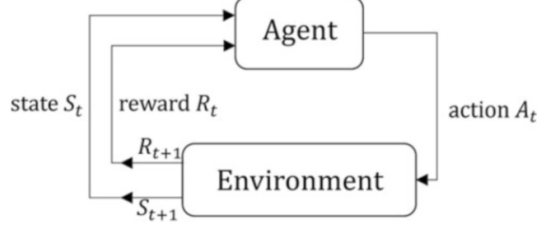


Figure 3.8.: Agent and environment. Taken from Dong et al. [18], Figure 2.1.

However, environments can be either partially observable or fully observable environments from the perspective of the agent. In the former, the agent’s observation only contains partial state information, whereas in the latter, the agent can observe the environment fully. Since the observation is usually a function of the state, it makes it hard to separate observations containing all state information with observations that only partially contain all state information. Hence, in a partially observable environment, the agent’s decision depend on its observation of the environment and not on the environment’s state [18, 20].

In reinforcement learning the agent learns by engaging in a feedback-loop with the environment. The reward function R emits the reward signal R_t depending on the environmental state S_t and is transmitted to the agent in every time step. The reward function may depend only on the current state, i.e. $R_t = R(S_t)$, but it may also depend on the states and actions at other time steps. Thus, the reward can depend on a sequence of state-action pairs. In reinforcement learning, a trajectory or episode represents a sequence of states, actions, and rewards, i.e. $\tau = (S_0, A_0, R_0, S_1, A_1, R_1, \dots)$. It captures the agent’s interaction with the environment and the resulting next states and next rewards until a terminal state is reached and the episode ends. The transmission from a state to the next state can either be a deterministic transmission process $S_{t+1} = f(S_t, A_t)$, or a stochastic transition process $S_{t+1} \sim p(S_{t+1}|S_t, A_t)$, where the next state S_{t+1} is described as a probabilistic distribution [18]. The number of episodes and the number of steps within an episode taken by the agent can be defined in advance, i.e. the episode ends when the agent violates a particular rule that has been defined beforehand. In the learning process, usually the agent is faced by the decision to rather exploit or explore its environment. The so-called exploration-exploitation trade-off describes the difficulty to maximise the agent’s performance by

3. Methods

using existing knowledge on the one hand (exploitation), and to increase the existing knowledge on the other hand (exploration). Finding the optimal balance between both concepts is an important part of research in reinforcement learning.

In this work, an agent’s action is a specific test strategy that is evaluated in the environment by Monte-Carlo SI model simulations. The output of these simulations, i.e. a prediction of the average number of infected animals at the time the outbreak is detected for the agent’s proposed unique test strategy (or action of an agent), is used to set the reward. In a feedback-loop the agent’s goal is to select actions (test strategies) which maximize its reward (here: minimize the expected number of infected animals). Deep reinforcement learning algorithms differ in terms of a range of assumptions about what quantities are represented, how to update the agent’s behaviour (or policy), and hence how that influences its decisions [20].

In so called on-policy methods, the algorithm attempts to evaluate or improve the behaviour that lead to actions by evaluating its previous behaviour(s). Each time the agent’s behaviour is changed, it starts over, limiting the access it has to past experiences [18, 20]. So-called off-policy methods leverage from a past time horizon that is much larger compared to on-policy methods. Rather to recollect samples whenever a policy is changed, off-policy methods comprise all past experiences in a (replay) buffer to update the behaviour, they sample from ‘memory’ [18, 20].

Proximal Policy Optimisation (PPO) is a model-free on-policy algorithm, whose advantages are an easy implementation and that it has a high convergence rates compared to other RL algorithms [19].³ In this work, the main novelty is the application of a single-step proximal policy optimisation (PPO) algorithm with graph neural networks as both its actor and critic. Single-step PPO has already been demonstrated to be successful in a number of open-loop control problems [19, 21, 22], but its comparison against state-of-the-art optimisation techniques such as evolution strategies or MCMC has yet not been researched extensively. In single-step PPO the optimal policy to be learnt is state-independent, and the actor gets only one attempt per episode to find the optimal action [21]. PPO is a so-called actor-critic (AC) approach. Both the actor and the critic are neural networks.

³For a stable and uncomplicated setup, we use the stable-baselines3 library in PyTorch and its out-of-the box algorithms. This limits our ability to customize the RL framework, i.e. its environment and policies, to some degree and potentially lowers the overall performance.

3. Methods

We implement the algorithm by incorporating graph neural networks as both its actor and critic, since a livestock trade network can be represented as a graph consisting of vertices or nodes and edges [1, 23]. In our case vertices or nodes represent different types of animal farms, whereas edges represent animal movements between two nodes. To utilise this information in our optimisation process, we apply a class of neural networks that Battaglia et al. [24] propose which is capable to operate on graphs, and hence structure their computations accordingly. We give a general introduction to graph neural networks in the next section.

3.2. Graph neural networks

Graphs are data structures that allow to model a set of objects or nodes and their relationships or edges to each other [25]. In general, graphs can be defined by a number of characteristics. In undirected graphs, edges or links represent unordered pairs of vertices, whereas in directed graphs edges or arcs represent ordered pairs of vertices, for instance see Figure 3.9.

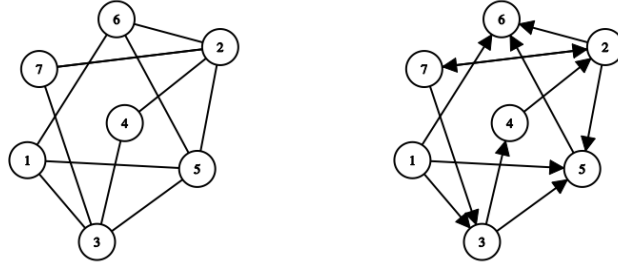


Figure 3.9.: Undirected graph (left) and directed graph (right) with 7 nodes and 11 edges in non-Euclidean space.

If node and edge types differ, we can also refer to graphs as heterogeneous graphs or if the nodes and edges have same types, the graph is homogeneous. A further distinction can be made, when the input features or the topology of the graph change with time (dynamic),

3. Methods

or when they remain the same (static). A graph can incorporate some characteristics simultaneously, i.e. a graph could be a static directed heterogeneous graph [25].

In recent years many areas that work with network or graph structures, i.e. social networks, physical systems, or knowledge graphs, have started to utilise machine learning to analyse the network topology [24]. First introduced in the late 1990s, convolutional neural networks (CNNs) were a neural network architecture designed to learn spatial hierarchies of features and construct complex representations of them [25]. However, CNNs are only able to operate on regular Euclidean data like images (two-dimensional grids) and texts (one-dimensional sequences) by utilising local connections, shared weights and multiple layers for processing the data. To extend deep neural models to non-Euclidean domains, geometric deep learning or graph neural networks (GNNs) have emerged as a research branch in deep learning. GNNs are a class of deep learning methods designed to perform inference on data that can be described by graphs. They have been developed and explored rather extensively for decades but have risen in popularity only in recent years [24]. Zhou et al. [25] give a very extensive review of methods and applications by describing most prominent graph neural networks and its applications. Today, GNNs are usually utilised for tasks like node classification, link prediction, and clustering [24, 25]. They also have been applied to solve traditional computer science problems such as combinatorial optimisation, boolean satisfiability, program representation and verification and performing inference in graphical models [24].

Depending on the task, we can apply machine learning with GNNs node-wise, edge-wise, and graph-wise. When focusing on nodes, node classification categorises node into different classes, node regression predicts numerical values for each node, and node clustering separates the nodes into several groups with distinct characteristics [25]. We can also categorise the edge types or predict whether an edge or link exist between two given nodes. Graph-wise, the GNNs may learn to classify graphs, cluster graphs, or perform regression on graph representations [25]. The general design for a GNN model is shown in Figure 3.10.

3. Methods

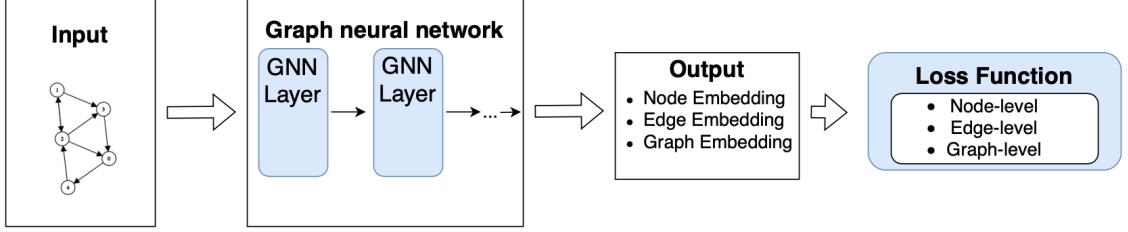


Figure 3.10.: The general design of a graph neural network. The input data is graph structured data. The GNN consists of one or more GNN layer, where the output is an embedding and the loss function depends on the specific task. Adapted from Zhou et al. [25], Fig. 2.

3.3. Blackbox optimisation with deep reinforcement learning and graph neural networks

In this section all strings come together. We introduce all individual building blocks to then explain the combinatorial approach connecting deep reinforcement learning and graph neural networks for blackbox optimisation.

3.3.1. The building blocks

In our work, we utilise a number of methods that may individually be exchangeable. Therefore, we introduce each building block successively, before we put them all together.

3.3.1.1. Transmission networks

As already mentioned in section 2.1, we analyse test strategies on three different network types (WX, AB, SYN) that differ topological from each other. In addition, to make our

3. Methods

analysis even more robust, we generate 100 samples for each network type and for each network size, i.e. 120 and 1040. The network topologies of the first samples from our dataset for each network type with 120 nodes are shown in Fig. 3.11.

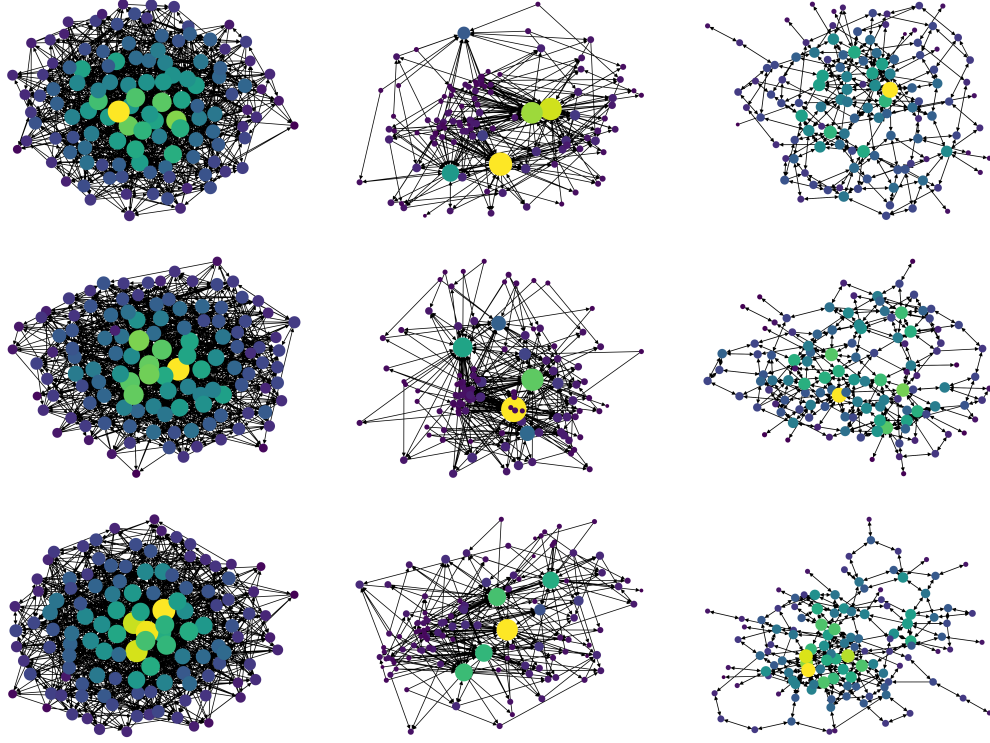


Figure 3.11.: First column: AB networks. Second column: SYN networks. Third column: WX networks. All networks consist of 120 nodes. The larger the size of the node and the brighter the node color, the higher its degree. The SYN network lies in between the AB network and the WX network in terms of degree distribution and centrality.

3.3.1.2. Test strategy

In this work, a test strategy S is a distribution of a test budget b among all network nodes. It represents a vector including n entries, where each entry shows the amount of budget (or tests) assigned to a farm: $S = (b_1, \dots, b_N)$. The total test budget B describes the maximum number of tests that can be carried out during the observation period, which is usually one year. Hence, the total test budget B equals the sum over all individual budgets

3. Methods

assigned to farms, i.e. $B = \sum_{i=1}^N b_i$. For instance, a commonly applied test strategy is to test each farm on average once per year [30, 23] which results in a total test budget equal to the total number of farms in the network ($S_{B=N} = \{b_1 = 1, \dots, b_N = 1\}$, where N is the total number of farms in the network). As stated earlier (see section 1.2), literature suggests test strategies that focus on nodes that are highly interconnected with other nodes. For instance, Ansari et al. [30] show that for SYN networks already simple heuristics such as distributing the total budget among network nodes with higher degree lead to a considerably reduction in infected animals predicted by a SI model at the time the outbreak is detected.

Thus, if we call a total test budget B and the total number of nodes in the network N , then we aim to optimise a test strategy $S_{B=N}$ so that the predicted average number of infected animals at the time the outbreak is detected becomes minimal. However, to find a balance point containing an outbreak and economic costs, we also optimise test strategies which total test budget is equal to half of the total number of nodes in the network, i.e. $S_{B=N/2}$, and which total test budget is equal to a third of the total number of nodes in the network, i.e. $S_{B=N/4}$.

3.3.1.3. SI model

By running the SI model simulation, we can predict the number of infected animals at the time the outbreak is detected for a specific transmission network and a unique test strategy. We define a number of parameters in the initialisation process of the model that have great influence on the overall dynamics of the disease spreading in the simulation. The parameter values were suggested by the epidemiologists and farm animal biologists from the overall project team:

- The transmission probability β (infectious rate in Ansari et al. [30]) is 0.5.
- The expected time of first infection t_{exp} in the simulation is 30 days. Each node in the network can become infected from outside by a probability of $1/(N * t_{exp})$, where N is the overall number of nodes in the network.
- The longest time the simulation runs is 181 days.

3. Methods

- We assume tests may be false-negative, i.e., the probability to test positive if indeed infected is 0.9.
- We assume tests can not be false positive, meaning a positive test result implies the node is infected.
- We set the number of days between becoming infected and becoming positively testable to 0 days.
- We set the number of days between becoming infected and becoming infectious to 0 days.
- We set the number of days after which an infection is automatically detected without test because of obvious symptoms (inconspicuous time) to 60 days.
- The simulation terminates when an outbreak is detected either by a positive test or automatically.

The evaluation of the SI model is computationally costly and simulations are very stochastic, i.e. the output of the simulation tends to differ substantially when executed using the same input more than once. Therefore, we make the SI model predict the average number of infected animals at the time the outbreak is detected 10,000 times for a single test strategy. We then take the average over all 10,000 simulation runs and call it the mean damage for a particular test strategy and a particular transmission network. Eventually, we compare it to the mean damage of simple heuristics, such as to test each farm on average once per year or to distribute the total test budget only among nodes with a high degree. We also compare our findings to Ansari's et al. [30] results ⁴.

3.3.1.4. Single step proximal policy optimisation

Proximal policy optimisation (PPO) is a model free, on-policy, policy gradient, and actor-critic reinforcement algorithm [19, 21, 90]. Actor-critic methods are so-called temporal-difference (TD) learning methods that benefit from separate memory structures which allow to explicitly learn the policy independently from the value function [18]. TD learning

⁴Although Ansari's et al. [30] use the same network structure (SYN), their approach differs methodological w.r.t. the SI model and distribution of the test budget during the optimisation process. Thus, a direct comparison of the results is not quite possible.

3. Methods

methods utilise ideas from Monte Carlo (MC) and dynamic programming (DP). Similar to MC methods, TD learning methods do not depend on a model of the environment's dynamics but rather learn from raw experiences (hence, model-free algorithms). In comparison to DP, they also update estimates in part on some other learned estimates [18]. TD learning utilises a bootstrapping process, where the value of state s is evaluated by a value function V and the update of the value function is based in part on an existing estimate. TD methods only wait until the next time step to update the estimates of the value function V . Thus, $V(s_t)$ is updated using the current value of $V(s_{t+1})$, where s_{t+1} is a successor state to s_t . When the agent chooses action a_t in state s_t , the system converts to state s_{t+1} and emits the reward r_{t+1} , which is immediately utilised by the agent to update the value function $V(s_t)$:

$$\begin{aligned} V(s_t) &\leftarrow (1 - \alpha)V(s_t) + \alpha(r_t + \gamma V(s_{t+1})) \\ &= V(s_t) + \alpha(r_t + \gamma V(s_{t+1}) - V(s_t)), \end{aligned} \tag{3.1}$$

where α is the learning rate, γ is a discount factor, and the term in brackets represents the TD error δ :

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{3.2}$$

The TD error is computed by the reward in the next state r_t plus the discounted state value for the current policy in the next state $\gamma V(s_{t+1})$ and the state value for the current policy in the current state $V(s_t)$ [18]. At each time it is the error in the estimate made at the time. Since the TD error at state s_t depends on the next state s_{t+1} and the next reward r_t , it is actually not available until step s_{t+1} .

In actor-critic methods, the policy or actor selects the actions, whereas the critic evaluates the actions made by the actor.

3. Methods

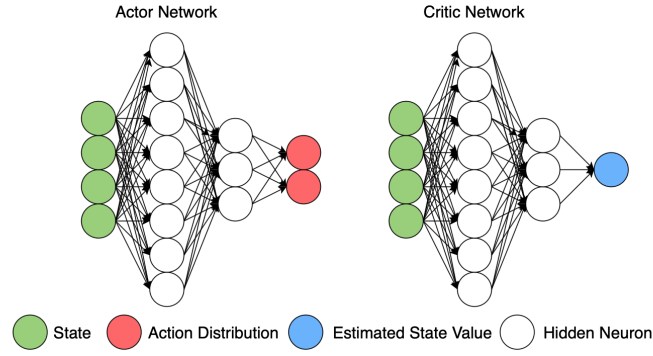


Figure 3.12.: The actor and critic networks.

The learning happens on-policy, i.e. the algorithm improves the policy used to generate the training data, and the critic exclusively learns from the current policy [18]. In general, the critic is a state-value function. After each action selected by the actor, the critic evaluates the new state. The evaluation of the state is a TD error which is a scalar signal that is responsible for all learning in both actor and critic. If the TD error is positive, action a_t is strengthened in the future, whereas if the TD error is negative, it is weakened [18]. The general architecture of actor-critic methods is shown in Figure 3.13.

3. Methods

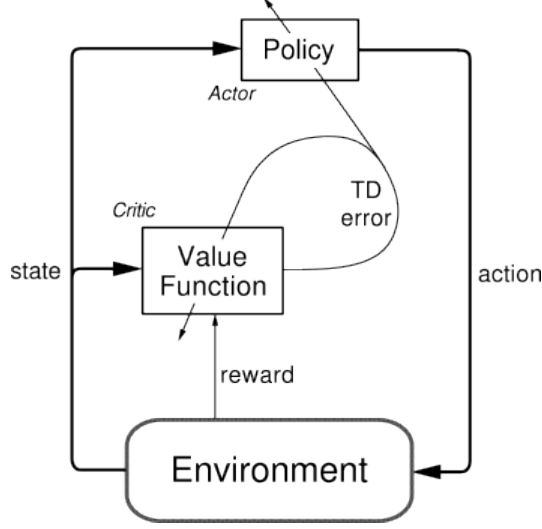


Figure 3.13.: The actor-critic architecture. In Sutton and Barto [18], Fig. 3.5.

The actor selects its actions entirely defined by a probability distribution $\pi(s, a)$ over actions given states (a policy) that is optimized by gradient ascent (equal to taking gradient descent of the negative of this function) [19]. In PPO, both the actor and critic are neural networks. The weights and biases of the networks are optimised during the learning process by backpropagation, which is described in section 3.1.2.1.

In general, policy gradient methods compute an estimator of the policy gradient and plug it into a stochastic gradient ascent algorithm. The most common policy gradient estimator $\hat{g}$ is:

$$\hat{g} = \hat{E}_t[\nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \hat{A}_t], \quad (3.3)$$

where π_{θ} is a stochastic policy, and $\hat{A}$ is an estimator of the advantage function at time step t [90]. The policy gradient estimator $\hat{g}$ is obtained by differentiating the objective function L^{PG} [90]⁵:

$$L^{PG}(\theta) = \hat{E}_t[\log \pi_{\theta}(a_t|s_t) \hat{A}] \quad (3.4)$$

⁵Although in literature [19, 21, 90] it is referred to as L^{PG} (the policy loss of the policy gradient), the goal is to maximise the value of L^{PG} . Thus, it is actually not a loss function but rather a gain function or profit function.

3. Methods

PPO is a simpler implementation of the trust region policy optimisation (TRPO) algorithm which is more sophisticated but requires a lot of tuning [90]. Trust region refers to an area surrounding the current objective in which an approximation of the true objective is recognized as valid. Taking a step within the trust region improves the current objective, whereas a step outside of the trust region makes the approximation diverge. In PPO a clipping mechanism or penalty is used which limits the rate of divergence between the new and the former policy. In TRPO, the Kullback–Leibler (KL) divergence functions as a constrain to achieve this [90].

PPO is a policy gradient method that utilises a policy gradient that is approximated by that of the policy objective function:

$$L_{PG} = E_{\tau \sim \pi} \left[\sum_{t=0}^T \log(\pi(a_t|s_t)) \widehat{A}^\pi(s, a) \right], \quad (3.5)$$

where $\tau = (s_0, a_0, \dots, s_T, a_T)$ is a trajectory of state and actions with horizon T . The advantage function A^π measures the gain associated with taking action a in state s , compared to taking the average over all possible actions, and $\widehat{A}^\pi$ is the generalized advantage estimate (GAE) [19]:

$$\widehat{A}_t = -V(s_t) + r_t + \gamma r_{t+1} + \dots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V(s_T), \quad (3.6)$$

where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$. This is the **expectation** of the TD error w.r.t. to the successor state s_{t+1} , where V is the current value function, i.e. the critic. In PPO it is called the advantage:

$$A^\pi(s_t, a_t) = r_t + \gamma V(s_{t+1}) - V(s_t), \quad (3.7)$$

where $V(s)$ represents the **expected** value of the return of the policy in state s and $\gamma \in [0, 1]$ is a discount factor that determines the trade-off between immediate and future rewards [19].

Policy updates in PPO are determined by either clipping or implementing the KL, whereas the former defines a clipping range that defines how far the new policy is allowed to go from the former, and the latter utilises a penalty to constrain the deviation from the

3. Methods

former policy in the update process [19, 90]. In this work, we apply clipping to optimise the surrogate objective function

$$E_{(s,a) \sim \pi} [\min(\frac{\pi(a|s)}{\pi_{old}(a|s)}, 1 + \epsilon \operatorname{sgn}(\hat{A}^\pi(s, a))) \hat{A}^\pi(s, a)], \quad (3.8)$$

where ϵ defines the clipping range, π the policy and $\pi(a|s)/\pi_{old}(a|s)$ is the importance sampling (IS) weight between a new and an old policy. Here, importance sampling allows to sample trajectories from the old policy while updating the new one. In general, IS gathers information about a target distribution with fewer samples from another distribution compared to i.e. a Monte-Carlo method. L_{PG} (see equation 3.5) is the objective function for the policy gradient as its derivative gives the policy gradient. To overcome the sample efficiency problem, a surrogate advantage function is formed using equation 3.8, which differentiated will result in the same gradient as equation 3.5 [90]. The surrogate advantage is a measure of how the current policy performs relative to the former one using the data from the old policy, thus it finds the advantage function for the current policy with samples from the old policy [20]. In PPO, the variance of the surrogate advantage is calculated which has the IS weight $\pi(a|s)/\pi_{old}(a|s)$ in it. If the new policy differs by a lot, the ratio becomes high, and will eventually explode. Therefore, the old samples are only used for a limited time, and re-sampled after a couple of iterations [20].

In general, a positive advantage increases the probability of taking action a in state s and vice versa [20]. However, the change must be a proportion smaller than ϵ , otherwise the term $\min$ in equation 3.8 ensures that the term following hits a ceiling of $1 + \epsilon$ or respective a floor of $1 - \epsilon$ such that no updates too far from the current policy can be made [19]. If the advantage is non-negative, it is equal to the return from the current state-action pair minus the baseline which is the TD error (or Monte-Carlo or the Q-function) [20].

In this work, we apply a rudimentary version of PPO, so-called single step PPO. It is suitable for environments in which the optimal policy is state-independent [19, 21]. In standard PPO, the task is to find the optimal set of actions a_{opt} that yield the largest cumulative reward over a number of steps in the policy space in an episode. In single step PPO, however, we seek the optimal mapping $f_{\theta_{opt}}$ such that $a_{opt} = f_{\theta_{opt}}(s_0)$, where θ is the network free parameters and s_0 is some input state that is consistently fed to the actor and critic [19, 21]. Since we seek one optimal action for a single transmission network which feature representation, i.e. the state, remains unchanged during optimisation. Thus, we

3. Methods

seek a single optimal state-action pair in single step PPO in comparison to standard PPO that is used to find an optimal trajectory of state-action transitions.

In the reinforcement process, the actor has to learn a policy to find the optimal set of actions a_{opt} in one attempt (or one interaction with the environment) per episode for the constant state s_0 . This means that the actor's task is to draw a population of actions $a_t = f_{\theta_t}(s_0)$ from the current policy, whereas the critic is computing the advantage by the returned associated rewards. Hence, the free parameters for the next populations of actions $a_{t+1} = f_{\theta_{t+1}}(s_0)$ can be updated to increase the value of the returned rewards [19, 21].

The basic principle of single step PPO follows the standard PPO method. The actor suggests a policy parameterized by the mean and variance of the probability density function of a d -dimensional multivariate normal distribution, where d is the dimension required by the environment. The actions are drawn from an action space that can be either discrete or continuous. In the following, the drawn actions are mapped into the relevant physical ranges such that it maximises the reward emitted by the environment. By limiting the number of steps in an episode to 1, the surrogate objective function in equation 3.8 becomes:

$$E_{a \sim \pi}[\min(\frac{\pi(a|s)}{\pi_{old}(a|s)}, 1 + \epsilon \operatorname{sgn}(\hat{A}^\pi(a)))\hat{A}^\pi(a)], \quad (3.9)$$

The advantage A^π is simplified to the reward r_t received by converging from state s_t to state s_{t+1} minus the state value for the current policy $V(s_t)$ since the trajectory is reduced to a single state-action pair. Thus, the value in the next state with the current policy $V(s_{t+1})$ is always zero, and the discount factor can be set to an arbitrary value with no loss of generality since the whole term becomes zero nevertheless [19]. This means, the advantage becomes simply the reward r_t minus the state value for the current policy $V(s_t)$ [19]⁶:

$$A^\pi(a_t) = r_t - V(s_t) \quad (3.10)$$

⁶Ghraieb et al. [19] mention that single step PPO does not require the value function evaluations of the critic network, making it not actually an actor-critic method. However, the critic is still required to estimate the value of the current state with the current policy $V(s)$ to compute the advantage $A^\pi = r_t - V(s_t)$.

3. Methods

In other words, this is positive if the reward received from taking action a_t is larger than the agent’s value evaluation of the current state on the basis of past rewards, as encoded in $V(s_t)$, and negative otherwise. Thus, in single step PPO, the time horizon of the trajectories has the length one, $T = 1$, and the generalised advantage estimate, see equation 3.11, becomes equal to the advantage:

$$\hat{A}^\pi = A^\pi \quad (3.11)$$

Both the actor and critic are neural networks. For that, we utilise graph neural networks that operate on network structure of the transmission networks, and which structure their computations accordingly. We describe the framework we use in this work in next section.

3.3.1.5. MetaLayer

A very general graph network (GN) framework has been proposed by Battaglia et al. [24] as a joint-work between DeepMind, GoogleBrain and MIT. It provides “a straightforward interface for manipulating structured knowledge and producing structured behaviors” [24]. In their paper, they introduce a GN block that generalizes and extends various graph neural network, message passing neural network (MPNN), and non-local neural network (NLNN) approaches [24]. In general, it allows the user to construct complex architectures from simple building blocks and it can also be implemented with functions besides neural networks, thus they call it GN block rather than graph neural network (GNN) block [24].

The GN block is the main unit in the proposed framework. It functions as a graph-to-graph module that takes graph data as input, performs computations over the structure, and returns a graph as output [24]. In this work, we utilise directed and attributed graphs. We remember from section 3.2, directed graphs have edges through which information flows one-way, i.e. from a sender node to a receiver node. In attributed graphs, the edges and nodes or vertices have attributes or features associated with them. We also assign graph-level attributes. Thus, following Battaglia et. al [24], we denote a node as $\mathbf{v}_i$, and an edge as $\mathbf{e}_k$, and the global attributes as $\mathbf{u}$. Since we deal with directed graphs, s_k indicates the sender nodes, whereas r_k indicates receiver nodes. In a graph, entities are represented

3. Methods

by the graph's nodes or vertices, relations by the edges, and system-level characteristics by global attributes [24].

In the GN framework, Battaglia et. al [24] define a graph as a 3-tuple $G = (\mathbf{u}, V, E)$, where $\mathbf{u}$ represents global attributes, i.e. in this work, they are a number of SI model parameters such as the transmission probability. Further, $V = \{v_i\}_{i=1:N^v}$ is the set of nodes (of cardinality N^v), whereby each v_i is a node's attribute. We define the node's attributes, i.e. as the node's degree, the nodes' capacity or the node's total number of live animal transmission the node receives or transmits. The $E = \{(e_k, r_k, s_k)\}_{k=1:N^e}$ represents the set of edges (of cardinality N^e), where e_k is the edge's attribute, r_k is the index of the receiver node, and s_k is the index of the sender node [24]. In this work, the edge's attribute is the total number of transmission from one node to another node.

Mathematically, a GN block consists of three *update* functions, ϕ , and three *aggregation* functions, ρ [24]. In our case, the *update* functions ϕ are neural networks (NN) and the *aggregation* functions ρ compute the elementwise average, hence [24]⁷:

$$\begin{aligned}
 \phi^e(\mathbf{e}_k, \mathbf{v}_{r_k}, \mathbf{v}_{s_k}, \mathbf{u}) &:= f^e(\mathbf{e}_k, \mathbf{v}_{r_k}, \mathbf{v}_{s_k}, \mathbf{u}) = NN_e([\mathbf{e}_k, \mathbf{v}_{r_k}, \mathbf{v}_{s_k}, \mathbf{u}]) \\
 \phi^v(\bar{\mathbf{e}}'_i, \mathbf{v}_i, \mathbf{u}) &:= f^v(\bar{\mathbf{e}}'_i, \mathbf{v}_i, \mathbf{u}) = NN_v([\bar{\mathbf{e}}'_i, \mathbf{v}_i, \mathbf{u}]) \\
 \phi^u(\bar{\mathbf{e}}', \bar{\mathbf{v}}', \mathbf{u}) &:= f^u(\bar{\mathbf{e}}', \bar{\mathbf{v}}', \mathbf{u}) = NN_u([\bar{\mathbf{e}}', \bar{\mathbf{v}}', \mathbf{u}])
 \end{aligned}$$

$$\begin{aligned}
 \rho^{e \rightarrow v}(E'_i) &= \frac{1}{N_k} \sum_{\{k:r_k=i\}} (\mathbf{e}'_k) \\
 \rho^{v \rightarrow u}(V') &= \frac{1}{N_i} \sum_i (\mathbf{v}_i) \\
 \rho^{e \rightarrow u}(E') &= \frac{1}{N_k} \sum_k (\mathbf{e}'_k)
 \end{aligned} \tag{3.12}$$

⁷See similar implementations Hamrick et al. (2018) and Sanchez-Gonzalez et al. (2018), where the aggregation function computes the elementwise summation.

3. Methods

where

$$\begin{aligned} E'_i &= \{(\mathbf{e}'_k, r_k, s_k)\}_{r_k=i, k=1:N^e} \\ V' &= \{(\mathbf{v}'_i)\}_{i=1:N^v} \\ E' &= \cup_i E'_i = \{(\mathbf{e}'_k, r_k, s_k)\}_{k=1:N^e} \end{aligned} \quad (3.13)$$

The edge *update* neural network NN_e maps across all edges such that it computes per-edge updates, whereas the node *update* neural network NN_v is mapped across all nodes such that it computes per node updates. Finally, the global-level *update* neural network NN_u is applied once as the global update. A major characteristic to the *aggregation* functions ρ are that they must be invariant to permutations to their inputs [24]. The ρ functions receive a list as an input, and emit aggregated information as a single element by applying a variable numbers of arguments (e.g., elementwise summation, mean, maximum, etc.) to the input data [24]. In this work, we generally define that ρ computes the elementwise average of the input data, and $[\mathbf{x}, \mathbf{y}, \mathbf{z}]$ indicates vector/tensor concatenation.

In practice, the update process of the graph attributes in a GN block with neural networks that we implement in our method is adapted from Battaglia et al. [24]:

1. The edge *update* function NN^e is edge-wise applied by utilising $(\mathbf{e}_k, \mathbf{v}_{r_k}, \mathbf{v}_{s_k}, \mathbf{u})$ which results in $\mathbf{e}'_k$. The outcome for the set of per-edge outputs for each node, i , is $E'_i = \{(\mathbf{e}'_k, r_k, s_k)\}_{r_k=i, k=1:N^e}$ as well as $E' = \cup_i E'_i = \{(\mathbf{e}'_k, r_k, s_k)_{k=1:N^e}\}$
2. The *aggregate* function $p^{e \rightarrow v}$ is applied to E'_i , and computes the edge updates for edges that project to vertex i , which result in the updated edge attribute $\bar{\mathbf{e}}'_i$ that is used in the next step's node update.
3. The node *update* function NN^v is applied to each node i such that it computes updated node attributes, $\bar{\mathbf{v}}'_i$. The set of the resulting per node outputs becomes: $V' = \{\bar{\mathbf{v}}'_i\}_{i=1:N^v}$.
4. The *aggregate* function $p^{e \rightarrow u}$ is applied to E' such that it computes all edge updates, into $\bar{\mathbf{e}}'$ that is used in the next step's global update.
5. The *aggregate* function $p^{v \rightarrow u}$ is applied to V' to aggregate all node updates into $\bar{\mathbf{v}}'$ which is used in the global update step.

3. Methods

6. The *update* function NN^u is applied once per graph such that it computes an update for the global attributes, $\bar{u}'$.

The update and aggregation process is shown in Figure 3.14. We implement hidden layers that are relatively large, however, fewer dimensions may work, too.

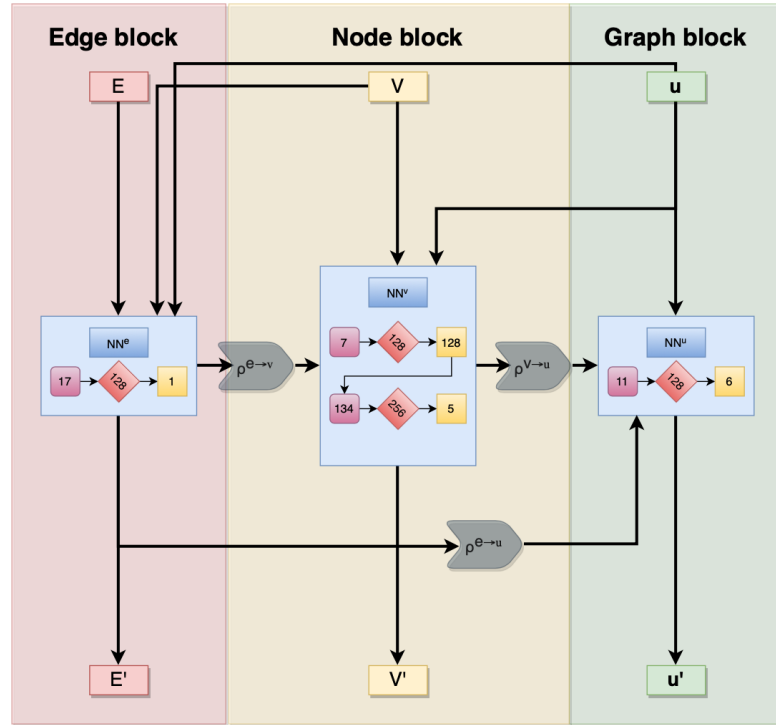


Figure 3.14.: A full GN block predicts node, edge, and global output attributes based on incoming node, edge, and global attributes. It follows the *update* and *aggregation* process that Battaglia et al. [24] introduce. The neural networks are multilayer perceptrons with different layer sizes. Adapted from Battaglia et al. [24], Fig. 4a.

3.3.2. All parts come together

In this section we are combining the methods described earlier. Regarding the implementation, PyTorch released a so called MetaLayer (ML) module [26] which represents the GN block with neural networks as the *update* function ϕ that Battaglia's et al. [24] propose.

3. Methods

We use ML in combination with stable-baselines3, which is a DRL open-source project by the German Aerospace Center (Deutsches Zentrum für Luft- und Raumfahrt) - Institute of Robotics and Mechatronics (RM) [27]. To implement single step PPO and ML, we modify stable-baselines3 PPO algorithm, and create a custom environment as well as a custom policy for both the actor and critic ⁸.

3.3.2.1. The set-up

For each transmission network we define a number of node attributes $\mathbf{v}$, edge attributes $\mathbf{e}$, and graph attributes $\mathbf{u}$.

The node attributes $\mathbf{v}$ we utilise are:

1. The degree of a node, i.e. the number of connections or links from a node to others.
2. The capacity of a node, i.e. the size of the live animal herd of a node.
3. The total number of transmissions of a node, i.e. the total number of trades from a node to others.
4. The number of incoming transmissions of a node, i.e. the number of transmissions a node receives.
5. The number of outgoing transmissions of a node, i.e. the number of transmissions a node transmits.

And the edge attribute $\mathbf{e}$:

1. The number of transmissions between two nodes.

The graph-level attributes $\mathbf{u}$:

1. The probability of an infection from outside.
2. The probability of an infection by transmission.
3. The probability to test positive.
4. delta t testable.

⁸The code is publicly accessible via GitHub: <https://github.com/pik-gane/pyoptes> [15 Jan 2023].

3. Methods

5. Δt infectious.

6. Δt symptoms.

We extract node attributes, edge attributes and graph attributes for every transmission network such that we obtain the state s_0 for each of these. In the optimisation process, the state s_0 functions as both the input for the actor and the critic. They consist of two consecutive MetaLayer blocks each, whereby their structure follows the general GN block with neural networks as shown in Figure 3.14. The exact dimensions of the neural network layers are also shown in Figure 3.14.

The action space in the DRL environment is defined to be continuous, whereas the actions drawn are in $[-100, 100]$. The observation space of the environment is unconstrained. In our model, the action selected by the actor represents an unconstrained budget allocation (i.e. an unconstrained test strategy) that has to be transformed to a constrained budget allocation to fulfill our assumptions, i.e that the total budget equals the total number of nodes in the network $B = N$ (resp. $B = N/2$ and $B = N/4$) and that the nodes that represent slaughterhouses in the network do not receive a share of the test budget, before we can feed it into the SI model.

3.3.2.2. The optimisation process

We optimise the test strategy for every transmission network separately for three total budgets, three network types, and two network sizes. To make our results more robust, for each network type and network size we generate 100 different transmission networks. However, we also create datasets consisting of networks of only a particular network type and network size to then split it into a training and test set such that we are able to evaluate how well our model works to predict an optimal test strategy in a single step for unseen transmission networks that are the same network size and network type. In a further step one may evaluate how well the model performs for unseen transmission networks with different a network size and network type.

To compute baselines, we analyse simple test strategies that literature suggest to be effective (see section 1.2), to which we refer as “naïve” heuristics in the following:

3. Methods

1. We distribute the total test budget uniformly among all network nodes, i.e. $b_t = (b_1 = 1, \dots, b_{n=N} = 1)$.
2. We concentrate the total test budget on nodes with the high degree.
3. We concentrate the total test budget on nodes with high number of incoming transmissions.
4. We concentrate the total test budget on nodes with high capacities.

If we look at each transmission network separately, the optimisation process of a test strategy or budget allocation (from now on, we call it budget allocation to be sure it is not mistaken for a strategy related to the PPO algorithm) is the following:

1. On the base of the German swine trade network, we select three graphs that differ in terms of degree distribution and centrality.
2. We create 100 transmission networks for each graph type and graph size.
3. The node attributes, edge attribute, and graph attributes are derived from the network structure of a particular transmission network.
4. We compute the z-scores for each attribute by subtracting the attribute's mean from an individual raw score and then dividing the difference by the attribute's standard deviation.
5. We concatenate the standardized attributes of each transmission network such that we obtain s_0 .
6. The iterative optimisation process begins:
 - a) We pick one transmission network and derive the node, edge and graph attributes from it to obtain the state s_0 . We then feed the state s_0 to the actor that draws a population of actions $a_t = f_{\theta_t}(s_0)$ from its policy θ_t , where $a_t = (a_1, \dots, a_N) \in [-100, 100]^N$.
 - b) The population of actions a_t is transformed to a budget allocation b_t that corresponds to the assumptions we made:
 - $b_s = 0$, where s identifies nodes that represent slaughterhouses, and slaughterhouses do not receive a budget share.

3. Methods

- $b_t = \{b_1, \dots, b_N\} \in [0, B]^N$, the budget share a node receives lies in between zero and the amount of the total budget.
 - $\sum b_t = \sum_{i=1}^N b_i = B$, the sum of all budget shares is equal to the predetermined amount of the total budget.
- c) The budget allocation and the source transmission network is fed to the SI model.
 - d) The SI model evaluates the budget allocation and predicts the average number of infected animals for the budget allocation and its corresponding source transmission network. By running the simulation 10,000 times and then taking the estimated average over all simulation runs, we estimate the damage d_t .
 - e) We compute the reward $r_t = d_{t-1} - d_t$, where d_t is current damage for the current population of actions or the current policy a_t . For $t = 0$, the initial damage is the damage for the baseline, i.e. the test strategy to distribute the total test budget B uniformly among all nodes in the network.
 - f) In single step PPO the GAE is equal to the advantage which is equal to the reward minus the value estimation of the current, i.e. $\hat{A}^\pi = A_t^\pi = r_t - V(s_t)$.
 - g) The surrogate objective function L , see equation 3.9, w.r.t. θ_t (the current policy) is optimised.
 - h) The current policy θ_t is replaced by the new policy θ_{t+1} .

The optimisation process for a single transmission network is shown in Figure 3.15.

3. Methods

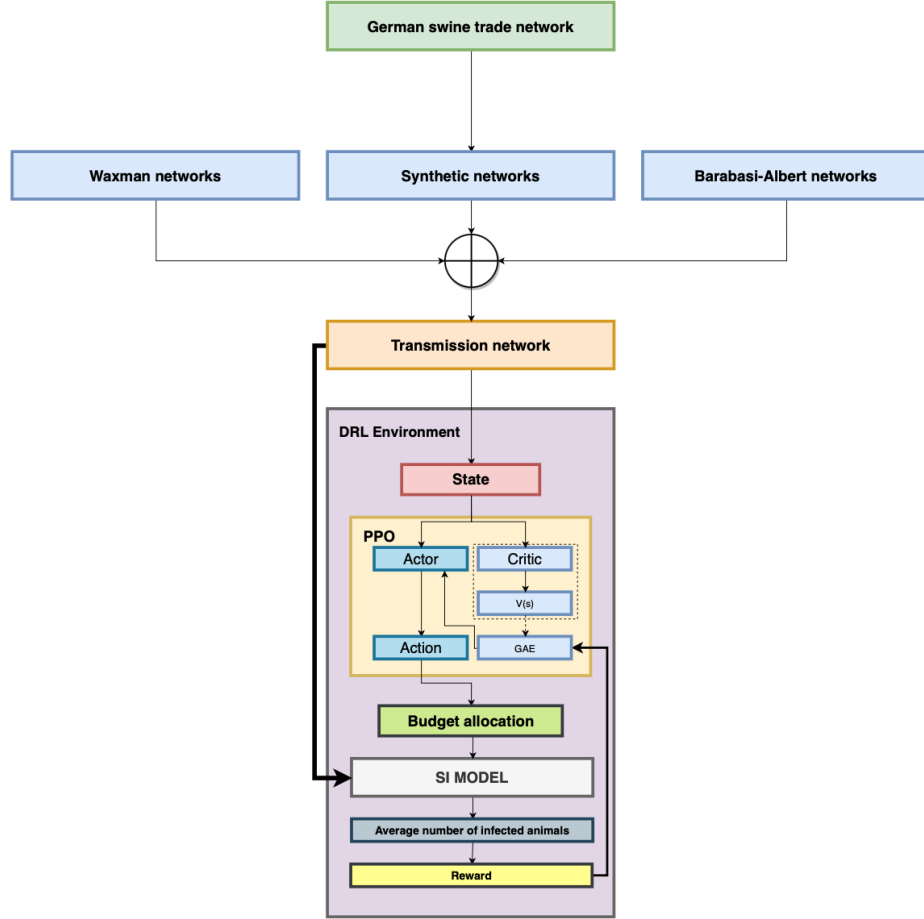


Figure 3.15.: The optimisation process for a single transmission network.

3.3.2.3. Proximal policy optimisation parameters

PPO has a number of parameters that play a central role in the process of making the algorithm converge. Due to computational costs and time restraints we focus on the most important ones and leave the rest on default, see the stable-baselines3 documentation regarding the PPO implementation [27]:

- The learning rate is $10^{-5.5}$ and decreases further during training.

3. Methods

- In PPO, we gather trajectories over a predetermined time horizon. The number of steps to run for each environment per update (compute the general advantage estimate and adjust the weights of the neural network) is set to 16. As an optimiser, we use Adam.⁹
- The size of the mini-batches per update is equal to the number of steps (=16) for each environment update, hence we process the whole batch all at once.
- The entropy coefficient for the loss calculation is set to 0.001.
- The discount factor gamma is set to $\gamma = 1$, however, it can be set arbitrarily in single step PPO since it has no effect on the advantage.
- The factor that balances exploration and exploitation is called $GAE(\gamma, \lambda)$ and it is set to 0.95 as recommended.
- The action space is constrained in $a_{space} = [-100, 100]$ and the observation space unconstrained in $o_{space} = (-\infty, \infty)$.
- For a particular test strategy the SI model runs 10,000 simulations and takes the estimated average of the numbers of infected animals over all runs.
- For networks with 120 nodes we do 10,000 episodes (steps) in the environment, whereas for networks with 1040 nodes we do 50,000 episodes (steps).
- Computations were done on 16 cores (2x Intel Xeon E5-2667 v3 8C, 3.4GHz, 20MB Cache, 135W processors) - Intel Haswell architecture ¹⁰.

⁹Adam is an algorithm for first-order gradient-based optimisation of stochastic objective functions, based on adaptive estimates of lower-order moments introduced by Kingma and Ba [91].

¹⁰A full documentation of the hardware specifications can be found here: <https://www.pik-potsdam.de/en/institute/about/it-services/hpc/#section-3>.

4. Results

As a first step, we analyse the effectiveness of simple testing heuristics suggested by literature (see section 1.2) to contain epidemics of infectious diseases in live trade animal networks. Therefore, we focus on four strategies, i.e. to distribute the test budget on nodes with high degree (HD), nodes with high number of incoming transmissions (HIT), nodes with high capacities (HC) in the network, and the commonly applied test strategy to test every farm in the network on average once per year (UNI) that we define as the baseline. We compare the findings with the results of Ansari’s et al. [30] test strategy optimisation method utilising MCMC Metropolis–Hastings and SA. Eventually, we show that the results correspond with the outcome of our optimisation method in which we combine deep reinforcement learning with graph neural networks. However, direct comparison in terms of the mean damage between Ansari’s et al. [30] MCMC Metropolis–Hastings and SA method and our approach is not possible due to small differences regarding the SI model. Also, Ansari et al. [30] seek an optimal set of nodes among which they distribute the total test budget uniformly, and the number of nodes in the set is predetermined before the optimisation process, whereas the optimal distribution of the total budget we seek is only constrained by the assumptions we made in section 3.3.2.2.

4.0.1. “Naïve” heuristics

As literature suggests (see section 1.2), distributing a test budget among a particular set of farms with specific properties (high degree, high capacities, high number of transmissions) in a live animal trade network may be more beneficial in terms of surveillance and prevention of epidemic outbreaks of infectious diseases compared to other common heuristics, i.e. testing each farm on average once per year. To confirm that the proposals also correspond to the transmissions networks we generated, we analyse the effectiveness of the

4. Results

four “naïve” heuristics. Therefore, we evaluate the heuristics by SI model simulations as described in section 3.3.1.3 for different numbers of nodes.

The results for the “naïve” test strategies are shown in Figures A.9 – A.11 in section A.3 in the appendix. In general, both the test strategies HD and HIT perform considerably better in terms of the estimated average number of infected animals (= mean damage) compared to UNI if we concentrate the test budget on a few nodes. However, the difference of the overall mean damage between UNI, HD, and HIT remains relatively small for the less centralised WX transmission networks. We can also see that the larger the AB and SYN networks the more effective is HD and HIT compared to UNI, whereas it is the opposite for the WX networks. To find a sweet spot that balances effectiveness and economic costs, we decrease the total budget to $B = N/2$ and evaluate the “naïve” test strategies. Our goal is to find a test strategy that is at least as effective as $UNI_{S_{B=N}}$ but requires only a portion of the baseline total budget. With a total budget equal to half of the total number of nodes in the network, $HD_{S_{B=N/2}}$ and $HIT_{S_{B=N/2}}$ still perform better in comparison to $UNI_{S_{B=N}}$ for small (120) and large (1040) SYN networks as well as large (1040) AB networks. However, for small (120) WX and AB networks and large (1040) WX networks, $HD_{S_{B=N/2}}$ and $HIT_{S_{B=N/2}}$ are not as effective as $UNI_{S_{B=N}}$, see Figures A.12 – A.14 in section A.3 in the appendix.

If we reduce the total budget even further to $B = N/4$ and evaluate the “naïve” test strategies once again, $HD_{S_{B=N/4}}$ and $HIT_{S_{B=N/4}}$ keep performing better in comparison to $UNI_{S_{B=N}}$ for small (120) and large (1040) SYN networks as well as large (1040) AB networks. However, for small (120) and large (1040) WX networks and small (120) AB networks, $HD_{S_{B=N/4}}$ and $HIT_{S_{B=N/4}}$ are not as effective as $UNI_{S_{B=N}}$, see Figures A.15 – A.17 in section A.3 in the appendix.

4.0.2. Deep reinforcement learning and graph neural networks

For the optimisation process, we run the algorithm 10,000 episodes for networks with 120 nodes, and 100,000 episodes (50,000 for WX and AB due to time limits) for networks with 1040 nodes. In contrast to Ansari et al. [30], we exclude networks with 57,590 nodes due to computational costs, see section 2. However, we expand our analysis to also include WX and AB networks, and allow the budget to be distributed among all nodes, except slaughterhouses. The optimisation process for a single network with 120 nodes takes

4. Results

about 8 hours, whereas it takes about 75 hours for a network with 1040 nodes (see section 3.3.2.3 for details regarding the hardware). We face high computational costs since we evaluate a test strategy by taking the average over 10,000 SI simulations in every single episode. The learning progress in terms of the estimated mean damage over all networks with 120 and 1040 nodes is shown in Figures A.6–A.8 in section A.2 in the appendix.

By optimising testing strategies we are able to decrease the overall mean damage for each network type (SYN, WX, AB), each network size (120, 1040), and each total budget ($B = N, B = N/2, B = N/4$) compared to the baseline testing scheme UNI and the best “naïve” heuristics HD and HIT if all testing strategies have the same total budget B . Although the found optimal test budget distributions perform better compared to the “naïve” heuristics introduced in section 4.0.1, the improvements are relatively small, see Table 4.1, and for a more extensive analysis A.18–A.20 in section A.4 in the appendix.

Network type	Zero	UNI	Best “naïve” strategy	$S_{B=N}^*$	$S_{B=N/2}^*$	$S_{B=N/4}^*$
SYN _{N=120}	4780 (41)	3795 (36)	3030 (32)	2792 (30)	3269 (34)	3750 (36)
SYN _{N=1040}	6731 (62)	4928 (48)	3605 (35)	3441 (34)	4280 (38)	4789 (41)
WX _{N=120}	3943 (40)	2986 (31)	2754 (29)	2451 (27)	2892 (30)	3263 (33)
WX _{N=1040}	5639 (84)	3831 (69)	3691 (67)	3296 (61)	4054 (71)	4415 (73)
AB _{N=120}	4863 (52)	3502 (37)	3135 (33)	2862 (29)	3452 (36)	3959 (44)
AB _{N=1040}	7669 (135)	4629 (68)	3224 (51)	3039 (48)	3940 (59)	4493 (65)

Table 4.1.: Performances in terms of the mean damage of optimised strategies $S_{B=N}^*$, $S_{B=N/2}^*$, and $S_{B=N/4}^*$ in comparison to the baselines for all network types and network sizes and the respective standard deviation in parentheses.

If we reduce the total budget to $B = N/2$, and $B = N/4$, the found testing schemes keep performing better in comparison to $UNI_{S_{B=N}}$ for small (120) and large (1040) SYN networks as well as large (1040) AB networks. If the budget is reduced to $B = N/2$, they still perform better in small (120) AB and small (120) WX networks compared to $UNI_{S_{B=N}}$. However, for large (1040) WX networks, $B = N/2$, and $B = N/4$, they are not as effective as the UNI testing scheme with baseline budget $UNI_{S_{B=N}}$, see Table 4.1, and for a more extensive analysis Figures A.18 – A.20 in section A.4 in the appendix.

4. Results

To measure the effective number of nodes that receive a budget, we compute a form of the Rényi entropy¹ e_b that measures how widely the budget is spread among the nodes in the network:

$$e_b = \frac{1}{\sum (b^*/N)^2} \quad (4.1)$$

The vector b^* represents the fund budget allocation, and N is the total number of nodes in the network. The term b^*/N is a (probability) distribution if the total budget equals N . If squared and summed up, it can be written as $\sum (b^*/N)^2 = 1/k$, where k represents the effective number of nodes that receive a budget. Hence, if the budget is only assigned to a single node in the network, e_b is 1. If the budget is split uniformly among two nodes, it becomes $e_b = \frac{1}{1/2} = 2$. For instance, as shown for all SYN and WX networks with 1040 nodes in Figure 4.1, the agent continuously decreases e_b , eventually focusing the total budget on relatively few nodes. Simultaneously, we measure the correlation² between the assigned budget of each node and each node's number of incoming transmissions during the optimisation process in each episode for every SYN network with 1040 nodes. We notice that the budget share assigned to a node shows a moderately positive correlation with the node's number of incoming transmissions over time, i.e. $r \sim 0.5$. However, this does not apply to AB networks with 1040 nodes, where we notice only a low correlation between the two variables budget share and number of incoming transmissions.

¹The Rényi entropy was introduced by Alfréd Rényi [92]. Entropy and relative entropy are core concepts of information theory. The entropy of a system gives a measure of uncertainty about its actual structure [93]. The actual Rényi entropy would be the log of our measure e_b

²The Pearson correlation coefficient measures the ratio between the covariance of two variables and the product of their standard deviations [94].

4. Results

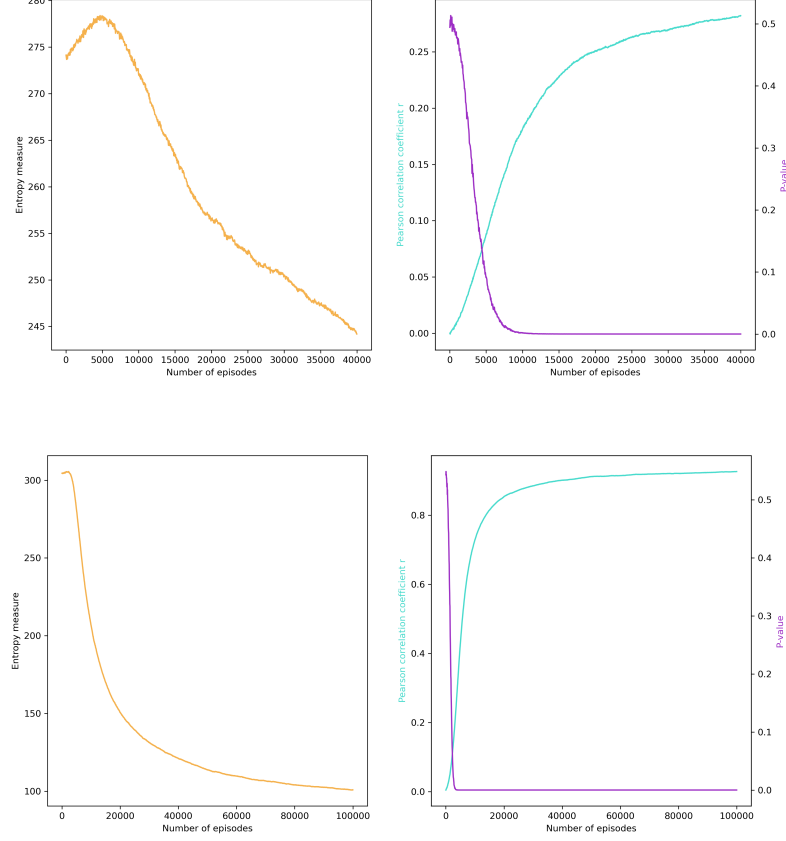


Figure 4.1.: The mean entropy measure, mean pearson correlation coefficient and the corresponding mean p-value over all WX networks (top) and SYN networks (bottom) with 1040 nodes during the optimisation process. While the effective number of nodes that receive budget shares decreases over time, the correlation between the nodes' assigned budget shares and their numbers of incoming transmissions increases.

If we visualise the first test budget allocation the agent selects in comparison to its selected test strategy in the last episode for AB, WX, and SYN networks, we see that initially the budgets are allocated without real structure. When looking at the final test strategies, we see that in modular networks (SYN, WX), budget shares are distributed more heterogeneously compared to the rather centralised AB networks for which the budget shares are almost solely distributed among nodes with high degree in the center of the networks, see Figure 4.2.

4. Results

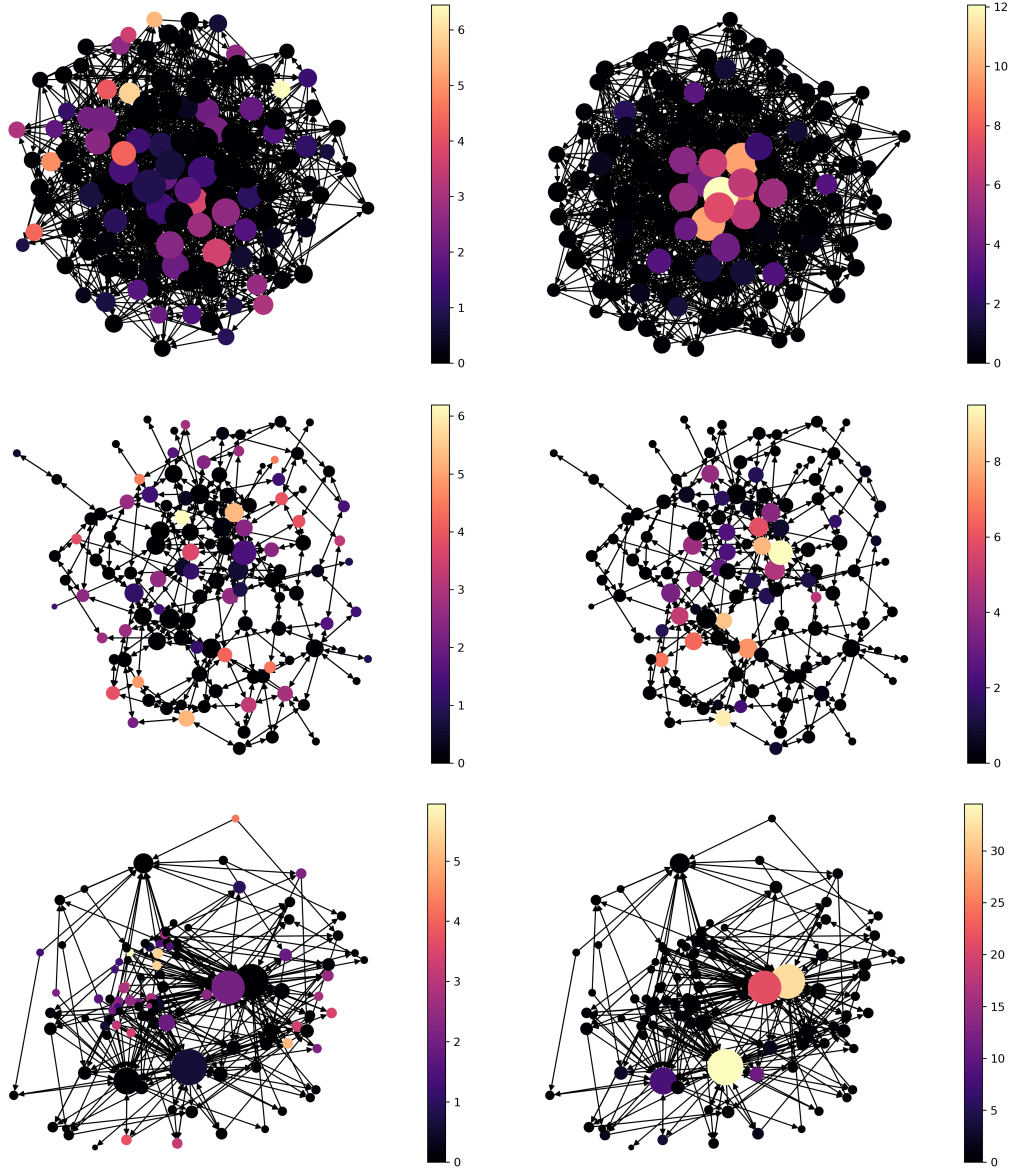


Figure 4.2.: The first test budget allocation selected by the agent (left) and the test budget allocation selected in the last episode (right) for the first samples of the transmission networks ensemble with 120 nodes. The first row shows the first AB network, the second row shows the first WX network, and the first SYN network is shown in the last row.

4. Results

For further analysis, we visualise the optimal budget share distribution for each total budget $B = N, B = N/2, B = N/4$ and each network type with 120 nodes for the second sample of every network ensemble, see Figure 4.3. Again, we see that networks nodes with high degree receive most budget shares. In the less centralised and more modular WX and SYN networks, the nodes receiving budget shares are more spread out and with mixed degree. In contrast to AB and WX, significantly fewer nodes receive budget shares in the SYN network. Also it seems that although reducing the total budget B , the nodes receiving budget shares almost are the same as with baseline budget $B = N$.

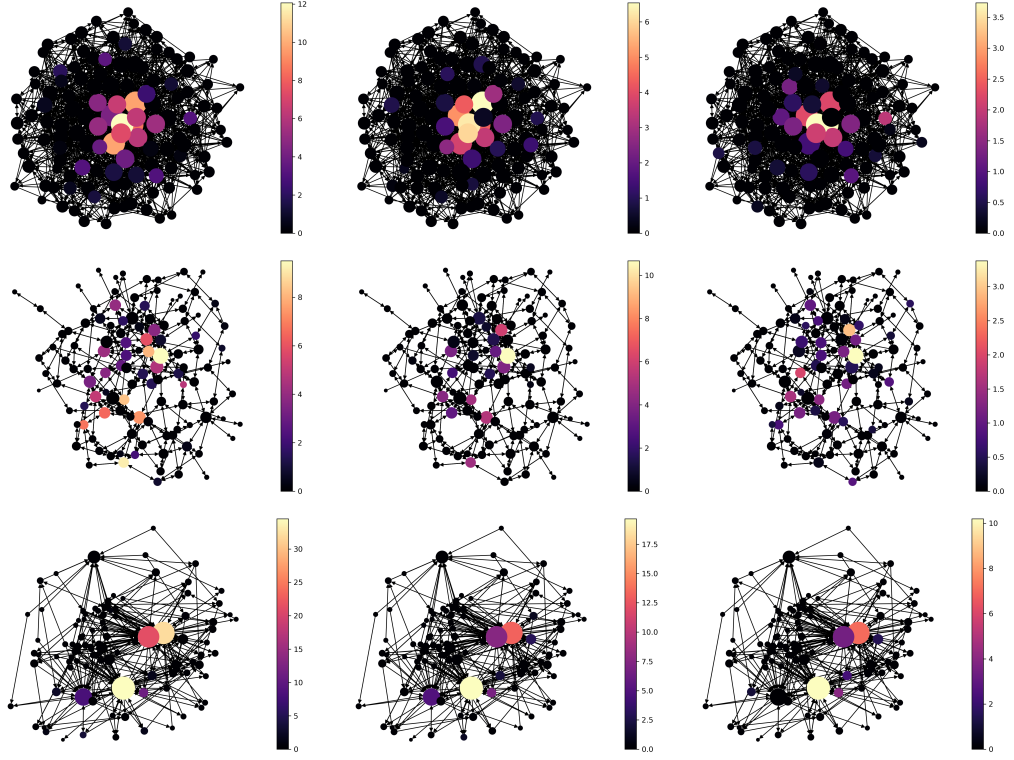


Figure 4.3.: The optimised budget allocations for the first sample from the transmission networks ensembles with 120 nodes. From left to right the budgets are $B = N, B = N/2$, and $B = N/4$. The node color represents the node's budget share received, whereas the larger the node size the greater its degree. The first AB network (top), the first WX network (middle), and the first SYN network (bottom).

In general, our results correspond to Ansari's et al. findings that by applying machine learning, it is possible to find test strategies that reduce the estimated average number

4. Results

of infected animals in comparison to prevailing test policies [30]. However, it is worth underlining that we seek a test budget that may include all nodes compared to Ansari's et al. approach to find a set of "sentinel nodes", where the size of the set is predetermined and constant, and the budget is distributed uniformly. Their findings suggest that in terms of the optimal number of sentinel nodes, the best choice appears to be $k = N/12$ for SYN networks with 120 nodes, while for SYN networks with 1040 nodes and real networks with 57,590 nodes it is $k = N/52$, where N is the number of total nodes in the network, and k is the number of sentinel nodes. Ansari et al. assign budget shares to significant fewer nodes in comparison to the results of the earlier introduced entropy measure for our found budget allocations. Whereas the nodes with the highest numbers of incoming transmissions receive the most test budget shares in both methods, the algorithm we apply distributes the test budget more widely and unevenly. Due to differences in the SI model simulations, we can not compare the estimated mean damage of the test strategies we found by our approach directly to the effectiveness of Ansari's et al. method [30].

However, our results suggest that animal trade market participants with many links to other participants play a major role in spreading dynamics. Gradually reducing the total test budget available reveals that the optimisation algorithm concentrates the budget on almost the same nodes, only making minor re-distributions. In particular, participants with the highest number of incoming transmissions or a very high number of both incoming and outgoing connections receive the largest test budget shares. These farms also appear to be epicenters in the network, being surrounded by multiple other participants and may be part of communities in the less centralised and more modular SYN and AB networks. Some network participants with fewer connections, that lie between two epicenters, connecting nodes with large degree, also receive budget shares that are significantly smaller. Thus, test strategies that not only concentrate budget shares on a few but highly connected participants but also assign budget shares to nodes that may have less degree and fewer number of incoming transmissions, lead to the best overall performances in terms of estimated average numbers of infected animals.

4. Results

4.0.3. Evaluation of PPO and SI model parameters, and the performance of MetaLayer

We explain the influence of a number of PPO and SI model parameters in the following, and eventually compare the performance of GNNs (MetaLayer) and simple MLPs in terms of the mean damage for SYN networks with 120 and 1040 nodes.

4.0.3.1. PPO parameters

When evaluating a number of single step PPO parameters, we can see that:

- The algorithm appears to converge best if the learning rate is $10^{-5.5}$ w.r.t MetaLayer as both actor and critic.
- The entropy coefficient for the loss calculation is set to 0.001, however, entropy coefficients from 0.0001 to 0.01 lead to similar results.
- The factor that balances exploration and exploitation is set to 0.95 as recommended, increasing it to 0.99 does not significantly influence the outcome.
- For a particular test strategy the SI model runs 10,000 simulations and takes the estimated average of the numbers of infected animals over all runs. If the number of simulations is reduced, we increase the standard deviation of the mean damage.
- Due to computational costs, we do 10,000 episodes (steps) in the environment for small networks and 50,000 to 100,000 steps for networks with 1040 nodes. In general, the number of steps is relatively small in comparison to Ansari's et al. [30] MCMC and SA approach making 1,000,000 steps in the optimisation process.
- Even if we reduce the dimensions of the hidden layers in the Edge-model, Node-model, and Graph-model in the MetaLayer framework to be only twice as large as the dimension of their respective input layer, we still achieve similar results to the layer dimensions that are much larger, for instance as shown in chapter 3.3.1.5 in Figure 3.14.

4. Results

4.0.3.2. SI model parameters

By defining the inconspicuous time in the SI model simulations, we determine the period between the beginning of the infection and the appearance of symptoms that reveal the carrier of an infectious disease which causes an immediate detection without a test, i.e. making the simulation stop, see also Ansari et al. [30]. Since infectious diseases, and in particular zoonotic ones have different inconspicuous times, we analyse the effectiveness of a number of “naïve” testing strategies on the outbreak size for varying inconspicuous times. We show that by increasing the time until an infection becomes obvious, the more testing heuristics such as HD and HIT outperform the baseline UNI. Thus, testing strategies that concentrate budget shares on a number of nodes with HD or HIT appear more effective when the inconspicuous time of an infectious disease lies roughly above 30 days, see Figure 4.4. Ansari et al. show similar findings for their SI model [30].

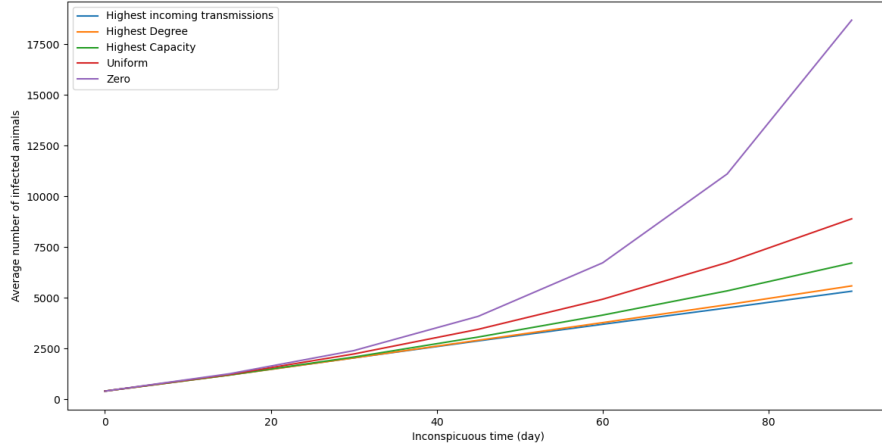


Figure 4.4.: The performance of “naïve” testing strategies for varying inconspicuous times for one SYN transmission network with 1040 nodes. The results for AB and WX networks are similar.

4.0.3.3. GNNs and MLPs

Instead of graph neural networks, we also can use simple multilayer perceptrons (MLPs) as actor and critic in PPO. In this case, the state represents a test strategy or test budget

4. Results

allocation, thus neither the actor nor the critic receive any information regarding the topological structure of the transmission network. We show that for the purpose of optimising a test strategy for a single transmission network, it is sufficient to apply MLPs as both actor and critic for networks with 120 and 1040 nodes, see Figure 4.5. However, we lose the ability to process graphs different sizes since MLPs are not scale invariant like GNNs, hence we can not generalise from small to large graph structured data.

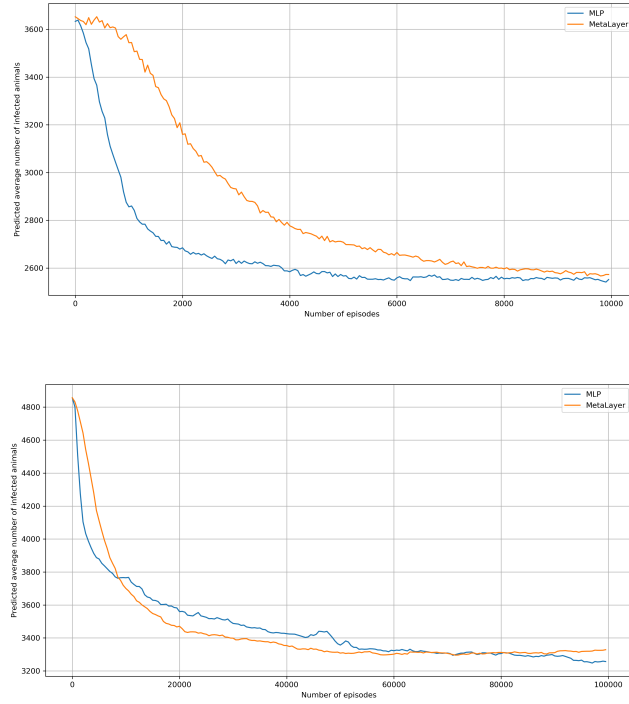


Figure 4.5.: The performance of GNNs (MetaLayer) as both actor and critic in comparison to MLPs. Top: The mean damage for a SYN network with 120 nodes. Bottom: The mean damage for a SYN network with 1040 nodes. We apply MLPs with one hidden layer that has twice the dimension of the input layer for both actor and critic.

5. Discussion

Computational simulations modeling complex and dynamic real world phenomena play an increasingly important role for stakeholders and policy-makers who rely on accurate measurements of uncertainty. In particular, given the SARS-CoV-2 virus pandemic, state authorities are driven towards implementing effective countermeasures for preventing and containing future outbreaks of infectious diseases as early as possible. Since most epidemic outbreaks originate in animals, finding efficient test strategies for infectious diseases in live animal trade networks can be of great help in this regard. In this thesis, we investigated an alternative approach to conventional blackbox optimisation algorithms by utilising deep reinforcement learning and graph neural networks. While proximal policy optimisation is a state-of-the-art deep reinforcement algorithm that has gained a lot of attention in recent years, literature researching its combination with graph neural networks is rare.

5.1. The findings and the curse of dimensionality

As we showed, by connecting both approaches it is possible to find “close-to-optimal” testing strategies for graph structured data that outperform “naïve” or prevailing testing schemes in AB, WX and SYN transmission networks in terms of mean damage up to 35%. However, the improvements in comparison to the best “naïve” strategies HD and HIT remain small, which can either be due to the fact that

1. they are already “close-to-optimal” in this specific application case, and hence the optimisation process was successful in finding slightly better solutions or
2. the optimisation algorithm is not suitable for finding “close-to-optimal” solutions to high dimensional problems, and we are far off. In non-convex problems, we can

5. Discussion

assume that we always end up in local optima. Particularly in high-dimensional spaces, finding the global optima is very unlikely, i.e. "the curse of dimensionality" [68].

5.2. The findings in context of related work

If we take findings from social network analysis, and in particular research regarding swine trade networks into account (see section 1.2), it has already been shown that test strategies that focus on network participants with many links to other participants or on participants with a high number of animal transmissions appear to be effective in several swine trade networks to contain epidemics. Our findings, indeed, suggest that by concentrating test budget allocations on network nodes that possess such characteristics, it is possible to reduce damages in terms of the average number of infected animals, i.e. the outbreak size. A direct comparison to Ansari's et al. [30] results in terms of the mean damage is not possible, however, their conclusions are consistent with our results regarding SYN networks. Although they focus on seeking an optimal set of sentinel nodes by utilising MCMC and SA, while we pursue a more general approach distributing the budget among all nodes in the network, both approaches eventually lead to test budget distributions that mainly focus on highly connected nodes while simultaneously assigning smaller budget shares to nodes connecting hubs. However, in comparison to Ansari et al., the test budgets we found are spread more widely and more unevenly distributed.

5.3. The effect of reducing the test budget

Most of all, the findings of this thesis show that even if the budget is reduced drastically up to a quarter of the baseline total budget, by utilising DRL and GNN, we are still able to reduce the mean damage substantially, and even as much as the prevailing test strategy that test every farm on average once per year. If there were no trade data at all, it appears that often it is already sufficient to apply test strategies that focus on a number of highly connected nodes ("naïve" testing) that have many incoming animal transmissions to reduce the average number of infected animals considerably. If we compare the findings with regard to the respective network topology, we see that nodes with a high degree

5. Discussion

and a high number of incoming transmissions receive the largest budget shares for every network size and network types. However, for centralised networks (AB) it appears to be sufficient to assign test budgets mainly to nodes that have a high degree and/or a high number of incoming transmissions, while a more widely spread test budget performs best in more modular structured ones (SYN, WX).

5.4. Limitations of our approach and future work

Still since the scope of the study itself is limited, and the literature that focuses on similar research is rare, efficiency measurements must be taken into account with care. For future work, we suggest the following broadening of the methods introduced in this thesis:

1. Although the transmissions data was available, we were not able to optimise test strategies for the real German swine network that is much larger (57,590 nodes) compared to the small (120 nodes) and the large (1040 nodes) transmission networks we worked with due to computational costs. We highly suggest analysing deep reinforcement learning optimisation methods for networks that are higher dimensional.
2. PPO is a state-of-the-art deep reinforcement learning algorithm, however, a number of actor-critic algorithms such as A2C, SAC, and A3C have been proposed more recently [27]. Since research regarding single step PPO is rare, the performances of alternative algorithms in comparison to single step PPO should be investigated further.
3. In this thesis, we seek test strategies for transmissions networks one after the other. However, graph neural networks are particularly well suited for generalisation tasks. In a further step, we highly suggest to evaluate the potential of deep reinforcement learning and graph neural networks to find optimal test strategies by training a model for a number of transmission networks such that the agent is able to predict “close-to-optimal” test strategies for unseen transmission networks that may be of different network type and network size.
4. Single step PPO is a model-free on-policy deep reinforcement learning algorithm that purely samples from experience and relies on samples acquired by interaction

5. Discussion

with the environment. In a further step to reduce the computational costs arising by evaluating the SI model in every step in each episode, it may be beneficial to apply model-based algorithms that are able to learn the dynamics of the simulations of the SI model by creating a model of it during the agent-environment interaction process. However, it may be difficult to learn accurate representations due to the very stochastic predictions of the mean damage by the SI model.

In summary, as similar studies show [19, 21], deep reinforcement learning may be an alternative for blackbox optimisation in particular cases. However, since only recently it has been given much more attention, literature that focuses on researching possible applications is rare. By these findings, we hope to make a contribution to future research in this regard.

Bibliography

- [1] Sara Ansari et al. “A Temporal Network Model for Livestock Trade Systems”. In: *Frontiers in veterinary science* 8 (2021) (cit. on pp. ii, 1, 2, 5, 6, 7, 10, 12, 18, 24, 39, 86).
- [2] Stephen Eubank et al. “Modeling Disease Outbreaks in Realistic Urban Social Networks”. In: *Nature* 429 (June 2004), pp. 180–4. DOI: 10.1038/nature02541 (cit. on pp. ii, 7).
- [3] S. Rautureau, B. Dufour, and B. Durand. “Structural vulnerability of the French swine industry trade network to the spread of infectious diseases”. In: *animal* 6.7 (2012), pp. 1152–1162. DOI: 10.1017/S1751731111002631 (cit. on pp. 1, 5, 6, 7).
- [4] Stephen Morse et al. “Prediction and prevention of the next pandemic zoonosis”. In: *Lancet* 380 (Dec. 2012), pp. 1956–65. DOI: 10.1016/S0140-6736(12)61684-5 (cit. on p. 2).
- [5] M.H. Hilderink and I.I. de Winter. “No need to beat around the bushmeat—The role of wildlife trade and conservation initiatives in the emergence of zoonotic diseases”. In: *Heliyon* 7.7 (2021), e07692. ISSN: 2405-8440. DOI: <https://doi.org/10.1016/j.heliyon.2021.e07692>. URL: <https://www.sciencedirect.com/science/article/pii/S2405844021017953> (cit. on p. 2).
- [6] Gijs Klous et al. “Human–livestock contacts and their relationship to transmission of zoonotic pathogens, a systematic review of literature”. In: *One Health* 2 (2016), pp. 65–76. ISSN: 2352-7714. DOI: <https://doi.org/10.1016/j.onehlt.2016.03.001> (cit. on p. 2).
- [7] Maria Cristina Rulli et al. “The nexus between forest fragmentation in Africa and Ebola virus disease outbreaks”. In: *Scientific Reports* 7.1 (Feb. 2017), p. 41613. ISSN: 2045-2322. DOI: 10.1038/srep41613. URL: <https://doi.org/10.1038/srep41613> (cit. on p. 2).

Bibliography

- [8] Jason Rohr et al. “Emerging human infectious diseases and the links to global food production”. In: *Nature Sustainability* 2 (2019), pp. 445–456 (cit. on p. 3).
- [9] Oliver Bent. “Machine learning applied to prediction, control and planning from dynamic epidemiological models”. dissertation. University of Oxford, 2020. URL: https://www.robots.ox.ac.uk/~parg/pubs/theses/oliver_bent_thesis.pdf (cit. on pp. 3, 9).
- [10] W. O. Kermack and A. G. McKendrick. “A Contribution to the Mathematical Theory of Epidemics”. In: *Proceedings of the Royal Society of London. Series A, Containing Papers of a Mathematical and Physical Character* 115.772 (1927), pp. 700–721. ISSN: 09501207. URL: <http://www.jstor.org/stable/94815> (visited on 10/13/2022) (cit. on pp. 3, 15).
- [11] Charles Audet. “A Survey on Direct Search Methods for Blackbox Optimization and their Applications”. In: 2014 (cit. on pp. 3, 21).
- [12] C. Audet and W. Hare. *Derivative-Free and Blackbox Optimization*. Springer Series in Operations Research and Financial Engineering. Springer International Publishing, 2018. ISBN: 9783319886800. URL: <https://books.google.de/books?id=Csv5wQEACAAJ> (cit. on pp. 3, 21, 22).
- [13] Donald R. Jones, Matthias Schonlau, and William J. Welch. “Efficient Global Optimization of Expensive Black-Box Functions”. In: *Journal of Global Optimization* 13 (1998), pp. 455–492 (cit. on p. 3).
- [14] Jeffrey Larson, Matt Menickelly, and Stefan M Wild. “Derivative-free optimization methods”. In: *Acta Numerica* 28 (2019), pp. 287–404 (cit. on p. 3).
- [15] Elad Sarafian et al. “Explicit Gradient Learning for Black-Box Optimization”. In: *Proceedings of the 37th International Conference on Machine Learning*. Ed. by Hal Daumé III and Aarti Singh. Vol. 119. Proceedings of Machine Learning Research. PMLR, 13–18 Jul 2020, pp. 8480–8490. URL: <https://proceedings.mlr.press/v119/sarafian20a.html> (cit. on pp. 3, 21, 30).
- [16] Charles Audet and Michael Kokkolaras. “Blackbox and derivative-free optimization: theory, algorithms and applications”. In: *Optimization and Engineering* 17.1 (Mar. 2016), pp. 1–2. ISSN: 1573-2924. DOI: 10.1007/s11081-016-9307-4. URL: <https://doi.org/10.1007/s11081-016-9307-4> (cit. on pp. 3, 21).

Bibliography

- [17] Yutian Chen et al. *Learning to Learn without Gradient Descent by Gradient Descent*. 2016. DOI: 10.48550/ARXIV.1611.03824. URL: <https://arxiv.org/abs/1611.03824> (cit. on p. 3).
- [18] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second. The MIT Press, 2018. URL: <http://incompleteideas.net/book/the-book-2nd.html> (cit. on pp. 4, 19, 35, 36, 37, 38, 44, 45, 46, 47).
- [19] Hassan Ghraieb et al. “Single-step deep reinforcement learning for open-loop control of laminar and turbulent flows”. In: *Physical Review Fluids* (2021). URL: <https://hal.archives-ouvertes.fr/hal-03027908> (cit. on pp. 4, 5, 10, 38, 44, 47, 48, 49, 50, 75).
- [20] Hao Dong et al. *Deep Reinforcement Learning: Fundamentals, Research, and Applications*. Ed. by Shanghang Zhang Hao Dong Zihan Ding. Springer Nature (cit. on pp. 4, 5, 9, 34, 35, 36, 37, 38, 49).
- [21] Jonathan Viquerat et al. *Policy-based optimization: single-step policy gradient method seen as an evolution strategy*. 2021. DOI: 10.48550/ARXIV.2104.06175. URL: <https://arxiv.org/abs/2104.06175> (cit. on pp. 5, 22, 23, 38, 44, 47, 49, 50, 75).
- [22] Satinder Singh et al. “Convergence Results for Single-Step On-Policy Reinforcement-Learning Algorithms”. In: *Machine Learning* 38 (2004), pp. 287–308 (cit. on pp. 5, 38).
- [23] Hartmut H. K. Lentz et al. “Disease Spread through Animal Movements: A Static and Temporal Network Analysis of Pig Trade in Germany”. In: *PLOS ONE* 11.5 (May 2016), pp. 1–32. DOI: 10.1371/journal.pone.0155196. URL: <https://doi.org/10.1371/journal.pone.0155196> (cit. on pp. 5, 7, 8, 18, 19, 26, 39, 43).
- [24] Peter Battaglia et al. “Relational inductive biases, deep learning, and graph networks”. In: *arXiv* (2018). URL: <https://arxiv.org/pdf/1806.01261.pdf> (cit. on pp. 5, 9, 39, 40, 51, 52, 53, 54).
- [25] Jie Zhou et al. “Graph neural networks: A review of methods and applications”. In: *AI Open* 1 (2020), pp. 57–81 (cit. on pp. 5, 39, 40, 41).
- [26] Matthias Fey and Jan E. Lenssen. “Fast Graph Representation Learning with PyTorch Geometric”. In: *ICLR Workshop on Representation Learning on Graphs and Manifolds*. 2019 (cit. on pp. 5, 54).

Bibliography

- [27] Antonin Raffin et al. “Stable-Baselines3: Reliable Reinforcement Learning Implementations”. In: *Journal of Machine Learning Research* 22.268 (2021), pp. 1–8. URL: <http://jmlr.org/papers/v22/20-1364.html> (cit. on pp. 5, 55, 59, 74).
- [28] Bernard M Waxman. “Routing of multipoint connections”. In: *IEEE journal on selected areas in communications* 6.9 (1988), pp. 1617–1622 (cit. on pp. 5, 13).
- [29] Réka Albert and Albert-László Barabási. “Statistical mechanics of complex networks”. In: *Reviews of modern physics* 74.1 (2002), p. 47 (cit. on pp. 6, 13).
- [30] Sara Ansari et al. “A Temporal Network Model for Livestock Trade Systems”. In: *Frontiers in veterinary science* 8 (2022) (cit. on pp. 6, 7, 8, 10, 11, 12, 15, 16, 17, 18, 19, 20, 22, 24, 25, 26, 27, 28, 29, 30, 43, 44, 61, 62, 68, 69, 70, 73).
- [31] Sara Ansari et al. “Moving the epidemic tipping point through topologically targeted social distancing”. In: *The European Physical Journal Special Topics* 230.16-17 (June 2021), pp. 3273–3280. DOI: 10.1140/epjs/s11734-021-00138-5. URL: <https://doi.org/10.1140/epjs/s11734-021-00138-5> (cit. on p. 7).
- [32] Victor M. Preciado et al. “Optimal Resource Allocation for Network Protection Against Spreading Processes”. In: *IEEE Transactions on Control of Network Systems* 1.1 (2014), pp. 99–108. DOI: 10.1109/TCNS.2014.2310911 (cit. on pp. 7, 8).
- [33] Matt Keeling and Ken Eames. “Networks and Epidemic Models”. In: *Journal of the Royal Society, Interface / the Royal Society* 2 (Oct. 2005), pp. 295–307. DOI: 10.1098/rsif.2005.0051 (cit. on p. 7).
- [34] Caroline Dubé et al. “A Review of Network Analysis Terminology and its Application to Foot-and-Mouth Disease Modelling and Policy Development”. In: *Transboundary and emerging diseases* 56 (May 2009), pp. 73–85. DOI: 10.1111/j.1865-1682.2008.01064.x (cit. on p. 8).
- [35] Mathieu Andraud et al. *Modelling African swine fever virus spread in pigs using time-respective network data: scientific support for decision-makers*. 2021. DOI: 10.48550/ARXIV.2111.13412. URL: <https://arxiv.org/abs/2111.13412> (cit. on pp. 8, 26).
- [36] Nicolas Cardenas et al. “Multiple species animal movements: network properties, disease dynamics and the impact of targeted control actions”. In: *Veterinary Research* 53 (Dec. 2022). DOI: 10.1186/s13567-022-01031-2 (cit. on p. 8).

Bibliography

- [37] Karin Lebl et al. “Impact of Network Activity on the Spread of Infectious Diseases through the German Pig Trade Network”. In: *Frontiers in Veterinary Science* 3 (2016). ISSN: 2297-1769. DOI: 10.3389/fvets.2016.00048. URL: <https://www.frontiersin.org/articles/10.3389/fvets.2016.00048> (cit. on p. 8).
- [38] Quentin Cappart et al. *Combinatorial optimization and reasoning with graph neural networks*. 2021. DOI: 10.48550/ARXIV.2102.09544. URL: <https://arxiv.org/abs/2102.09544> (cit. on p. 9).
- [39] Davide Bacciu et al. “A gentle introduction to deep learning for graphs”. In: *Neural Networks* 129 (Sept. 2020), pp. 203–221. DOI: 10.1016/j.neunet.2020.06.006. URL: <https://doi.org/10.1016/j.neunet.2020.06.006> (cit. on p. 9).
- [40] Peter W. Battaglia et al. *Interaction Networks for Learning about Objects, Relations and Physics*. 2016. DOI: 10.48550/ARXIV.1612.00222. URL: <https://arxiv.org/abs/1612.00222> (cit. on p. 9).
- [41] Jessica B. Hamrick et al. *Relational inductive bias for physical construction in humans and machines*. 2018. DOI: 10.48550/ARXIV.1806.01203. URL: <https://arxiv.org/abs/1806.01203> (cit. on p. 9).
- [42] Tingwu Wang et al. “NerveNet: Learning Structured Policy with Graph Neural Networks”. In: *International Conference on Learning Representations*. 2018. URL: <https://openreview.net/forum?id=S1sqHMZCb> (cit. on p. 9).
- [43] Thomas D. Barrett et al. “Exploratory Combinatorial Optimization with Reinforcement Learning”. In: (2019). DOI: 10.48550/ARXIV.1909.04063. URL: <https://arxiv.org/abs/1909.04063> (cit. on p. 9).
- [44] Irwan Bello et al. *Neural Combinatorial Optimization with Reinforcement Learning*. 2016. DOI: 10.48550/ARXIV.1611.09940. URL: <https://arxiv.org/abs/1611.09940> (cit. on p. 9).
- [45] Alex Nowak et al. *Revised Note on Learning Algorithms for Quadratic Assignment with Graph Neural Networks*. 2017. DOI: 10.48550/ARXIV.1706.07450. URL: <https://arxiv.org/abs/1706.07450> (cit. on p. 9).
- [46] Hanjun Dai et al. *Learning Combinatorial Optimization Algorithms over Graphs*. 2017. DOI: 10.48550/ARXIV.1704.01665. URL: <https://arxiv.org/abs/1704.01665> (cit. on p. 9).

Bibliography

- [47] Thomas Kipf et al. *Neural Relational Inference for Interacting Systems*. 2018. DOI: 10.48550/ARXIV.1802.04687. URL: <https://arxiv.org/abs/1802.04687> (cit. on p. 9).
- [48] William L. Hamilton, Rex Ying, and Jure Leskovec. *Inductive Representation Learning on Large Graphs*. 2017. DOI: 10.48550/ARXIV.1706.02216. URL: <https://arxiv.org/abs/1706.02216> (cit. on p. 9).
- [49] Arthur L. Samuel. "Some Studies in Machine Learning Using the Game of Checkers". In: *IBM Journal of Research and Development* 3.3 (July 1959), pp. 210–229 (cit. on p. 9).
- [50] Volodymyr Mnih et al. *Playing Atari with Deep Reinforcement Learning*. 2013. DOI: 10.48550/ARXIV.1312.5602. URL: <https://arxiv.org/abs/1312.5602> (cit. on p. 9).
- [51] Julian Schrittwieser et al. "Mastering Atari, Go, chess and shogi by planning with a learned model". In: *Nature* 588.7839 (Dec. 2020), pp. 604–609. DOI: 10.1038/s41586-020-03051-4 (cit. on p. 9).
- [52] Xinwei Wei et al. "Deep Reinforcement Learning-based Vaccine Distribution Strategies". In: *2021 2nd International Conference on Electronics, Communications and Information Technology (CECIT)*. 2021, pp. 427–436. DOI: 10.1109/CECIT53797.2021.00082 (cit. on p. 9).
- [53] Harshad Khadilkar, Tanuja Ganu, and Deva P Seetharam. *Optimising Lockdown Policies for Epidemic Control using Reinforcement Learning*. 2020. DOI: 10.48550/ARXIV.2003.14093. URL: <https://arxiv.org/abs/2003.14093> (cit. on p. 9).
- [54] Pieter Libin et al. *Deep reinforcement learning for large-scale epidemic control*. 2020. DOI: 10.48550/ARXIV.2003.13676. URL: <https://arxiv.org/abs/2003.13676> (cit. on p. 9).
- [55] Hanrui Wang et al. "GCN-RL Circuit Designer: Transferable Transistor Sizing with Graph Neural Networks and Reinforcement Learning". In: *2020 57th ACM/IEEE Design Automation Conference (DAC)*. 2020, pp. 1–6. DOI: 10.1109/DAC18072.2020.9218757 (cit. on p. 9).
- [56] Paul Almasan et al. "Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case". In: *Computer Communications* 196 (2022), pp. 184–194. ISSN: 0140-3664. DOI: <https://doi.org/10.1016/j.comcom.2022.09.029>. URL: <https://www.sciencedirect.com/science/article/pii/S0140366422003784> (cit. on p. 9).

Bibliography

- [57] Bo Chen et al. “An Approach to Combine the Power of Deep Reinforcement Learning with a Graph Neural Network for Routing Optimization”. In: *Electronics* 11 (Jan. 2022), p. 368. DOI: 10.3390/electronics11030368 (cit. on p. 10).
- [58] Jonathan Viquerat et al. *Direct shape optimization through deep reinforcement learning*. 2019. DOI: 10.48550/ARXIV.1908.09885. URL: <https://arxiv.org/abs/1908.09885> (cit. on p. 10).
- [59] Elie Hachem et al. *Deep reinforcement learning for the control of conjugate heat transfer with application to workpiece cooling*. 2020. DOI: 10.48550/ARXIV.2011.15035. URL: <https://arxiv.org/abs/2011.15035> (cit. on p. 10).
- [60] M.Y. Li. *An Introduction to Mathematical Modeling of Infectious Diseases*. Mathematics of Planet Earth. Springer International Publishing, 2018. ISBN: 9783319721224. URL: <https://books.google.de/books?id=7PJJDwAAQBAJ> (cit. on pp. 15, 18).
- [61] “The Milroy Lectures ON EPIDEMIC DISEASE IN ENGLAND—THE EVIDENCE OF VARIABILITY AND OF PERSISTENCY OF TYPE.” In: *The Lancet* 167.4307 (1906). Originally published as Volume 1, Issue 4307, pp. 733–739. ISSN: 0140-6736. DOI: [https://doi.org/10.1016/S0140-6736\(01\)80340-8](https://doi.org/10.1016/S0140-6736(01)80340-8). URL: <https://www.sciencedirect.com/science/article/pii/S0140673601803408> (cit. on p. 15).
- [62] Ronald Ross. “The mathematics of malaria”. In: *British medical journal* 1.2626 (1911), p. 1023 (cit. on p. 15).
- [63] Frank Ball and Peter Neal. “Network epidemic models with two levels of mixing”. In: *Mathematical biosciences* 212.1 (2008), pp. 69–87 (cit. on p. 18).
- [64] Sushant S Garud, Nivethitha Mariappan, and Iftekhar A Karimi. “Surrogate-based black-box optimisation via domain exploration and smart placement”. In: *Computers & Chemical Engineering* 130 (2019), p. 106567 (cit. on p. 21).
- [65] Thomas Bäck. *Evolutionary Algorithms in Theory and Practice: Evolution Strategies, Evolutionary Programming, Genetic Algorithms*. Oxford University Press, Feb. 1996. ISBN: 9780195099713. DOI: 10.1093/oso/9780195099713.001.0001. URL: <https://doi.org/10.1093/oso/9780195099713.001.0001> (cit. on p. 23).
- [66] H. Beyer. “Evolution strategies”. In: *Scholarpedia* 2.8 (2007). revision #193589, p. 1965. DOI: 10.4249/scholarpedia.1965 (cit. on p. 23).
- [67] Christophe Andrieu et al. “An introduction to MCMC for machine learning”. In: *Machine learning* 50.1 (2003), pp. 5–43 (cit. on p. 24).

Bibliography

- [68] R.J.N.J. Bellman. "Dynamic programming princeton university press princeton". In: *New Jersey Google Scholar* (1957) (cit. on pp. 24, 73).
- [69] Frances Y Kuo and Ian H Sloan. "Lifting the curse of dimensionality". In: *Notices of the AMS* 52.11 (2005), pp. 1320–1328 (cit. on p. 24).
- [70] S Kirkpatrick, C Gelatt, and M Vecchi. "Simulated annealing methods". In: *J. Stat. Phys* 34 (1984), p. 975 (cit. on p. 26).
- [71] Vladimír Černý. "Thermodynamical approach to the traveling salesman problem: An efficient simulation algorithm". In: *Journal of optimization theory and applications* 45.1 (1985), pp. 41–51 (cit. on p. 26).
- [72] Vincent D Blondel et al. "Fast unfolding of communities in large networks". In: *Journal of statistical mechanics: theory and experiment* 2008.10 (2008), P10008 (cit. on p. 29).
- [73] Augustin-Louis Cauchy. "Mémoire sur les quantités géométriques". In: *Œuvres Complètes* (1847), pp. 175–202 (cit. on p. 31).
- [74] Jacques Hadamard. "Sur quelques questions du calcul des variations". In: *Annales scientifiques de l'école normale supérieure*. Vol. 24. 1907, pp. 203–231 (cit. on p. 31).
- [75] Warren S McCulloch and Walter Pitts. "A logical calculus of the ideas immanent in nervous activity". In: *The bulletin of mathematical biophysics* 5.4 (1943), pp. 115–133 (cit. on p. 31).
- [76] Ben Kröse et al. "An introduction to neural networks". In: *J Comput Sci* 48 (Jan. 1993) (cit. on pp. 31, 32).
- [77] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA, USA: MIT Press, 1969 (cit. on p. 31).
- [78] Seppo Linnainmaa. "The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors". PhD thesis. Master's Thesis (in Finnish), Univ. Helsinki, 1970 (cit. on p. 31).
- [79] Paul Werbos. "Beyond regression:" new tools for prediction and analysis in the behavioral sciences". In: *Ph. D. dissertation, Harvard University* (1974) (cit. on p. 31).
- [80] Paul J Werbos. "Generalization of backpropagation with application to a recurrent gas market model". In: *Neural networks* 1.4 (1988), pp. 339–356 (cit. on p. 31).

Bibliography

- [81] Raúl Rojas. *Neural networks: a systematic introduction*. Springer Science & Business Media, 2013 (cit. on pp. 31, 32, 33, 34, 35).
- [82] Uri Hasson, Samuel A. Nastase, and Ariel Goldstein. “Direct Fit to Nature: An Evolutionary Perspective on Biological and Artificial Neural Networks”. In: *Neuron* 105.3 (2020), pp. 416–434. ISSN: 0896-6273. DOI: <https://doi.org/10.1016/j.neuron.2019.12.002>. URL: <https://www.sciencedirect.com/science/article/pii/S089662731931044X> (cit. on p. 31).
- [83] Jeremy Watt, Reza Borhani, and Aggelos K. Katsaggelos. *Machine Learning Refined: Foundations, Algorithms, and Applications*. 2nd ed. Cambridge University Press, 2020. DOI: 10.1017/9781108690935 (cit. on p. 33).
- [84] Wojciech Samek et al. “Explaining deep neural networks and beyond: A review of methods and applications”. In: *Proceedings of the IEEE* 109.3 (2021), pp. 247–278 (cit. on p. 34).
- [85] Yong Yu et al. “A review of recurrent neural networks: LSTM cells and network architectures”. In: *Neural computation* 31.7 (2019), pp. 1235–1270 (cit. on p. 34).
- [86] Feng Ren et al. *AlphaFold Accelerates Artificial Intelligence Powered Drug Discovery: Efficient Discovery of a Novel Cyclin-dependent Kinase 20 (CDK20) Small Molecule Inhibitor*. 2022. DOI: 10.48550/ARXIV.2201.09647. URL: <https://arxiv.org/abs/2201.09647> (cit. on p. 35).
- [87] John Jumper et al. “Highly accurate protein structure prediction with AlphaFold”. In: *Nature* 596 (Aug. 2021), pp. 1–11. DOI: 10.1038/s41586-021-03819-2 (cit. on p. 35).
- [88] Richard Evans et al. “Protein complex prediction with AlphaFold-Multimer”. In: *bioRxiv* (2022). DOI: 10.1101/2021.10.04.463034. eprint: <https://www.biorxiv.org/content/early/2022/03/10/2021.10.04.463034.full.pdf>. URL: <https://www.biorxiv.org/content/early/2022/03/10/2021.10.04.463034> (cit. on p. 35).
- [89] Mihaly Varadi et al. “AlphaFold Protein Structure Database: massively expanding the structural coverage of protein-sequence space with high-accuracy models”. In: *Nucleic Acids Research* 50.D1 (Nov. 2021), pp. D439–D444. ISSN: 0305-1048. DOI: 10.1093/nar/gkab1061. eprint: <https://academic.oup.com/nar/article-pdf/50/D1/D439/43502749/gkab1061.pdf>. URL: <https://doi.org/10.1093/nar/gkab1061> (cit. on p. 35).

Bibliography

- [90] John Schulman et al. “Proximal policy optimization algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017) (cit. on pp. 44, 47, 48, 49).
- [91] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2014. DOI: 10.48550/ARXIV.1412.6980. URL: <https://arxiv.org/abs/1412.6980> (cit. on p. 60).
- [92] Alfréd Rényi et al. “On measures of entropy and information”. In: *Proceedings of the fourth Berkeley symposium on mathematical statistics and probability*. Vol. 1. 547-561. Berkeley, California, USA. 1961 (cit. on p. 64).
- [93] N.R. Pal and S.K. Pal. “Entropy: a new definition and its applications”. In: *IEEE Transactions on Systems, Man, and Cybernetics* 21.5 (1991), pp. 1260–1270. DOI: 10.1109/21.120079 (cit. on p. 64).
- [94] Jacob Benesty et al. “Pearson correlation coefficient”. In: *Noise reduction in speech processing*. Springer, 2009, pp. 1–4 (cit. on p. 64).

A. Appendix

A.1. The degree distributions of the transmission networks

The degree distributions for AB, SYN and WX network topologies with 120 and 1040 nodes are shown in A.2, A.3 and in A.4.

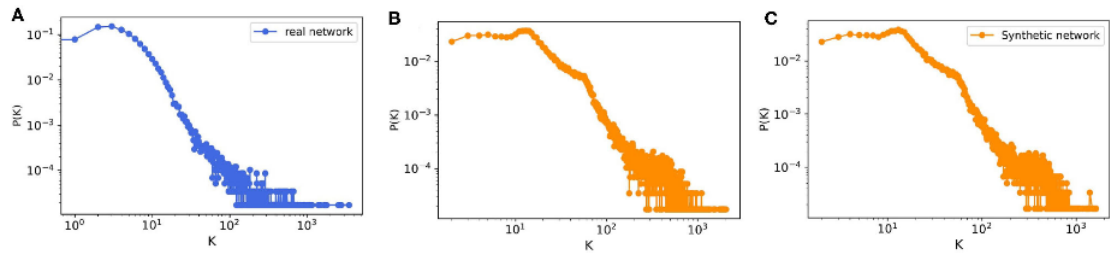


Figure A.1.: The degree distribution of (A) real pig trade network with $n = 57,590$, (B) one realization of a synthetic data with $n = 57,590$, and (C) ensemble of 10 synthetically generated networks with the same size from the model. In Ansari et al. [1], Figure 8.

A. Appendix

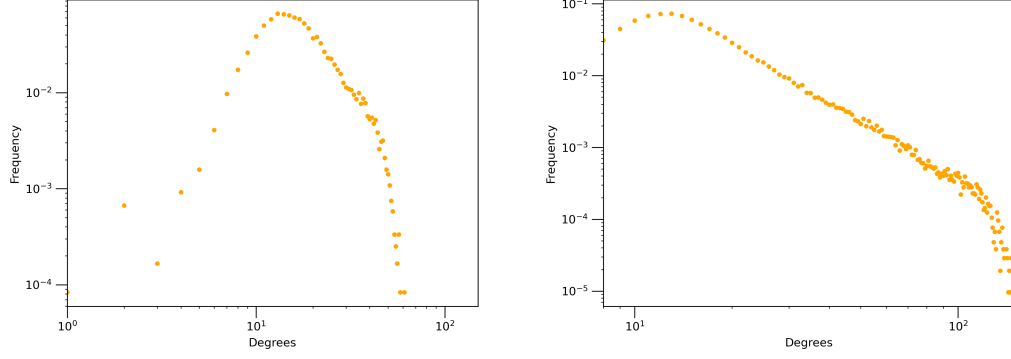


Figure A.2.: The degree distribution over 100 AB networks with 120 (left) and over 100 AB networks with 1040 nodes (right).

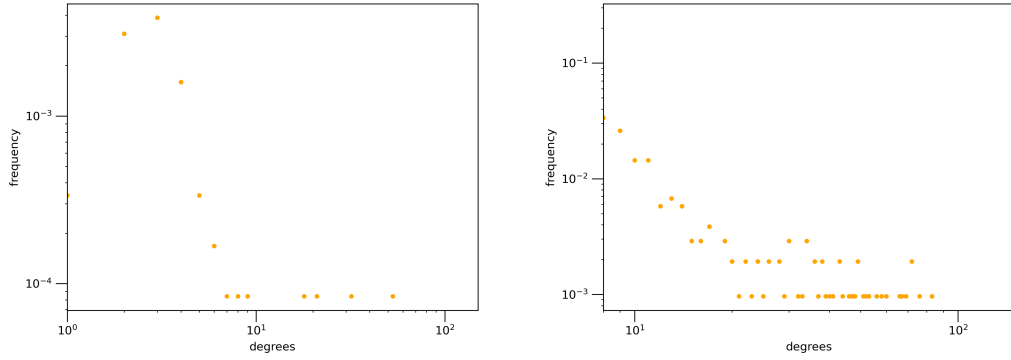


Figure A.3.: The degree distribution over 100 SYN networks with 120 (left) and over 100 SYN networks with 1040 (right) nodes.

A. Appendix

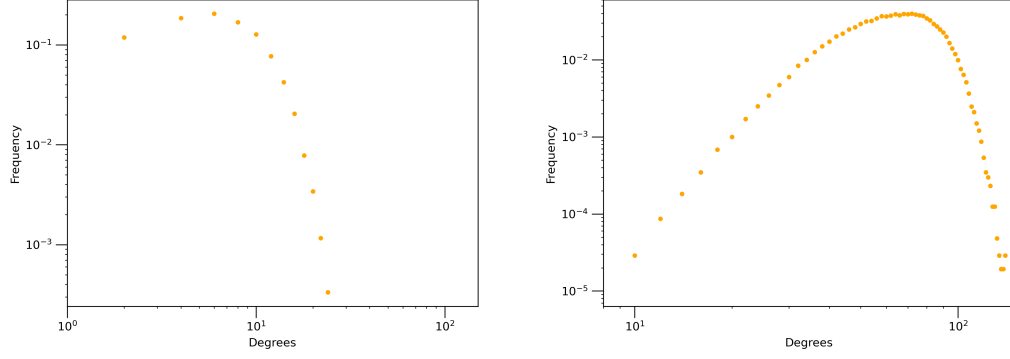


Figure A.4.: The degree distribution of 100 WX networks with 120 (left) and 100 WX networks with 1040 (right) nodes.

The differences in terms of degree distribution, capacity and number of transmission for both network sizes are shown in pair plots in Figure A.5.

A. Appendix

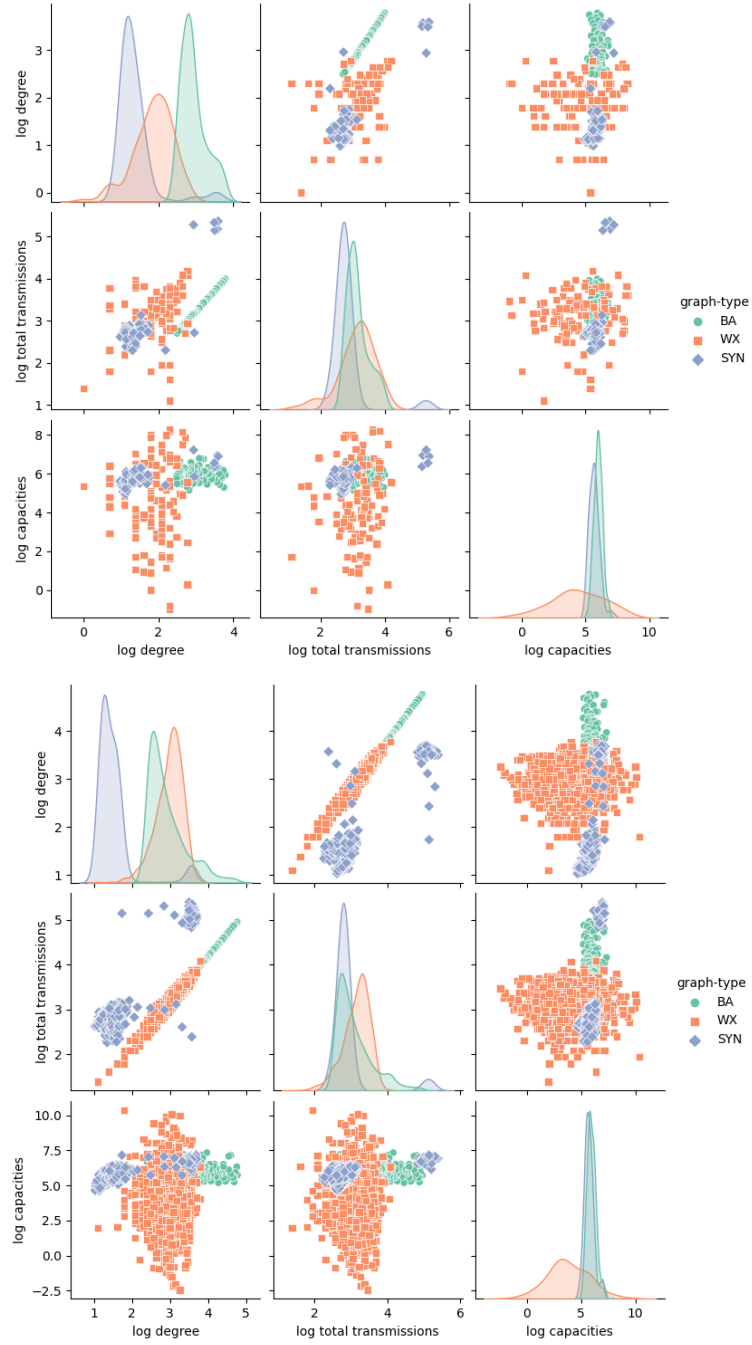


Figure A.5.: Pair plots including data from all generated networks showing the differences in the distribution of degree, the capacities and the number of total transmissions. Top: All networks with 120 nodes. Bottom: All networks with 1040 nodes.

A. Appendix

A.2. The learning progress during optimisation

The learning progress in terms of the mean damage is shown in Figures A.6–A.8 for all network types and network sizes.

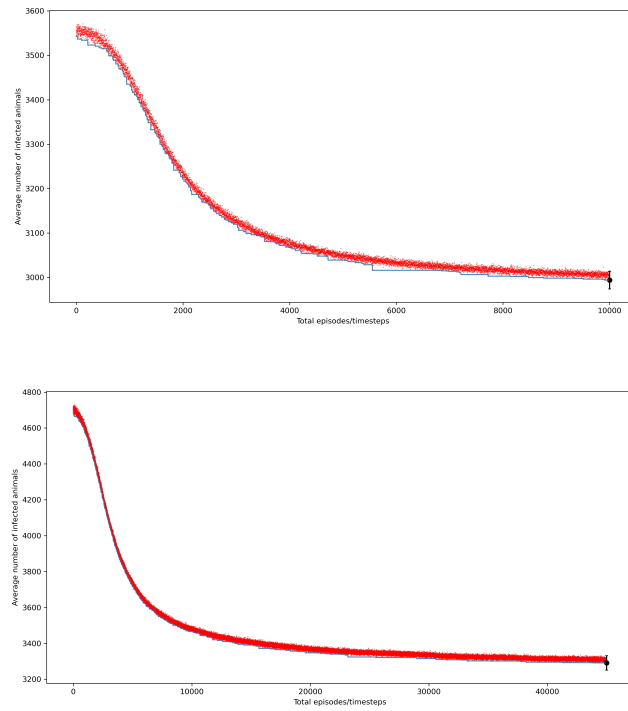


Figure A.6.: The change in the estimated average number of infected animals over time over 100 BA networks with 120 nodes and 1040 nodes.

A. Appendix

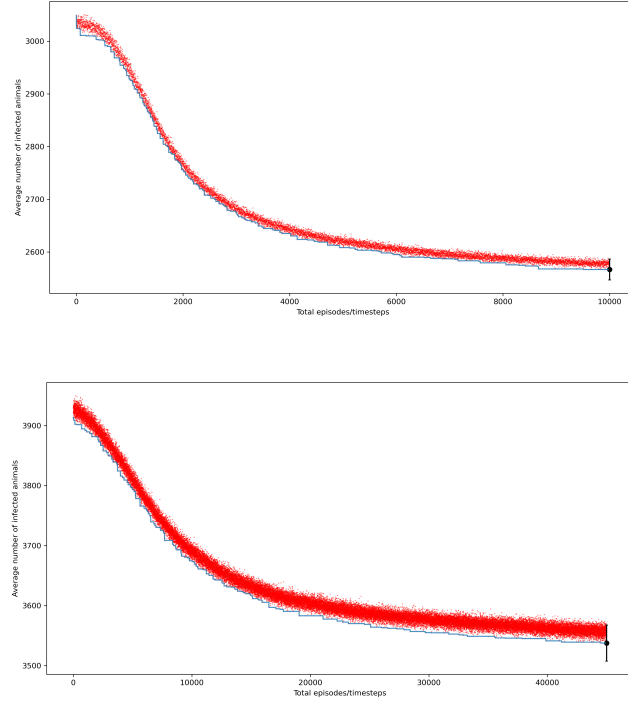


Figure A.7.: The change in the estimated average number of infected animals over time over 100 WX networks with 120 nodes and 1040 nodes.

A. Appendix

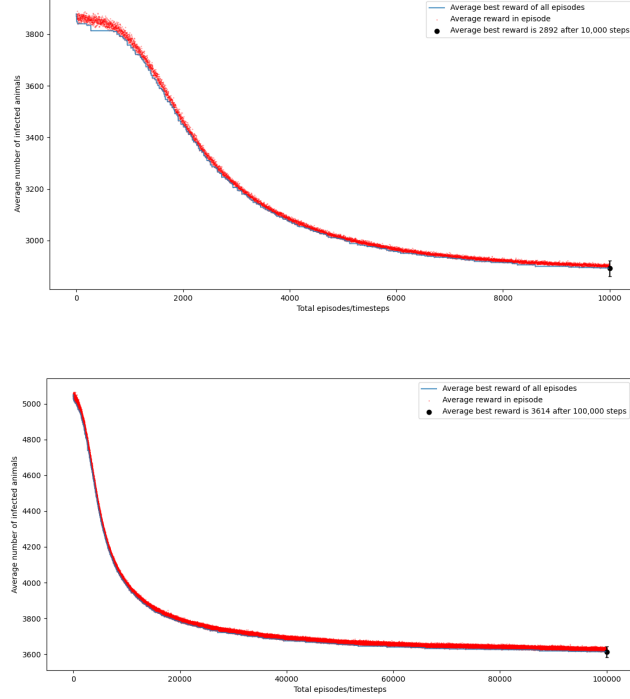


Figure A.8.: The change in the estimated average number of infected animals over time over 100 SYN networks with 120 nodes and 1040 nodes.

A.3. The mean damage for “naive” heuristics for all network types and network sizes

For both the transmission networks with 120 and 1040 nodes, the transmission network types WX, AB, and SYN and the total budgets $B = N, B = N/2, B = N/4$, the testing strategies HD and HIT perform best compared to the testing strategies HC and UNI given that the total budget is the **same** for all testing strategies. If we compare $HD_{S_{B=N/2}, B=N/4}$ and $HIT_{S_{B=N/2}, B=N/4}$ that only dispose **portions** of the baseline total budget to the baseline testing strategy $UNI_{S_{B=N}}$, limited total budgets of $B = N/2$, and $B = N/4$ are sufficient enough to reduce the mean damage at least to a similar value like the baseline testing strategy $UNI_{S_{B=N}}$ given the more centralised network types AB (1040) and SYN (120, 1040). It also appears that if we consider AB and SYN networks, the larger the network size

A. Appendix

the more effective the testing strategies HD and HIT perform compared to UNI. Hence, both testing schemes HD and HIT may be most beneficial for networks that are similar to SYN and large BA networks in terms of centralisation, modularization, and degree distribution.

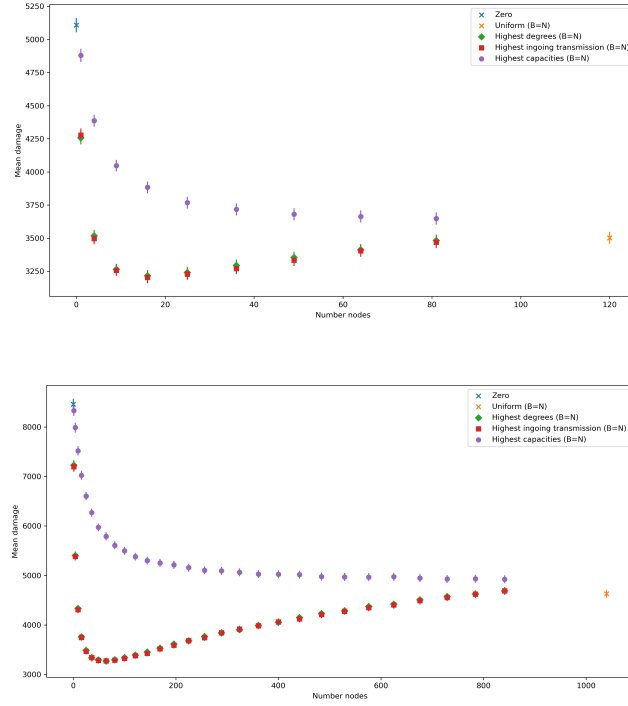


Figure A.9.: The mean damage for the “naive” test strategies over all AB transmission networks with 120 nodes (right) and 1040 (left) nodes if $S_{B=N}$. The test strategies $HD_{S_{B=N}}$ and $HIT_{S_{B=N}}$ are more effective than the test strategies $UNI_{S_{B=N}}$ and $HC_{S_{B=N}}$.

A. Appendix

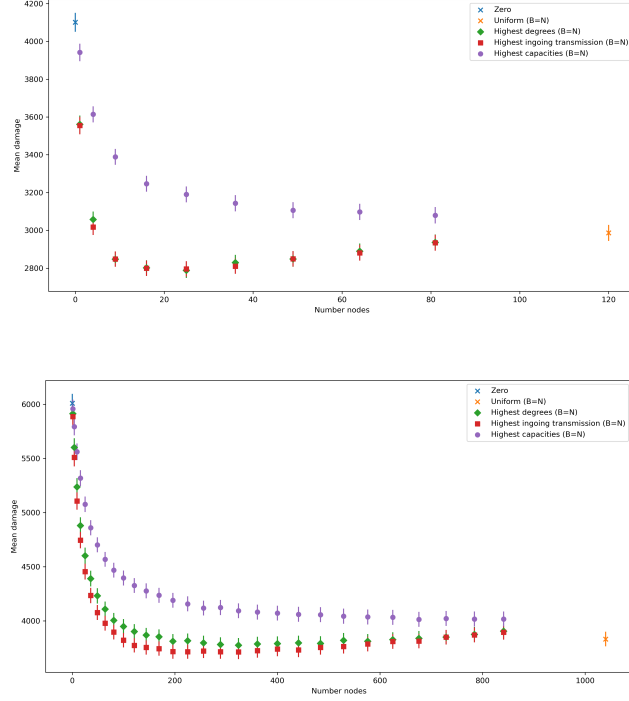


Figure A.10.: The mean damage for the “naive” test strategies over all WX transmission networks with 120 nodes (right) and 1040 nodes (left) if $S_{B=N}$. The test strategies $HD_{S_{B=N}}$ and $HIT_{S_{B=N}}$ are more effective compared to the test strategies $UNI_{S_{B=N}}$ and $HC_{S_{B=N}}$.

A. Appendix

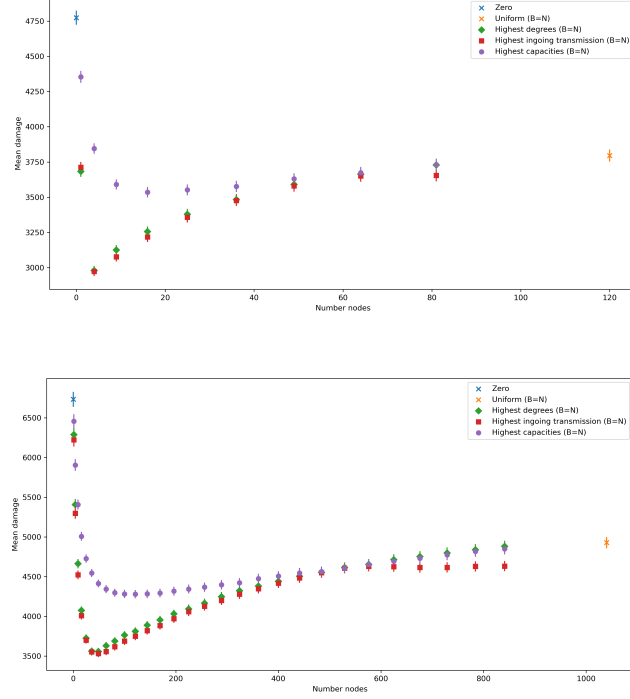


Figure A.11.: The mean damage for the “naive” test strategies over all SYN transmission networks with 120 nodes (right) and 1040 nodes (left) if $S_{B=N}$. The test strategies $HD_{S_{B=N}}$, $HIT_{S_{B=N}}$ and $HC_{S_{B=N}}$ are more effective compared to the test strategy $UNI_{S_{B=N}}$.

A. Appendix

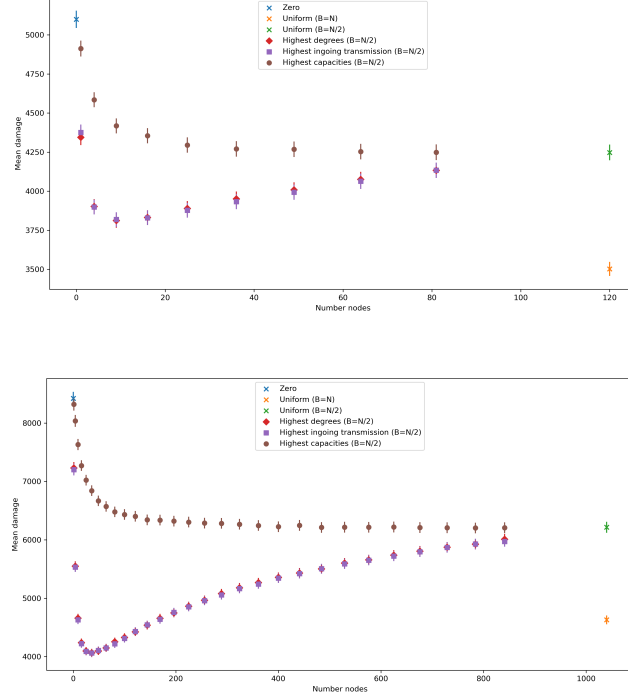


Figure A.12.: The mean damage for the “naive” test strategies over all AB transmission networks with 120 nodes (right) and 1040 nodes (left) if $S_{B=N/2}$. The test strategies $HD_{S_{B=N/2}}$ and $HIT_{S_{B=N/2}}$ are more effective compared to the test strategy $UNI_{S_{B=N/2}}$ and $HC_{S_{B=N/2}}$. However, $HD_{S_{B=N/2}}$ and $HIT_{S_{B=N/2}}$ perform worse than $UNI_{S_{B=N}}$.

A. Appendix

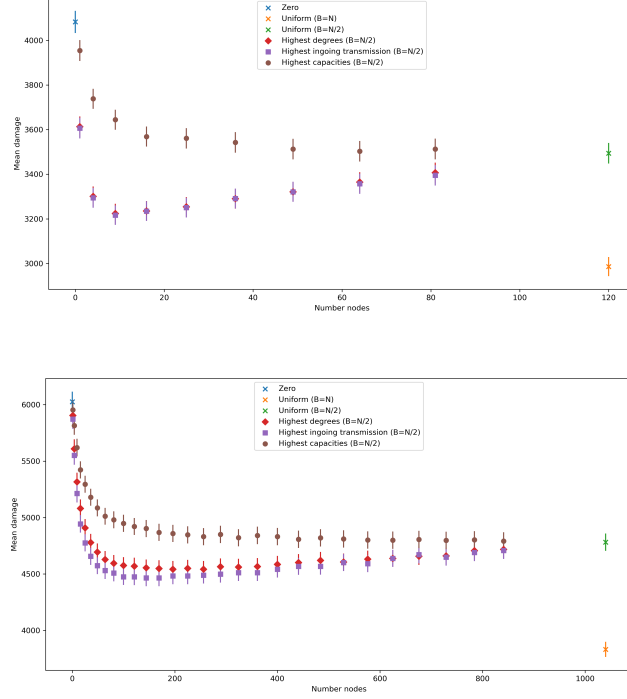


Figure A.13.: The mean damage for the “naive” test strategies over all WX transmission networks with 120 nodes (right) and 1040 nodes (left) if $S_{B=N/2}$. The test strategies $HD_{S_{B=N/2}}$ and $HIT_{S_{B=N/2}}$ are more effective compared to the test strategy $UNI_{S_{B=N/2}}$ and $HC_{S_{B=N/2}}$. However, $HD_{S_{B=N/2}}$ and $HIT_{S_{B=N/2}}$ perform worse than $UNI_{S_{B=N}}$.

A. Appendix

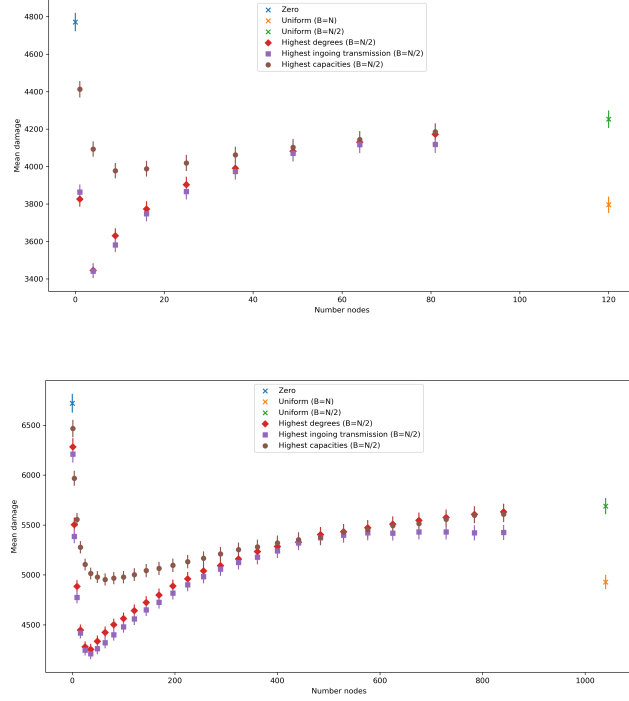


Figure A.14.: The mean damage for the “naive” test strategies over all SYN transmission networks with 120 nodes (right) and 1040 nodes (left) if $S_{B=N/2}$. The test strategies $HD_{S_{B=N/2}}$ and $HIT_{S_{B=N/2}}$ are more effective compared to the test strategy $UNI_{S_{B=N}}$, $UNI_{S_{B=N/2}}$ and $HC_{S_{B=N/2}}$.

A. Appendix

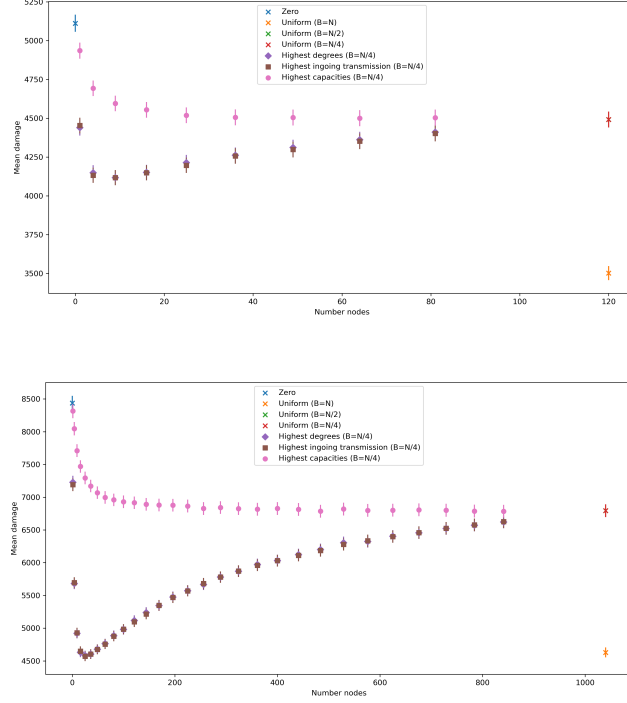


Figure A.15.: The mean damage for the “naive” test strategies over all AB transmission networks with 120 nodes (right) and 1040 nodes (left) if $S_{B=N/4}$. The test strategies $HD_{S_{B=N/4}}$ and $HIT_{S_{B=N/4}}$ are more effective compared to the test strategy $UNI_{S_{B=N/4}}$, $UNI_{S_{B=N/2}}$ and $HC_{S_{B=N/4}}$. However, $HD_{S_{B=N/4}}$ and $HIT_{S_{B=N/4}}$ perform worse than $UNI_{S_{B=N}}$.

A. Appendix

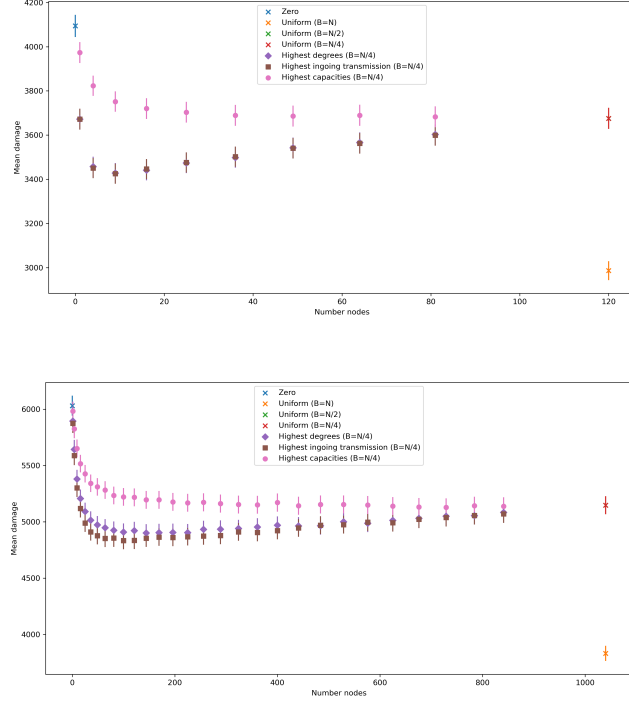


Figure A.16.: The mean damage for the “naive” test strategies over all WX transmission networks with 120 nodes (right) and 1040 nodes (left) if $S_{B=N/4}$. The test strategies $HD_{S_{B=N/4}}$ and $HIT_{S_{B=N/4}}$ are more effective compared to the test strategy $UNI_{S_{B=N/4}}$, $UNI_{S_{B=N/2}}$ and $HC_{S_{B=N/4}}$. However, $HD_{S_{B=N/4}}$ and $HIT_{S_{B=N/4}}$ perform worse than $UNI_{S_{B=N}}$.

A. Appendix

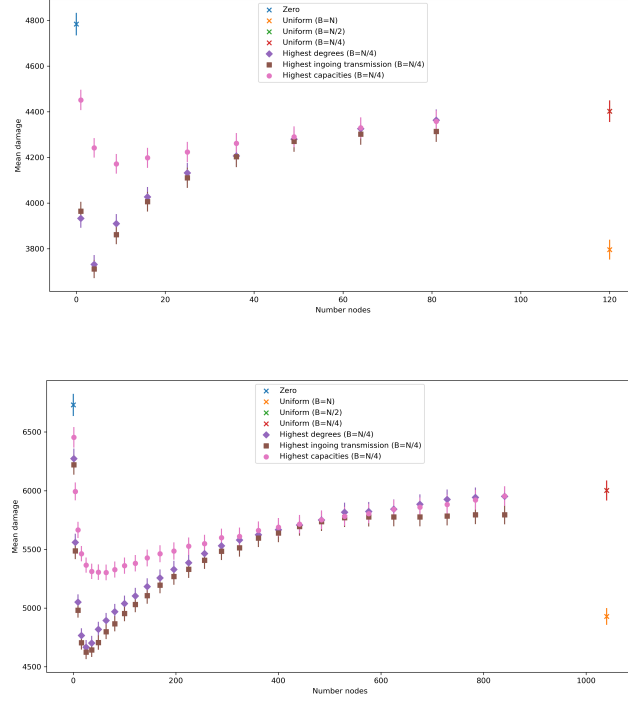


Figure A.17.: The mean damage for the “naive” test strategies over all SYN transmission networks with 120 nodes (right) and 1040 nodes (left) if $S_{B=N/4}$. The test strategies $HD_{S_{B=N/4}}$ and $HIT_{S_{B=N/4}}$ are more effective compared to the test strategies $UNI_{S_{B=N/4}}$, $UNI_{S_{B=N/2}}$, $UNI_{S_{B=N}}$ and $HC_{S_{B=N/4}}$.

A. Appendix

A.4. The mean damage for “naive” test strategies in comparison to optimal strategies

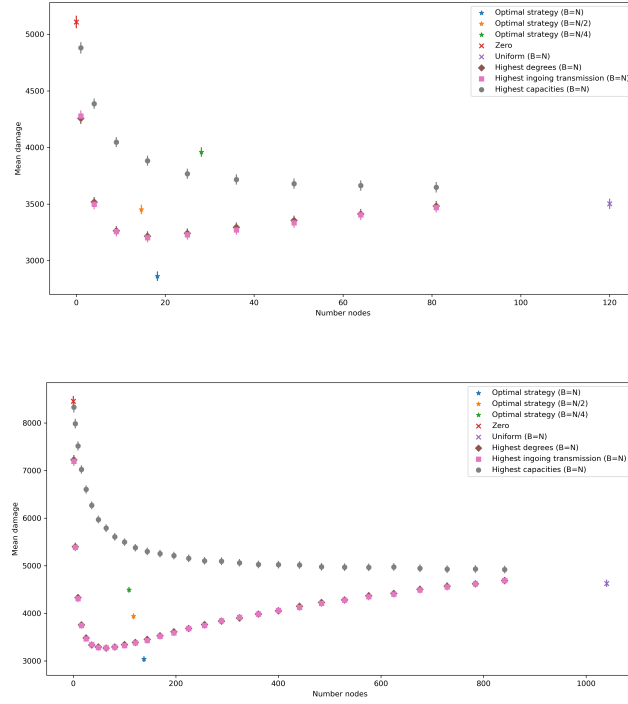


Figure A.18.: The mean damage for “naive” test strategies and optimal test strategies with three different total test budgets over all AB transmission networks with 120 nodes.

A. Appendix

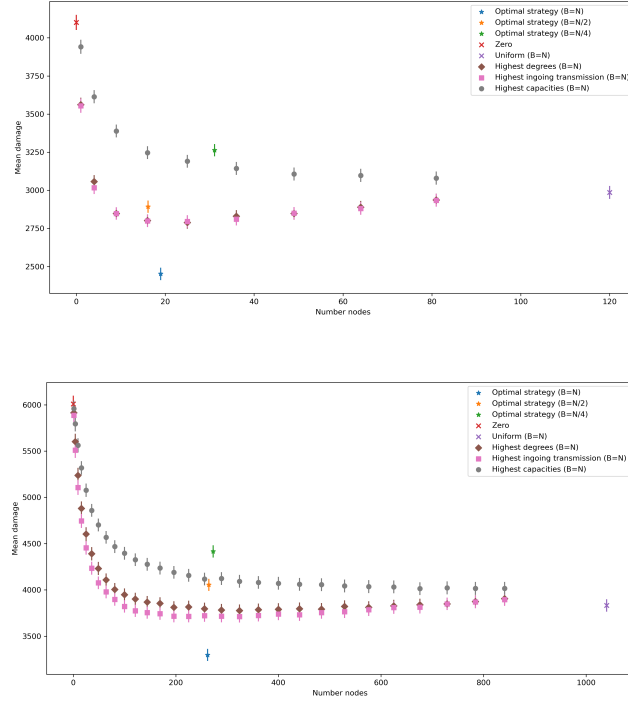


Figure A.19.: The mean damage for “naive” test strategies and optimal test strategies with three different total test budgets over all WX transmission networks with 120 nodes.

A. Appendix

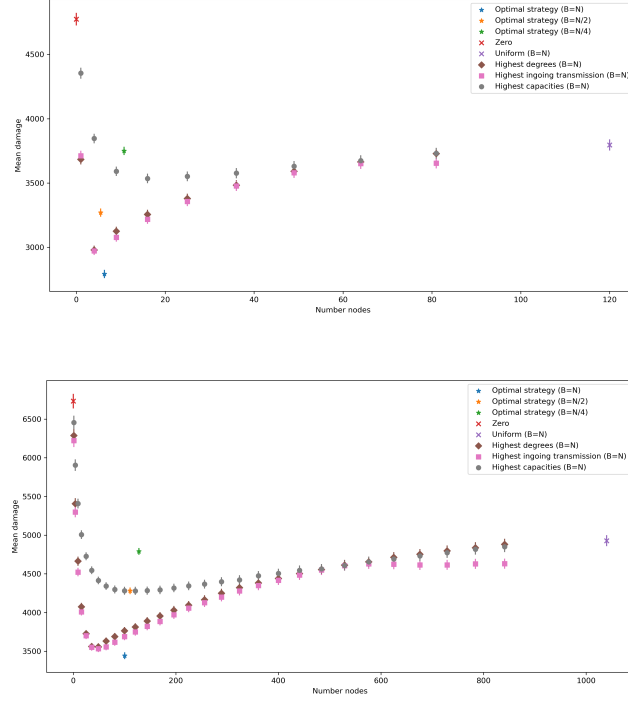


Figure A.20.: The mean damage for the “naive” test strategies and the optimal strategies with three different total test budgets over all SYN transmission networks with 120 nodes.